

Operations Research 1

[Notes by Piai Luca]

Academic Year 2024-2025

Contents

1	Mathematical Programming	1
1.1	Convex optimization	1
1.2	Linear Programming	4
1.2.1	Example	5
2	Simplex Algorithm	9
2.1	Geometry of Linear Programming	9
2.1.1	Vertices and basic solutions	10
2.2	The Simplex Method	15
2.2.1	Optimality test	15
2.2.2	Change of Basis	16
2.2.3	Tableau Form of the Simplex Method	19
2.2.4	The two phases method	22
2.2.5	Convergence and Degeneracy	24
2.3	Revised Simplex Method	28
3	Duality in linear programming	31
3.1	Valid inequalities	31
3.2	Dual problem	33
3.2.1	Fundamental Properties	35
3.3	Optimality Conditions	37
3.4	Analysis of the Simplex Algorithm	38
3.5	Dual Simplex Algorithm	40
3.5.1	Dual simplex vs primal simplex	42
3.6	Sensitivity analysis	43
4	Integer Linear Programming	49
4.1	Equivalent Formulations	50
4.2	Total Unimodularity	52

4.3	Cutting Plain Algorithm	56
4.3.1	Chvátal's Inequalities	57
4.3.2	Gomory's Cuts	59
4.4	Branch-and-Bound	61
4.5	Branch-and-Cut	64
5	Graph Theory	67
5.1	Undirected Graphs	67
5.2	Directed Graphs	70
5.3	Graph Representation	71
5.4	Minimum spanning tree	73
5.4.1	Prim-Dijkstra's Algorithm	74
5.4.2	Kruskal's Algorithm	77
5.5	Path in a directed graph	79
5.5.1	Reachability	79
5.5.2	Shortest Path Problem	80
5.5.3	Dijkstra's Algorithm	82
5.5.4	Floyd-Warshall's Algorithm	83
5.5.5	Project Planning	86
5.6	Flow problems	89
5.6.1	Fundamental Properties	91
5.6.2	Ford-Fulkerson Algorithm for the MAX-FLOW problem	95
6	NP-hard Problems	97
6.1	Knapsack Problem	97
6.1.1	Solving the Knapsack B&B + Dantzig solution	98
6.2	TSP	99
6.3	Steiner Tree Problem	101
6.4	Plant/Facilities Location Problem	102
6.5	Set Covering/Partitioning Problem	104

Chapter 1

Mathematical Programming

We define a Mathematical Programming problem as follows:

$$\begin{cases} \min f(x) \\ x \in X \end{cases} \quad (1.1)$$

where $f(x)$ is the *objective function*, $X \subseteq \mathbb{R}^n$ is the *set of feasible solutions*, $x \in X$ is a *feasible solution*.

We call x^* optimal solution if and only if:

1. $x^* \in X$;
2. $f(x^*) \leq f(x) \quad \forall x \in X$.

Solving a Mathematical Programming problem means find an optimal solution and prove its optimality or prove that the problem is either *infeasible* ($X = \emptyset$) or *unbounded* ($f \rightarrow -\infty$).

We call $\tilde{x} \in X$ *locally optimal* if $\exists \varepsilon \geq 0$ such that $f(\tilde{x}) \leq f(x) \forall x \in X$ with $\|x - \tilde{x}\| \leq \varepsilon$. Also we call $I_\varepsilon(\tilde{x}) := \{x \in X : \|x - \tilde{x}\| \leq \varepsilon\}$ the neighborhood of $\tilde{x}$.

1.1 Convex optimization

Definition 1 (Convex combination). Given $x, y \in \mathbb{R}^n$, the point $z = \lambda x + (1 - \lambda)y$ is said to be a convex combination of x, y for all $\lambda \in [0, 1]$. If $0 < \lambda < 1$ the combination is said to be strict.

More generally, the convex combination of k points $x^1, \dots, x^k \in \mathbb{R}^n$ is defined as $z := \sum_{i=1}^k \lambda_i x^i$, with $\lambda_1, \dots, \lambda_k \geq 0$ and $\sum_{i=1}^k \lambda_i = 1$.

Definition 2 (Convex set). A set $X \subseteq \mathbb{R}^n$ is said to be convex if $\forall x, y \in X$ we have that X contains all the convex combination of x and y , i.e.:

$$z := [\lambda x + (1 - \lambda)y] \in X, \quad \forall \lambda \in [0, 1]$$

Proposition 1. The intersection of two convex sets $A, B \subseteq \mathbb{R}^n$ is still a convex set.

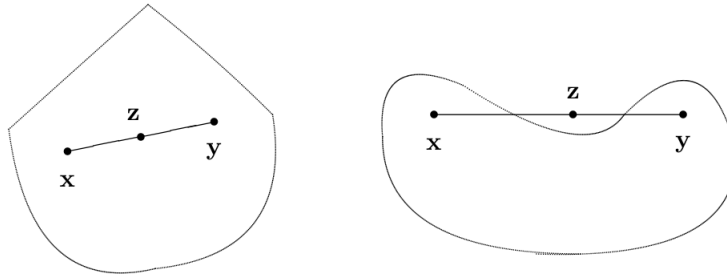


Figure 1.1: A convex set (on the left) and a non-convex set (on the right).

Definition 3 (Convex function). A function $f : X \rightarrow \mathbb{R}$ defined on a convex set $X \subseteq \mathbb{R}^n$ is said to be convex if $\forall x, y \in X$ and $\forall \lambda \in [0, 1]$ we have that

$$f(z) \leq \lambda f(x) + (1 - \lambda)f(y) \quad \text{where} \quad z = \lambda x + (1 - \lambda)y.$$

Note that if f is a linear function then it holds that $f(z) = \lambda f(x) + (1 - \lambda)f(y)$.

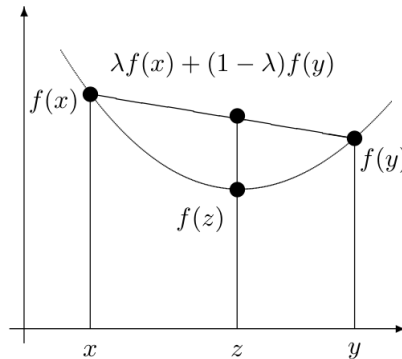


Figure 1.2: A convex function.

Theorem 1. *Let*

$$X = \{x \in \mathbb{R}^n : g_i(x) \leq 0, i = 1, \dots, m\}.$$

If for all $i \in \{1, \dots, m\}$ the functions $g_i : \mathbb{R}^n \rightarrow \mathbb{R}$ are convex, then the set X is convex.

Proof. Let

$$X = \bigcap_{i=1}^m X_i \quad \text{where} \quad X_i = \{x \in \mathbb{R}^n : g_i(x) \leq 0\}$$

Now all we need to do is demonstrate that X_i is a convex set. We can consider any $x, y \in X_i$ and we define $z = \lambda x + (1 - \lambda)y$ for any $\lambda \in [0, 1]$. We have

$$g_i(z) \stackrel{\text{by convexity}}{\leq} \lambda \underbrace{g_i(x)}_{\leq 0 \text{ because } x \in X_i} + (1 - \lambda) \underbrace{g_i(y)}_{\leq 0 \text{ because } y \in X_i}$$

so

$$\Rightarrow g_i(z) \leq 0 \Rightarrow z \in X_i \Rightarrow X_i \text{ is convex} \Rightarrow X \text{ is convex.}$$

□

We define a **convex problem** as follows:

$$\begin{cases} \min_{x \in X} f(x) \\ x \in X \end{cases} \quad (1.2)$$

where X is a convex set and f is a convex function.

Theorem 2 (Fundamental theorem of convex optimization). *Every locally optimal solution is also globally optimal.*

Proof. Let $\tilde{x}$ any locally optimal solution. We want to demonstrate that $f(\tilde{x}) \leq f(y)$ for every $y \in X$. By definition of locally optimal

$$\exists \varepsilon > 0 : f(\tilde{x}) \leq f(z) \quad \forall z \in I_\varepsilon(\tilde{x}).$$

Take any $y \in X$ and define $z = \lambda \tilde{x} + (1 - \lambda)y$ where $\lambda < 1$ is very close to 1 so that $z \in I_\varepsilon(\tilde{x})$.

We have

$$\begin{aligned} f(\tilde{x}) \leq f(z) &= f(\lambda \tilde{x} + (1 - \lambda)y) \stackrel{\text{by convexity}}{\leq} \lambda f(\tilde{x}) + (1 - \lambda)f(y) \\ &\Leftrightarrow f(\tilde{x}) \leq \lambda f(\tilde{x}) + (1 - \lambda)f(y) \\ &\Leftrightarrow f(\tilde{x}) \underbrace{(1 - \lambda)}_{>0} \leq f(y)(1 - \lambda) \Rightarrow f(\tilde{x}) \leq f(y) \end{aligned}$$

So $\tilde{x}$ is a globally optimal solution.

□

1.2 Linear Programming

In this course we'll consider *Linear Convex Optimization* problems: all the constraints are convex functions and the objective is a convex function. We define a Linear Programming problem as follows:

$$\begin{aligned} \min f(x) &\Rightarrow \min c^T x \\ g_i(x) \leq 0 &\Rightarrow \begin{cases} -a_i^T x + b_i \leq 0, & i \in \{1, \dots, m\} \\ -x_j \leq 0, & j \in \{1, \dots, n\} \end{cases} \end{aligned} \quad (1.3)$$

Grouping the row vectors $a_1^T, \dots, a_m^T$ into a $m \times n$ matrix A , we obtain the compact representation:

$$\min\{c^T x : Ax \geq b, x \geq 0\}. \quad (1.4)$$

The 1.5 is called **canonical form**, which is different from

$$\min\{c^T x : Ax = b, x \geq 0\}. \quad (1.5)$$

the **standard form**. The canonical form it's used to define a model. Usually the model is converted into the standard form by the LP algorithm that tries to solves it. The conversion from one form to the other is obtained by adding some variables and/or some constraints to the problem.

1. Conversion from "max" to "min":

$$\max w^T x = -\min c^T x, \quad \text{defining } c^T = -w^T$$

2. Constraint conversion from " $\geq$ " to "=":

$$a_i^T x \geq b_i \Rightarrow \begin{cases} a_i^T x - s_i = b_i \\ s_i \geq 0 \end{cases}$$

The new variable s_i is known as the *surplus variable* for the i -th constraint.

3. Constraint conversion from " $\leq$ " to "=":

$$a_i^T x \leq b_i \Rightarrow \begin{cases} a_i^T x + s_i = b_i \\ s_i \geq 0 \end{cases}$$

The new variable s_i is known as the *slack variable* for the i -th constraint.

4. Free variables:

$$x_i \geq 0 \quad \Rightarrow \quad \begin{cases} x_i = x_i^+ - x_i^- \\ x_i^+ \geq 0 \\ x_i^- \geq 0 \end{cases}$$

Replacing x_i with $x_i^+ - x_i^-$, we eliminate x_i from the problem. Alternatively, x_i may be derived (depending on the other variables) from an equation of the problem in the standard form and thus replaced everywhere. In this way, we eliminate one equation and one variable.

5. Constraint conversion from "=" to " $\geq$ ":

$$a_i^T x = b_i \quad \Rightarrow \quad \begin{cases} a_i^T x \geq b_i \\ -a_i^T x \geq -b_i \end{cases}$$

6. From canonical form to standard form:

$$\min\{c^T x : Ax \geq b, x \geq 0\} \quad \Rightarrow \quad \min\{c^T x : Ax - s = b, x \geq 0, s \geq 0\}$$

7. From standard form to canonical form:

$$\min\{c^T x : Ax = b, x \geq 0\} \quad \Rightarrow \quad \min\{c^T x : Ax \geq b, -Ax \geq -b, x \geq 0\}.$$

1.2.1 Example

Consider the following LP problem:

$$\begin{cases} \min & -x_1 & -x_2 \\ & 6x_1 & +4x_2 \leq 24 \\ & 3x_1 & -2x_2 \leq 6 \\ & x_1 & \geq 0 \\ & & x_2 \geq 0. \end{cases} \quad (1.6)$$

the first two inequalities are called *explicit inequalities*, the last two are called *implicit inequalities*. The Figure 1.3 shows the set of feasible solution: as we can see the set is convex. To draw the objective function (usually indicated with z) we only have to calculate the gradient (in this case $(-1, -1)$), z is his perpendicular line. Moving z (in the right direction) leads us to an optimal solution which is always in one of the vertices of the feasible region. In our example the optimal solution is $x^* = [0, 6]^T$.

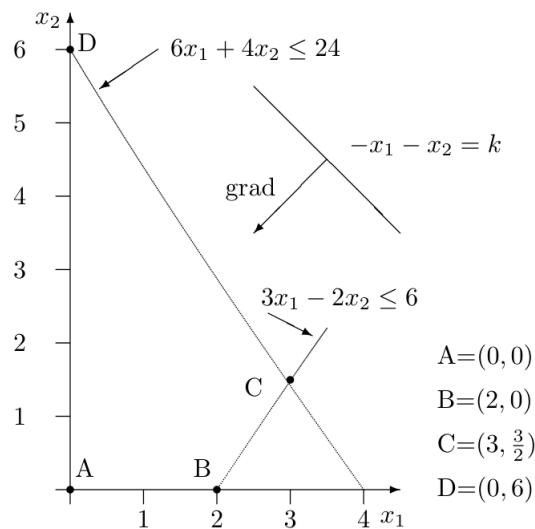


Figure 1.3: Graphical representation.

The research of an optimal solution is restricted to the vertices of the feasible region. In order to find a vertex we can simply consider two constraints and solve the linear system. For instance, we can find the vertex C as follows:

$$\begin{cases} 6x_1 + 4x_2 = 24 \\ 3x_1 - 2x_2 = 6 \end{cases} \Rightarrow x_1 = 3, x_2 = 3/2 \Rightarrow C = (3, 3/2).$$

Note that sometimes the solution is not a feasible solution.

At this point we're able to write an algorithm that find an optimal solution. The algorithm must:

1. calculate all the vertices;
2. try all the solution found at step (1);
3. return the best solution found at step (2). This solution is an optimal one.

The main problem of this approach is that the total number of vertices can be exponential, so step (2) in this case requires exponential time. A better approach is the following one:

1. start from a vertex;
2. compute the nearest vertices to the vertex computed at step (1);
3. move to the vertex that provides the best solution (the smallest the cost, the better the solution);

4. if the best vertex is the one we're in, then we have found an optimal solution.

By using this algorithm we can find a locally optimal solution that is also a globally optimal solution because we're considering convex optimization problems. What we have just introduced is the idea behind the *simplex algorithm*.

Chapter 2

Simplex Algorithm

2.1 Geometry of Linear Programming

The set of feasible solutions of an LP problem is defined by linear inequalities, each of which identifies a region of the vector space $\mathbb{R}^n$.

Definition 4 (Half-space and hyper-plane). *The set $\{x \in \mathbb{R}^n : \alpha^T x \leq \alpha_0\}$ is called half-space, the set $\{x \in \mathbb{R}^n : \alpha^T x = \alpha_0\}$ is called hyper-plane.*

Definition 5 (Convex polyhedron). *A (convex) polyhedron is defined as the intersection of a finite number of affine half-spaces and hyperplanes.*

Why finite and not infinite? That's because with infinite number of intersection we can obtain circular shape.

The sets of feasible solutions of LP problems are hence polyhedra.

Definition 6 (Polytope). *A bounded polyhedron P is called polytope.*

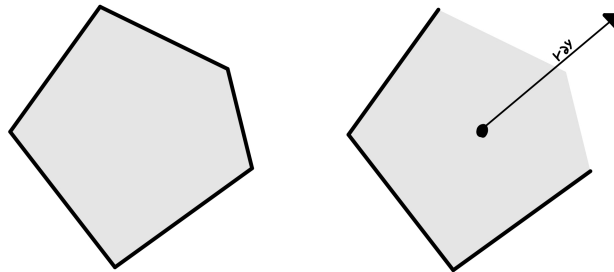


Figure 2.1: A polytope (left) and a polyhedron (right).

Definition 7 (Vertex). A point x of a polyhedron P is said to be a vertex of P if it cannot be expressed as a strict convex combination of other two points of the polyhedron, i.e., if there exist no $y, z \in P$, $z \neq y$ and $\lambda \in (0, 1)$ such that $x = \lambda y + (1 - \lambda)z$.

Theorem 3 (Minkowski-Weyl Theorem). Each point of a polytope can be obtained as the convex combination of its vertices.

An important consequence of the theorem is the following:

Theorem 4. If the set P of the feasible solution of a LP problem $\min \{c^T x : x \in P\}$ is bounded, then there exists at least one optimal vertex of P .

Proof. Let $x^1, \dots, x^k$ be the vertices of a polytope P and let $z^* = \min \{c^T x^i : i = 1, \dots, k\}$. So z^* is the optimal value considering only the vertices of the problem. Our goal is to prove that, for every value $y \in P$ it holds that $c^T y \geq z^*$. We have:

$$c^T y \stackrel{\text{by M-L theorem}}{=} c^T \left(\sum_{i=1}^k \lambda_i x^i \right) \stackrel{\text{by linearity}}{=} \sum_{i=1}^k \lambda_i \underbrace{(c^T x^i)}_{\geq z^*} \geq z^* \underbrace{\sum_{i=1}^k \lambda_i}_{=1} = z^*$$

Therefore $c^T y \geq z^*$ for all $y \in P \Rightarrow z^*$ is an optimal solution. $\square$

2.1.1 Vertices and basic solutions

As we know from the previous section, we can restrict the search of an optimal solution to the vertices of the polytope that represents the feasible region of a bounded LP problem. So the question now is: how we compute the vertices? Lets consider the problem introduced in 1.2.1, rewritten in the standard form:

$$\begin{cases} \min & -x_1 & -x_2 & & & \\ & 6x_1 & +4x_2 & +x_3 & & = 24 \\ & 3x_1 & -2x_2 & & +x_4 & = 6 \\ & x_1 & x_2 & +x_3 & +x_4 & \geq 0 \end{cases} \quad (2.1)$$

Setting a pair of variables to zero leads us to a system of equation. The solution is a vertex of P . For instance:

$x_1 = x_2 = 0 \rightarrow$ leads to the vertex A

$x_2 = x_4 = 0 \rightarrow$ leads to the vertex B

$x_3 = x_4 = 0 \rightarrow$ leads to the vertex C

$x_1 = x_3 = 0 \rightarrow$ leads to the vertex D

The general approach is to consider every possible pairs of variables and double check if the solution provided is a feasible one. In our example there are two cases that are *pathological*:

$$x_1 = x_4 = 0 \rightarrow \text{the interception has a } x_1 < 0 \rightarrow \text{infeasible solution}$$

$$x_2 = x_3 = 0 \rightarrow \text{the interception is outside the feasible region} \rightarrow \text{infeasible solution}$$

Sometimes the system that we get after setting the variables is infeasible, we have to skip those systems because they doesn't provide any vertices.

We can generalize the considerations above using the terminology of linear algebra and matrix calculus. Consider a LP problem in standard form in which $P = \min \{Ax = b, x \geq 0\}$, where $A \in \mathbb{R}^{m \times n}$ (i.e. A is a matrix with m rows and n columns), also:

- $n \geq m$ (if $n \leq m$ then P is a point, i.e. the problem has only one solution and that's not the usual case in LP);
- the n equations are linearly independent (so we need to remove the redundant ones before proceed);
- $\text{rank}(A) = m$ (i.e. *Full Row Rank*).

We have:

$$A = [A_1 | A_2 | \dots | A_n] \quad \text{where } A_i \text{ is a column of } A$$

now we define a squared matrix $B \in \mathbb{R}^{m \times m}$

$$B = [A_1 | A_2 | \dots | A_m]$$

where the columns A_i $i = 1, \dots, m$ are chosen from A : they can be placed in B in any order, the only constraint is that they must be linearly independent. The remaining columns of A (the ones that we haven't chose) are collected in a rectangular matrix F . So now we can rewrite A as follows:

$$A = [B | F]$$

(we also need to change the names of the variables in the vector x). Define $x = (x_B, x_F)$, we have

$$Ax = [B | F] \begin{bmatrix} x_B \\ x_F \end{bmatrix} = Bx_B + Fx_F \rightarrow Bx_B + Fx_F = b$$

now

$$\Rightarrow B^{-1}[Bx_B + Fx_F] = b \Rightarrow x_B + B^{-1}Fx_F = B^{-1}b \Rightarrow x_B = B^{-1}b - B^{-1}Fx_F$$

note that multiplying each member to B^{-1} doesn't change the solution of the system. We can rewrite the solution x as follows:

$$x = \begin{bmatrix} x_B \\ x_F \end{bmatrix} = \begin{bmatrix} B^{-1}b - B^{-1}Fx_F \\ x_F \end{bmatrix} \quad (2.2)$$

it still satisfies the given system $Ax = b$ by construction.

Definition 8 (Basic solution). *The solution obtained imposing $x_F = 0$ and $x_B = B^{-1}b$ is said to be the basic solution associated with basis B . The basic solution (and, by extension, basis B itself) is said to be feasible if $x_B = B^{-1}b \geq 0$.*

Definition 9 (Degenerate solution). *A basis B is said to be degenerate if $B^{-1}b$ has one or more zero components.*

Changing the basis B leads us to find all the vertices of a polytope.

Theorem 5. *A point $x \in P$ is a vertex of the not empty polyhedron $P := \{x \geq 0 : Ax = b\}$ if and only if x is a basic feasible solution of the system $Ax = b$.*

In the previous example we have seen how to compute a vertex using the geometrical method, the definition above says that we can use the algebra method.

Proof. 1. Let us first prove the implication

$$"x \text{ is a basic feasible solution} \Rightarrow x \text{ is a vertex}"$$

Wlog¹ let

$$x = (\underbrace{x_1, \dots, x_k}_{\text{positive}}, 0, \dots, 0)$$

a basic feasible solution associated with a basis B of A , where $k \geq 0$ is the number of non-zero components². Let

$$A = [\underbrace{A_1 | \dots | A_k}_B | \dots]$$

$A_1, \dots, A_k$ are part of B and possibly together with other columns of A (in case of degenerate solution). Note that $A_1, \dots, A_k$ are linearly independent due to the fact that they are in the basis B .

¹Without loss of generality.

²Some value might be zero, this is the case of a degenerate solution.

Let us assume by contradiction that x is not a vertex. There exist thus

$$y, z \in P, y \neq z, \text{ and } \lambda \in]0, 1[$$

such that

$$x = \lambda y + (1 - \lambda)z$$

(x can be obtained as a strict convex combination of y, z). Note that:

$$y = [y_1, \dots, y_k, 0, \dots, 0]^T \cdot \lambda$$

$$z = [z_1, \dots, z_k, 0, \dots, 0]^T \cdot (1 - \lambda)$$

$$x = [x_1, \dots, x_k, 0, \dots, 0]^T$$

the components of y and z going from index $k + 1$ through the end must be zero due to the fact that x has the same components equal to zero.

So we have:

$$y \in P \Rightarrow Ay = b \Rightarrow A_1 y_1 + \dots + A_k y_k = b$$

$$z \in P \Rightarrow Az = b \Rightarrow A_1 z_1 + \dots + A_k z_k = b$$

By subtracting the second equation from the first we obtain:

$$A_1 \underbrace{(y_1 - z_1)}_{\alpha_1} + \dots + A_k \underbrace{(y_k - z_k)}_{\alpha_k} = 0$$

define $\alpha_i := y_i - z_i$ for $i = 1, \dots, k$. Not all α_i for $i = 1, \dots, k$ equal to zero (Note³):

$$\Rightarrow A_1 \alpha_1 + \dots + A_k \alpha_k = 0$$

and by definition this means that $A_1, \dots, A_k$ are linearly dependent and cannot be part of the basis B . ($\Rightarrow$ contradiction).

2. We will now prove the implication

” x is a vertex $\Rightarrow x$ is a basic feasible solution (BFS) of a certain basis B ”.

First of all we know that x is feasible because is a vertex, so $x \in P$. Now as before:

$$x = (\underbrace{x_1, \dots, x_k}_{\text{positive}}, 0, \dots, 0)$$

³That is because $x = \lambda y + (1 - \lambda)z = (y - z)\lambda + z$ with $\lambda \in]0, 1[$ and $y \neq z$.

$$A = [A_1 | \dots | A_k | \dots | \dots]$$

We know that:

$$x \in P \Rightarrow A_1 x_1 + \dots + A_k x_k = b.$$

Now we have two cases:

- (a) $A_1, \dots, A_k$ are linearly independent (or $k = 0$). In this case we can construct the $m \times m$ matrix B by adding $m - k$ columns from A that are linearly independent. We have

$$B = [A_1 | \dots | A_k | \underbrace{\dots}_{m-k \text{ rows}}]$$

and

$$x = \begin{bmatrix} x_B \\ x_F \end{bmatrix} = \begin{bmatrix} B^{-1}b \\ 0 \end{bmatrix}$$

where x is a basic solution by definition.

- (b) $A_1, \dots, A_k$ are linearly dependent. That implies:

$$\exists \alpha_1, \dots, \alpha_k \text{ not all } = 0 \text{ such that } \alpha_1 A_1 + \dots + \alpha_k A_k = 0.$$

Let

$$y := (x_1 - \varepsilon \alpha_1, \dots, x_k - \varepsilon \alpha_k, 0, \dots, 0)$$

$$z := (x_1 + \varepsilon \alpha_1, \dots, x_k + \varepsilon \alpha_k, 0, \dots, 0)$$

where $\varepsilon > 0$, $\varepsilon \approx 0$, that implies⁴ $y, z \geq 0$. We have

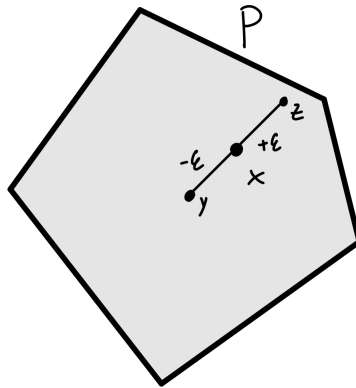


Figure 2.2: α is the direction.

$$\begin{aligned}
Ay &= (x_1 - \varepsilon\alpha_1)A_1 + \dots + (x_k - \varepsilon\alpha_k)A_k \\
&= (A_1x_1 + \dots + A_kx_k) - \varepsilon(A_1\alpha_1 + \dots + A_k\alpha_k) \\
&= b - \varepsilon \cdot 0 \\
&= b \Rightarrow y \in P
\end{aligned}$$

$$\begin{aligned}
Az &= (x_1 + \varepsilon\alpha_1)A_1 + \dots + (x_k + \varepsilon\alpha_k)A_k \\
&= (A_1x_1 + \dots + A_kx_k) + \varepsilon(A_1\alpha_1 + \dots + A_k\alpha_k) \\
&= b + \varepsilon \cdot 0 \\
&= b \Rightarrow z \in P
\end{aligned}$$

Therefore we have that $y, z \geq 0$, $y, z \in P$ and $y \neq z$, and by construction

$$x = 0.5y + 0.5z$$

this means that vertex x can be expressed as the strict convex combination of two distinct points of $P \Rightarrow x$ cannot be a vertex $\Rightarrow$ contradiction.

□

2.2 The Simplex Method

As we have mentioned in the previous sections, enumerating all the vertices of the feasible solution may require exponential time. The Simplex Method offers a more efficient strategy that consists in two main parts:

1. Verify the optimality of the current basic feasible solution (the current vertex).
2. Move from a basic feasible solution to another "adjacent" one with better value of the objective function.

By iterating over these two steps, the algorithm finds the local optimum solution. Note that in the worst case this method generate all the basic feasible solutions of the problem, but in the average case is very effective.

2.2.1 Optimality test

In this subsection we will explain the first main part of the Simplex Method. Let $\min\{c^T x : Ax = b, x \geq 0\}$ be the problem to solve, and consider a feasible basis B . Let also $\tilde{x} \in P$ a vertex

⁴It's a property.

(basic solution). Take any point $x \in P$ and define:

$$c^T x = [c_B^T, c_F^T] \begin{bmatrix} x_B \\ x_F \end{bmatrix} = c_B^T x_B + c_F^T x_F \Rightarrow$$

Using 2.2 we have that

$$\begin{aligned} c^T x &= c_B^T (B^{-1}b - B^{-1}F x_F) + c_F^T x_F \\ &= c_B^T B^{-1}b - c_B^T B^{-1}F x_F + c_F^T x_F \\ &= \underbrace{c_B^T B^{-1}b}_{c_0} + \underbrace{(c_F^T - c_B^T B^{-1}F)}_{\bar{c}_F^T} x_F \\ &= c_0 + \bar{c}_F^T x_F \end{aligned}$$

where: $c_0 = c^T \tilde{x}$ that is, the cost of the current vertex (basic solution); $\bar{c}_F^T$ is the objective function vector expressed as a function of the non-basic variables only. Note that:

$$\text{if } \bar{c}_F \geq 0 \Rightarrow c^T x \geq c_0 = c^T \tilde{x} \Rightarrow \tilde{x} \text{ is optimal.}$$

Definition 10 (Reduced cost vector). *The vector*

$$\bar{c}^T := c^T - c_B^T B^{-1}A \stackrel{A=[B|F]}{=} [\underbrace{c_B^T - c_B^T B^{-1}B}_{=0^T}, c_F^T - c_B^T B^{-1}F] = [0^T, c_F^T - c_B^T B^{-1}F]$$

is called the reduced cost vector with respect to basis B.

Note that, by definition, the reduced cost of a basic variable is zero (since the first component of $\bar{c}^T$ is 0^T). This condition uniquely determinates the reduced cost of non-basic variables.

Theorem 6. *Let B be a feasible basis. If $\bar{c}^T := c^T - c_B^T B^{-1}A \geq 0^T$ then the basic solution associated with B is optimal.*

If the reduced cost vector has a component that has a value < 0 , then the basic solution $\tilde{x}$ isn't an optimal one.

2.2.2 Change of Basis

In this subsection we will analyse the second main part of the Simplex Method. Lets say that that the vertex $\tilde{x}$ found previously isn't optimal, so we have found that the reduced cost vector $\bar{c}_F$ has an element that is negative. Let x_h be a non-basic variable with reduced cost $\bar{c}_h < 0$:

$$\bar{c}_h := c_h - c_B^T B^{-1}A_h, \quad \bar{c}_h < 0.$$

We want to increase the value of x_h (starting from 0) to the largest positive value (indicated with θ) and keep all the other non-basic variables to zero. Using this approach we can move in the direction of a near vertex that offers a better solution (the cost is lower).

We define the costs

$$z = c_0 + \bar{c}^T = c_0 + \underbrace{\bar{c}_h}_{\leq 0} \underbrace{x_h}_{\geq 0}$$

so by increasing x_h the objective function improves. The problem is that, by increasing x_h , at some point another variable can assume a negative value and that means we have reached a point outside the feasible region. So we have to increase x_h up to θ : in this situation another variable (one of the basic variables) has reached the value 0, that is, θ is the maximum value that preserves the feasibility.

The system $x_B = B^{-1}b - B^{-1}F x_F$ can be rewritten as⁵:

$$x_B = \underbrace{B^{-1}b}_{\bar{b}} - \underbrace{B^{-1}A_h}_{\bar{A}_h} x_h = \bar{b} - \bar{A}_h x_h.$$

$$\begin{bmatrix} x_{\beta[1]} \\ \vdots \\ x_{\beta[m]} \end{bmatrix} = \begin{bmatrix} \bar{b}_1 \\ \vdots \\ \bar{b}_m \end{bmatrix} - \begin{bmatrix} \bar{a}_{1h} \\ \vdots \\ \bar{a}_{mh} \end{bmatrix} x_h$$

where $x_{\beta[1]}, \dots, x_{\beta[m]}$ are the basic variables. As we mentioned earlier we have to preserve the feasibility, so we impose the condition $x_B \geq 0$ and we obtain the following system of inequalities:

$$\begin{cases} x_{\beta[1]} = \bar{b}_1 - \bar{a}_{1h}x_h \geq 0 \\ \vdots \\ x_{\beta[i]} = \bar{b}_i - \bar{a}_{ih}x_h \geq 0 \\ \vdots \\ x_{\beta[m]} = \bar{b}_m - \bar{a}_{mh}x_h \geq 0 \end{cases} \Rightarrow \begin{cases} \bar{a}_{1h}x_h \leq \bar{b}_1 \\ \vdots \\ \bar{a}_{ih}x_h \leq \bar{b}_i \\ \vdots \\ \bar{a}_{mh}x_h \leq \bar{b}_m \end{cases} \quad (2.3)$$

For each condition we have only two possibilities:

- if $\bar{a}_{ih} \leq 0$ then the i -th condition does not imply any constraint on variable $x_h \geq 0$.
- if $\bar{a}_{ih} > 0$ then the condition implies $x_h \leq \bar{b}_i / \bar{a}_{ih}$.

Since all the conditions must be satisfied at the same time, the maximum value that x_h can take

⁵That is because x_h is the only non-basic variable $\neq 0$, so we're only interested in the column A_h of A .

may be calculated as:

$$\theta = \min \left\{ \frac{\bar{b}_i}{\bar{a}_{ih}} : i \in \{1, \dots, m\}, \bar{a}_{ih} > 0 \right\} \quad (2.4)$$

When x_h takes on its limit value θ , the objective function improves by the quantity $|\bar{c}_h| \cdot \theta \geq 0$.

So when we impose $x_h = \theta$, one of the basic component becomes zero. We can find the index of that component by simply:

$$t = \arg \min_{i \in \{1, \dots, m\}} \left\{ \frac{\bar{b}_i}{\bar{a}_{ih}}, \bar{a}_{ih} > 0 \right\}.$$

In summary, during the transition to another vertex:

$$\begin{array}{ccc} x_h = 0 & \xrightarrow{\text{transition}} & x_h = \theta \\ x_{\beta[t]} > 0 & \xrightarrow{\text{transition}} & x_{\beta[t]} = 0 \end{array}$$

During the transition, $x_{\beta[t]}$ "leaves" the basis, instead x_h enters. In other words the column A_h enters the current basis B substituting column $A_{\beta[t]}$ associated with row t (Figura 2.3).

$$B = [A_{\beta[1]}, \dots, A_{\beta[t]}, \dots, A_{\beta[m]}] \Rightarrow \tilde{B} = [A_{\beta[1]}, \dots, A_{\beta[t-1]}, A_h, A_{\beta[t+1]}, \dots, A_{\beta[m]}].$$

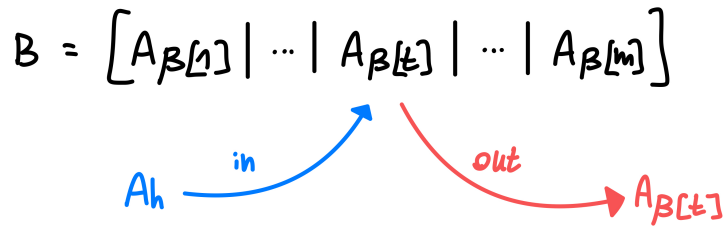


Figure 2.3: x_h enters the basis, $x_{\beta[t]}$ leaves the basis.

In this situation we have found a near vertex to the starting one that offers a better solution.

Algorithm 1 Simplex Algorithm (matrix form)

```

1: let  $\beta[1], \dots, \beta[m]$  be the indices of the columns of an initial feasible basis
2:  $unbounded \leftarrow false$ 
3:  $optimal \leftarrow false$ 
4: while  $optimal = false$  and  $unbounded = false$  do
5:    $B \leftarrow [A_{\beta[1]} | \dots | A_{\beta[m]}]$ 
6:    $B^{-1} \leftarrow invert\_matrix(B)$ 
7:    $u^T \leftarrow c_B^T B^{-1}$ 
8:    $\bar{c} \leftarrow c - u^T A$  ▷ Reduced cost vector.
9:   if  $\bar{c} \geq 0$  then ▷ Optimality test.
10:     $optimal = true$ 
11:   else
12:     $h \leftarrow$  choose any non-basic  $x_h$  with  $\bar{c}_h = c_h - u^T A_h < 0$  ▷ Usually the lowest  $\bar{c}_h$ .
13:     $\bar{b} \leftarrow B^{-1} b$ 
14:     $\bar{A}_h \leftarrow B^{-1} A_h$ 
15:    if  $\bar{a}_{ih} \leq 0 \forall i = 1, \dots, m$  then
16:       $unbounded \leftarrow true$ 
17:    else
18:       $t \leftarrow \arg \min_{i \in \{1, \dots, m\}} \{\bar{b}_i / \bar{a}_{ih} : \bar{a}_{ih} > 0\}$ 
19:       $\beta[t] \leftarrow h$  ▷ Update the current basis.
20:    end if
21:  end if
22: end while
23: if  $optimal = true$  then
24:   return  $x^* = (B^{-1} b | 0)$  ▷  $x^*$  is an optimal vertex.
25: end if
26: return  $-\infty$  ▷ The problem is unbounded.

```

2.2.3 Tableau Form of the Simplex Method

The original system ($Ax = b$, $z = c^T x$) is represented by the initial tableau in Figure 2.4. Considering a basis B and partitioning A in $[B, F]$, the initial tableau can be rewritten as seen in Figure 2.5

We want to convert this tableau into its *canonical form* with respect to B , to reflect the system

$$x_B = \underbrace{B^{-1}b}_{\bar{b}} - \underbrace{B^{-1}F}_{\bar{F}} x_F, \quad z = \underbrace{c_B^T B^{-1}b}_{-\bar{c}_0} + \underbrace{(c_F^T - c_B^T B^{-1}F)}_{\bar{c}_F} x_F.$$

To this end, we premultiply the original tableau by B^{-1} , obtaining the tableau shown in Figure 2.6.

In the tableau in canonical form, the basic columns form the identity matrix, while the reduced costs corresponding to the basic variables are all equal to zero. The basic components of the

	x_1	$\dots$	x_n	z
row 0	0	$\mathbf{c}^T$		-1
row 1	$\mathbf{b}$	A		0
$\dots$				$\vdots$
row m				0

Figure 2.4: Original tableau.

	x_1	$\dots$	x_m	x_{m+1}	$\dots$	x_n	z
0	$\mathbf{c}_B^T$			$\mathbf{c}_F^T$			-1
$\mathbf{b}$	B			F			0
							$\vdots$
							0

Figure 2.5: Original tableau.

	x_1	$\dots$	x_m	x_{m+1}	$\dots$	x_n	z
$-z$	$\bar{c}_0$	0	$\dots$	0	$\bar{\mathbf{c}}_F^T$		-1
$x_{\beta[1]}$	$\bar{\mathbf{b}}$	I			$\bar{\mathbf{F}}$		0
$\vdots$							$\vdots$
$x_{\beta[m]}$							0

Figure 2.6: Canonical tableau.

current basic solution can be read in the first column ($\bar{\mathbf{b}} := B^{-1}\mathbf{b}$). The value $\bar{c}_0$ appearing in position (0,0) of the tableau is the opposite of the value of the objective function calculated for the current basic solution ($-\bar{c}_0 := \mathbf{c}_B^T B^{-1}\mathbf{b}$). Columns $\bar{A}_h := B^{-1}A_h$ of matrix $\bar{\mathbf{F}} := B^{-1}\mathbf{F}$ contain the necessary values $\bar{a}_{ih}$ to compute θ . Note that the column corresponding to variable z has no information (it is always equal to $[-1, 0, \dots, 0]^T$), and may thus be omitted from the tableau.

The fundamental point of the algorithm is represented by the **pivot operation** on (the circled

element)⁶ $\bar{a}_{th}$ (called **pivot element**). This operation is aimed at

- bringing value 1 in position (t, h) , dividing the entire row t by $\bar{a}_{th} > 0$;
- bringing value 0 in position (i, h) , subtracting from row i the new row t multiplied by $\bar{a}_{ih}$ (for every row $i \neq t$).

Algorithm 2 Simplex Algorithm (Tableau Form)

```

1: Input: Let  $\beta[1], \dots, \beta[m]$  be the indices of the columns of an initial feasible basis
2: Create the initial tableau  $\bar{A} \leftarrow \{\bar{a}[i, j] : i = 0, \dots, m; j = 0, \dots, n\}$  in canonical form with
   respect to the initial basis
3: unbounded  $\leftarrow$  false
4: optimal  $\leftarrow$  false
5: while optimal = false and unbounded = false do
6:   if  $\bar{a}[0, j] \geq 0$  for all  $j = 1, \dots, n$  then
7:     optimal  $\leftarrow$  true
8:   else
9:     Choose a non-basic variable  $x_h$  with  $\bar{a}[0, h] < 0$  ▷ Reduced costs
10:    if  $\bar{a}[i, h] \leq 0$  for all  $i = 1, \dots, m$  then
11:      unbounded  $\leftarrow$  true
12:    else
13:      Compute  $t \leftarrow \arg \min \{\bar{a}[i, 0] / \bar{a}[i, h] : i \in \{1, \dots, m\}, \bar{a}[i, h] > 0\}$  ▷  $\bar{b}_i / \bar{a}_{i,h}$ 
14:      PIVOT( $t, h$ ) ▷ Tableau update
15:       $\beta[t] := h$  ▷  $x_h$  enters the basis,  $x_{\beta[t]}$  leaves
16:    end if
17:  end if
18: end while

```

⁶An example can be found on ipad notes.

Algorithm 3 PIVOT Procedure

```

1: procedure PIVOT( $t, h$ )
2:    $save \leftarrow \bar{a}[t, h]$  ▷ Value of the pivot element
3:   for  $j \leftarrow 0$  to  $n$  do
4:      $\bar{a}[t, j] \leftarrow \bar{a}[t, j] / save$ 
5:   end for
6:   for  $i \leftarrow 0$  to  $m$  do
7:     if  $i \neq t$  and  $\bar{a}[i, h] \neq 0$  then
8:        $save \leftarrow \bar{a}[i, h]$ 
9:       for  $j \leftarrow 0$  to  $n$  do
10:         $\bar{a}[i, j] \leftarrow \bar{a}[i, j] - save \cdot \bar{a}[t, j]$ 
11:      end for
12:     end if
13:   end for
14: end procedure

```

2.2.4 The two phases method

The goal of this subsection is define an efficient method to find an initial feasible basis. The two phase method consists in applying the simplex algorithm twice: in the first phase, an initial basis is identified (if it exists), while in the second phase the problem is actually solved⁷.

Let consider the linear problem $\min\{c^T x : Ax = b, x \geq 0\}$. Without loss of generality, we can consider $b \geq 0$ also we can define a new linear problem starting from the first one:

$$\min\{z = c^T x : Ax = b, x \geq 0\} \Rightarrow \min\{w = \sum_{i=1}^m y_i : Ax + Iy = b, x \geq, y \geq 0\}$$

where we have added an **artificial variable** y_i for each constraint. The new linear problem is called **artificial problem**. As we can see, if (and only if) the artificial variables $y_i = 0 \forall i = 1, \dots, m$ (equivalently $w = 0$) then the solution found by solving the artificial problem is also feasible for the original problem. This is why we have changed the objective function from z to w . Note that finding a feasible solution to the artificial problem is very easy: we can set $x = 0$ and we get that $y = b$ is a feasible solution and this is not a generic solution, is a vertex (a basic feasible solution). So we set $B = [y_1 \mid \dots \mid y_m]$ as a basis for the first iteration of the simplex method applied to the artificial problem.

By construction, the tableau associated to the artificial problem is almost in canonical form, we only have to modify the first row (row 0): the reduced cost associated to the basic variable must equal to zero ($\bar{c}_B = 0$). In order to bring the tableau in his canonical form, we cas use the

⁷An example can be found on ipad notes.

following trick:

$$\text{row } 0 = \text{row } 0 - \sum_{i=1}^m [\text{row } 0] \quad (2.5)$$

where row 0 is the current first row of the tableau. Another trick is to apply the *minipivot* operation⁸: we identify the pivot element as the element 1 under the column y_i (in total there are m pivot element); then we apply the pivot operation only for the first row of the tableau (hence we update the row 0). So in total we apply m minipivot operation and we get the same result of the formula above. This second trick is very useful in the implementation of the algorithm.

Now that we have the tableau associated to the artificial problem in his canonical form, we can apply the simplex method, using $B = [y_1 \mid \dots \mid y_m]$ as the first basis. Let w^* be the optimal value of the artificial problem. We have three cases:

1. $w^* = 0$: in this case we can use the optimal solution of the artificial problem. This solution is a valid feasible basic solution for the original problem $\Rightarrow$ we have found a feasible basis and we can use it for the first iteration of the simplex method applied to the original problem. The tableau associated to the original problem is obtained by deleting the columns $y_i \forall i = 1, \dots, m$ from the tableau associated to the artificial problem. We can use this operation because the simplex method applied to the artificial problem, does several pivot operation, hence linear combination of rows (i.e. $\text{row } i = \text{row } i + \text{row } 1 + 4\text{row } 5$); we never compute linear combination of the columns $\Rightarrow$ the solution of the problem is unchanged and we can simply remove the columns associated with the artificial variables. The initial tableau of the original problem is almost in his canonical form, we only have to apply some minipivot operations to the first row.
2. $w^* > 0$: this means that the original problem is infeasible so we skip the phase 2.

The phase 2 manage the case in which the original problem is infeasible.

If we're in the case (1) described above, we need to manage two small edge cases:

- (a) There exists a basic variable y_h on a row i : in this case we have $y_h^* = 0$ (since $w^* = 0$), hence we are in the presence of **degeneracy**. The situation is shown in Figure 2.7: If there exist a value $\bar{a}_{ij} \neq 0$ in correspondence to a variable x_j ($j \leq n$), then it is sufficient to perform a pivot operation on element (i, j) . Since $\bar{b}_i = 0$, this operation is admissible even if $\bar{a}_{ij} < 0$ and does not entail variations to the objective function w . In this way,

⁸Minipivot operation: is applied only for the first row of the tableau and not for all the rows.

		x_1	$\dots$	x_j	$\dots$	x_n	y_1	$\dots$	y_h	$\dots$	y_m
$-w$	0								0		
									0		
									0		
y_h	0	$\bar{a}_{i1}$	$\dots$	$\bar{a}_{ij}$	$\dots$	$\bar{a}_{in}$			1		
									0		
									0		

Figure 2.7

y_h leaves the basis. Repeating the procedure for all the basic y_h leads us back to the non-degenerate case (1) described above.

- (b) One last possibility remains: all values $\bar{a}_{i1}, \dots, \bar{a}_{in}$ equal zero (Figure 2.7). In this case, eliminating the artificial columns (an operation that is always admissible) we obtain a tableau equivalent to the original one, but with a zero row. This implies that the matrix A does not have full row rank m , and thus the i -th row of the current tableau may be eliminated without problems because it has no info associated (the corresponding equation of system the $Ax = b$ is linearly dependent from the others). So we remove the row and we proceed to the second phase.

2.2.5 Convergence and Degeneracy

In this section we will discuss about the convergence of the simplex method. Let call z the objective function of the problem, and $|\Delta z| = |\bar{c}_h| \theta_h$ is defined as the improvement that we get applying the pivot operation.

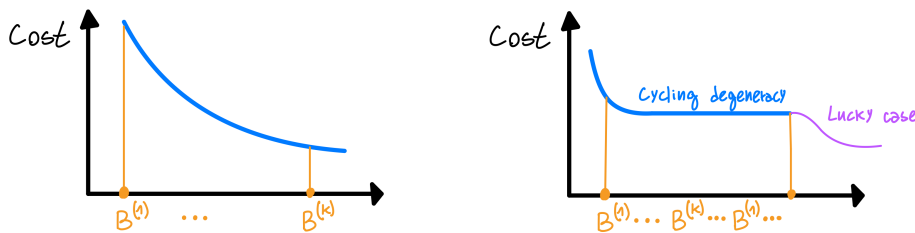


Figure 2.8

We have two cases:

1. $|\Delta z| > 0$ for all iterations. In this case the same basis B cannot appear twice among the sequence, hence each vertex can only be visited once. In the worst case we have to visit all the BFS and that is a finite number: $\binom{n}{m}$.
2. At some iteration $\Delta z = 0$. In this case the simplex can get stuck in a loop, this is because the simplex method is deterministic so, it will compute the same operation if the basis $B^{(i)}$ is the same as $B^{(j)}$ with $j < i$ (same basis $\Rightarrow$ same values in the tableau). So the question is, when this problem arise? In order to get $\Delta z = 0$:

$$\Delta z = 0 \stackrel{\text{def}}{=} \underbrace{|\bar{c}_h|}_{\theta_h} \theta_h \Rightarrow \theta_h = 0$$

and

$$\theta_h := \min\{\bar{b}_i/\bar{a}_{ih} : \bar{a}_{ih} > 0, i = 1, \dots, m\} \Rightarrow \exists b_i = 0, i = 1, \dots, m.$$

Having a $\bar{b}_i$ in *row* i of the tableau means that the cost $\bar{c}_0$ in the left high corner will not change, it remains the same as before, hence $\Delta z = 0$.

Geometrically speaking, if we're in case (2), than we have found the same vertex with two different basis: the result of the pivot operation has changed the basis, but this basis does not change the corresponding feasible solution (vertex), since variable x_h enter the basis at level $\theta = 0$. Hence, after a certain number of degenerate iterations, there is the concrete risk to pass twice through a certain basis, creating thus a previously generated tableau. This circumstance may cause the simplex method to cycle indefinitely through a sequence of degenerate bases, all associated with the same vertex. In this case we talk about **cycling degeneracy**.

Figure 2.9(b) illustrate a method used for avoiding the cycling degeneracy. Perturbation consists in varying, sometimes in a pseudo-random way, the vector b of the right-hand sides. If the perturbation is sufficiently strong, the algorithm typically manages to get out of the degenerate point. Obviously it will then be necessary to remove the introduced perturbation.

Another possibility is to use non determinism by randomize at each iteration the choice of x_h (choosing among the variable with reduced cost that is negative) and also randomizing the choice of Θ (choosing among the ratio $\bar{b}_i/\bar{a}_{ih}$ that identify the same value).

One of the easiest and most elegant anti-cycling rule has been proposed by R.G. Bland: using this rule, we can demonstrate that the simplex algorithm converges in a finite number of iterations.

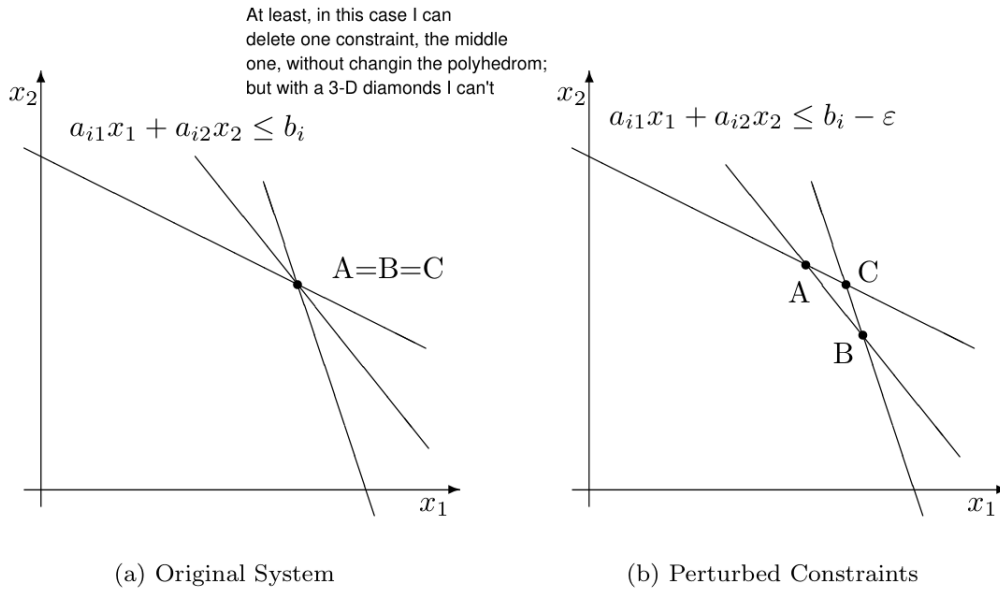


Figure 2.9: (a) The cycle may involve the bases associated with points A , B and C . (b) A simple solution to avoid cycling degeneracy: perturb the problem when the cycling risk occurs, i.e., when the objective function does not improve for many iterations.

Definition 11 (Bland's Rule). *Whenever it is possible choose, always choose the entering/leaving variable x_j with the smallest index j .*

In particular, we have to:

- choose variable x_h to enter the basis defining $h := \arg \min j : \bar{c}_j < 0$;
- among all rows t with $\bar{b}_t / \bar{a}_{th} = \theta$ that are eligible for the pivot operation, choose the one with minimum $\beta[t]$ so as to force the smallest-index variable $x_{\beta[t]}$ to leave the basis.

Theorem 7. *Using Bland's rule, the simplex algorithm converges after, at most, $\binom{n}{m}$ iterations.*

In other words, if we use the Bland's rule, the simplex method will visit a vertex only once, hence we avoid the cycling degeneracy.

Proof. We will prove the correctness of the theorem using the proof by contradiction. Let us consider as counterexample the smallest (in terms of number of columns and rows) LP problem for which there is no convergence, hence we get stuck in a loop. In this situation the simplex algorithm has to go through a cyclic sequence $B_1, \dots, B_k = B_1$ of bases. During this sequence, pivot operations are performed on all rows and all columns of the tableau, otherwise eliminating the not involved rows/columns we would obtain a smaller counterexample⁹. It

⁹If a column/row does not change in any pivot operations, then if we remove it from the tableau we will get stuck in the loop anyway.

follows that all variables enter and leave, in turn, the current basis. Also we must have that $\bar{b}_t = 0$ for all rows $t \in \{1, \dots, m\}$, otherwise in the iteration in which the pivot operation is performed on row t , we would have $\theta > 0$, hence the value of the objective function would change, preventing cycling in the sequence of bases¹⁰.

Consider now tableau T in which variable x_n leaves the current basis to let a given non-basic variable x_h enter the basis. Let us indicate with $x_{\beta[i]}$ the basic variable in row $i \in \{1, \dots, m\}$, and with t the row in which x_n is in the basis (i.e., $\beta[t] = n$).

Tableau T :

		...	$x_{\beta[i]}$	...	x_h	...	x_n	
$-z$	$=$		0		—		0	
...	0		0		—		0	...
$x_{\beta[i]}$	0		1		—		0	$\mu_i \geq 0$
...	0		0		—		0	...
$x_{\beta[t]}$	0		0		$\oplus$		1	$\mu_t < 0$
...	0		0		—		0	...

Figure 2.10

The main features of this tableau are:

- $\bar{b}_i = 0 \forall i$ (as already seen)
- $\bar{c}_h < 0$ (since x_h enters the basis)
- $\bar{c}_{\beta[i]} = 0 \forall i$ (reduced costs basic variables)
- $\bar{a}_{th} > 0$ (pivot element)
- $\bar{a}_{ih} \leq 0 \forall i \neq t$ (to force the Bland's rule)

Note that if there existed $i \neq t$ with $\bar{a}_{ih} > 0$, hence Bland's rule would certainly have preferred to let variable $x_{\beta[i]}$ (instead of x_n) leave the basis.

Consider now tableau $\tilde{T}$ in which x_n re-enters the basis:

Indicating with $\tilde{c}_j$ the reduced costs in row 0 of $\tilde{T}$, we must have:

- $\tilde{c}_n < 0$ (since x_n enters the basis);
- $\tilde{c}_j \geq 0 \forall j \neq n$ (Bland's rule).

¹⁰Having $\bar{b} = 0$ is a case with "extreme" degeneracy, because we cannot avoid it.

Tableau $\tilde{T}$:

	...	$x_{\beta[i]}$	...	x_h	...	x_n
	+	+	+	+	+	-

Figure 2.11

But then:

- $\underbrace{\tilde{c}_{\beta[t]}}_{=\tilde{c}_n < 0} = \underbrace{\bar{c}_{\beta[t]}}_{=0} + \mu_t \Rightarrow \mu_t < 0$
 $(\beta[t] = n \text{ is referred to the tableau } T, \text{ in } \tilde{T} \text{ we have } \tilde{\beta}[t] \text{ which identify a different variable}).$
- $\underbrace{\tilde{c}_{\beta[i]}}_{\geq 0} = \underbrace{\bar{c}_{\beta[i]}}_{=0} + \mu_i \Rightarrow \mu_i \geq 0 \forall i \neq t$
 $(\beta[i] \text{ is referred to the tableau } T \text{ using our convention}).$

Hence, there is a contradiction:

$$\underbrace{\tilde{c}_h}_{\geq 0} = \underbrace{\bar{c}_h}_{< 0} + \sum_{i \neq t} \underbrace{\bar{a}_{ih}}_{\leq 0} \underbrace{\mu_i}_{\geq 0} + \underbrace{\bar{a}_{th}}_{> 0} \underbrace{\mu_t}_{< 0} < 0.$$

which is a contradiction because we have assumed $\tilde{c}_j \geq 0 \forall j \neq n$. □

2.3 Revised Simplex Method

The main problem of the simplex method implemented using the tableau is that the memory space: the tableau requires $(m+1) \times (n+1)$ elements. Typically the matrix A associated to the problem is *sparse*: it has many zero entries.

Instead of using the tableau, it's preferable to use the simplex algorithm in matrix form (Algorithm 1). The fundamental steps of this algorithm are the computation of the inverse matrix B^{-1} and of vector $u^T := c_B^T B^{-1}$. We will now see an easy method that allows one to iteratively update B^{-1} and u^T by performing a pivot operation similar to the one seen for the simplex method in tableau form.

Lets start by considering the tableau adding m auxiliary columns like we have done in Figure 2.12.

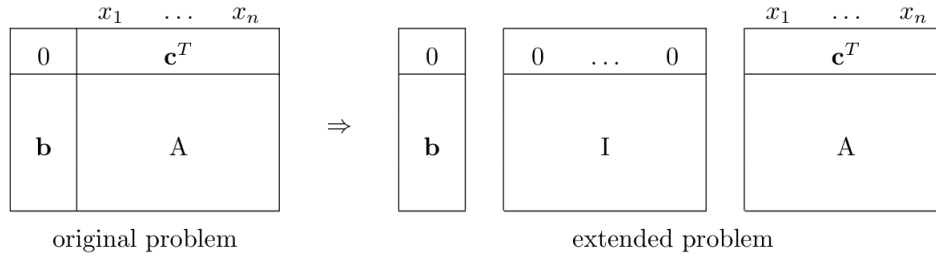


Figure 2.12

Starting from the extended tableau we can apply the simplex method in tableau form, updating at each iteration the entire tableau but preventing the auxiliary columns to enter the basis. Consider a generic basis B and the corresponding extended tableau in canonical form with respect to B . As we saw in the previous sections, we can obtain the rows $1, \dots, m$ of the tableau in canonical form with respect to B by premultiplying the original rows by B^{-1} . The reduced costs in row 0 can be computed using the formula:

$$[\text{row } 0] := [\text{original row } 0] - \sum_{i=1}^m u_i [\text{original row } i]$$

in other words:

$$\bar{\mathbf{c}}^T = \mathbf{c}^T - \mathbf{u}^T A \quad \text{where } \mathbf{u}^T = \mathbf{c}_B^T B^{-1}.$$

This means that the original tableau and the one in canonical form are of the following type:

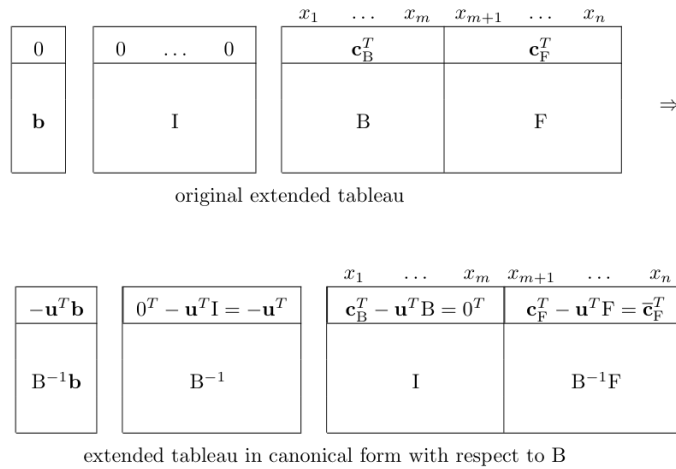


Figure 2.13

Note that $-u^T$ and B^{-1} explicitly appear in the auxiliary columns of the tableau.

The revisited simplex algorithm stores only the relevant part of the extended tableau: column 0 and the auxiliary m columns. Those columns identify the so called **Carry matrix**.

At each iteration:

1. Identify the variable x_h that enters the basis: as we know we have to compute the reduced cost to choose the variable x_h . To do that we simply compute the reduced cost $\bar{c}_h := c_h - u^T A_h$ until we find the first non-basic variable x_h with $\bar{c}_h < 0$. Note that we have all the data to compute $\bar{c}_h$. The complexity of the computation of $\bar{c}_h$ is $O(\text{number of non zero values in } A_h)$ (very fast).
2. The values $\bar{a}_{ih}$ aren't stored in the carry matrix so we need to compute the values. To do that we have to explicitly generate the column $\begin{bmatrix} \bar{c}_h \\ \bar{A}_h \end{bmatrix}$ where $\bar{A}_h := B^{-1}A_h$ (as we saw in subsection 2.2.2) associated with x_h in the tableau in canonical form with respect to B , and places it temporarily side by side with the current CARRY matrix (*augmentation*). The computation of $\bar{A}_h$ is a fast operation because A is a sparse matrix (so A_h isn't a very big vector).
3. Choose the pivot element: the minimum $\bar{b}_i/\bar{a}_{ih}$. At this point we have all the information that we need to compute the value of θ . So we compute $\theta \Rightarrow$ we find the variable $x_{\beta[t]}$ that leaves the basis, we perform the pivot operation on the augmented carry matrix and finally we can eliminate the column corresponding to the variable x_h .

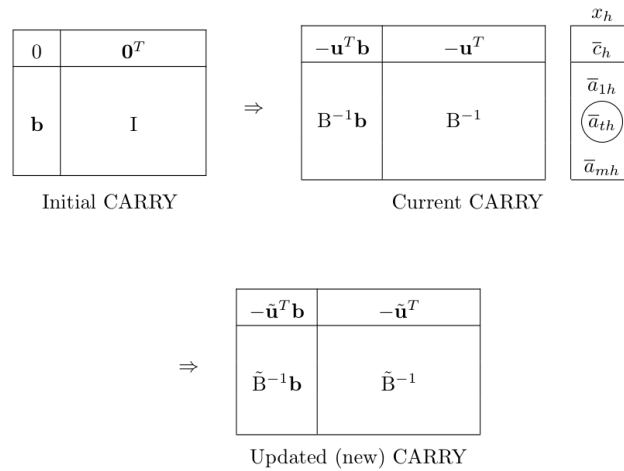


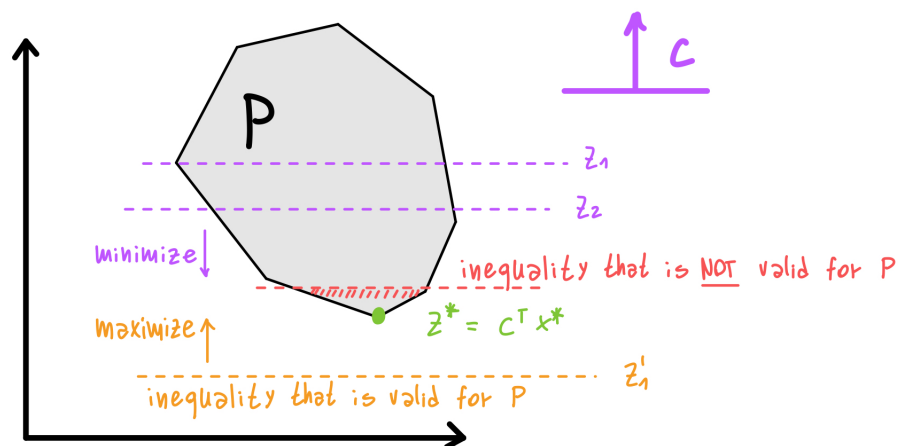
Figure 2.14

Chapter 3

Duality in linear programming

Any "primal" linear programming problem in minimization form is associated with a "dual" linear programming problem in maximization form. The two problems are defined on different spaces, but they have the same optimal value of the objective function (except for particular situations).

3.1 Valid inequalities



Definition 12. Given a set $X \subseteq \mathbb{R}^n$, an inequality $c^T x \geq c_0$ satisfied by all $x \in X$ is said to be valid for X .

We can consider a linear problem in standard form $\min\{c^T x : x \in P\}$ defined in the non-empty polyhedron $P := \{x \geq 0 : Ax = b\}$. Suppose that the finite minimum exists. It's easy to see that

following polarity relation applies:

$$\min\{c^T x : x \in P\} = \max\{c_0 : c^T x \geq c_0 \forall x \in P\}$$

The problem $\min\{c^T x : x \in P\}$ consists then in identifying the maximum value c_0 for which the linear inequality $c^T x \geq c_0$ is verified for all vertices of P , and thus for all $x \in P$. Obviously, c_0 does not exist if $c^T x$ is not bounded from below in P , meaning that the maximization problem is infeasible ($\min = \max = -\infty$).

So how we construct a valid inequality? Imposing the constraint $c^T x \geq c_0$ for each $x \in P$ in the maximization problem, requires a total number of constraints that is astronomically large even if we consider only the vertices of P . For this reason we have to algebraically characterize the valid inequalities of P . So the question now is: under which conditions a pair $(c, c_0) \in \mathbb{R}^{n+1}$ defines an inequality $c^T x \geq c_0$ that is valid for a given $P := \{x \geq 0 : Ax = b\}$? (See the numerical example in the book, page 66).

In general, starting from a system $Ax = b$ and choosing any $u \in \mathbb{R}^m$, we have the equation

$$u^T Ax = u^T b$$

from which we can obtain valid inequalities $c^T x \geq c_0$ with

$$c^T \geq u^T A \quad (\text{increase of coefficients on the left-hand side})$$

$$c_0 \leq u^T b \quad (\text{reduction of the right-hand side})$$

An interesting question is whether there are any valid inequalities that cannot be obtained using this method. Farkas' Lemma provides the answer to this question.

Theorem 8 (Farkas' Lemma). *The inequality $c^T x \geq c_0$ is valid for the non-empty polyhedron $P := \{x \geq 0 : Ax = b\}$ if and only if $u \in \mathbb{R}^m$ exists such that*

$$c^T \geq u^T A \tag{3.1}$$

$$c_0 \leq u^T b \tag{3.2}$$

Proof. First consider the case " $\Leftarrow$ ". Take any point $x \in P$. We know that $c^T \geq u^T A$ and $x \geq 0$, so we can write

$$c^T x \geq u^T Ax \stackrel{Ax=b}{=} u^T b \geq c_0 \Rightarrow c^T x \geq c_0.$$

Now consider the case " $\Rightarrow$ ". We define a linear problem

$$z^* = \min\{c^T x : Ax = b, x \geq 0\}.$$

We know that the problem is feasible because the polyhedron P is non-empty. Also, by hypothesis $c^T x \geq c_0$, so it must be that $z^* \geq c_0$ (c_0 is a finite number otherwise we cannot satisfy the inequality $c^T x \geq c_0$) $\Rightarrow z^*$ cannot be $-\infty$ (i.e. the problem is not unbounded). So we can use the simplex algorithm with the Bland's rule (remember that this version converges to a solution in a finite number of iteration), we optimize the problem and we find a basis B . Using the formula (see Definition 10) for the reduced cost associated to a basis B , we have

$$c^T - \underbrace{c_B^T B^{-1} A}_{:=u^T} \geq 0^T \quad \Rightarrow \quad c^T \geq u^T A$$

so the 3.1 is true. Using the observation on z^* we have

$$\begin{aligned} c_0 \leq z^* = c^T x^* &= [c_B^T, c_F^T] \begin{bmatrix} x_B^* \\ x_F^* \end{bmatrix} = [c_B^T, c_F^T] \begin{bmatrix} B^{-1}b \\ 0 \end{bmatrix} = c_B^T B^{-1}b = u^T b \\ &\Rightarrow c_0 \leq u^T b. \end{aligned}$$

so the 3.2 is verified. □

3.2 Dual problem

The considerations above show that, if problem $\min\{c^T x : Ax = b, x \geq 0\}$ has an optimal finite value, then:

$$\begin{aligned} \min_x \{c^T x : Ax = b, x \geq 0\} &= \max_{c_0} \{c_0 : c^T x \geq c_0 \text{ valid for } P\} \\ &= \max_{c_0, u} \{c_0 : c_0 \leq u^T b, c^T \geq u^T A\} \quad (\text{by Farkas' lemma}) \\ &= \max_u \{u^T b : c^T \geq u^T A\} \quad (\text{since, at the optimal solution, } c_0 = u^T b). \end{aligned}$$

Problem:

$$\left\{ \begin{array}{ll} \min & c^T x \\ & Ax = b \\ & x \geq 0 \end{array} \right. \quad \left\{ \begin{array}{ll} \max & u^T b \\ & c^T \geq u^T A \end{array} \right.$$

are called primal and dual problem, respectively. They are characterized by the same data (matrix A and vectors c and b) which, however, play different roles in the two problems. Note

that the dual variables u_i are free (they are multipliers for the equations $a_i^T x = b_i$). In addition, the constraints of the dual are inequalities (this is due to the fact that $x_j \geq 0$ implies that the inequality $u^T A x \geq u^T b$ remains valid even increasing coefficient $u^T A_j$).

$$\left\{ \begin{array}{ll} \min & c^T x \\ & Ax \geq b \\ & x \geq 0 \end{array} \right. \equiv \left\{ \begin{array}{ll} \min & c^T x + 0^T s \\ & Ax - Is = b \\ & x, s \geq 0 \end{array} \right. \Rightarrow \left\{ \begin{array}{ll} \max & u^T b \\ & c^T \geq u^T A \\ & 0^T \geq u^T (-I) \end{array} \right.$$

i.e.:

$$\left\{ \begin{array}{ll} \min & c^T x \\ & Ax \geq b \\ & x \geq 0 \end{array} \right. \Rightarrow \left\{ \begin{array}{ll} \max & u^T b \\ & c^T \geq u^T A \\ & u \geq 0 \end{array} \right.$$

Note that multipliers u_i are no longer free, since primal constraints of the type " $\geq$ " had to be combined. This is because:

$$u^T A \geq u^T b \quad \text{from which we obtain valid inequalities } c^T x \geq c_0 \text{ with } \begin{array}{l} c^T \geq u^T A \\ c_0 \leq u^T b \end{array}$$

in order to be valid inequalities we must have $u \geq 0$.

In we had $Ax \leq b$ then:

$$u^T A \leq u^T b \quad \text{from which we obtain valid inequalities } c^T x \geq c_0 \text{ with } \begin{array}{l} c^T \geq u^T A \\ c_0 \leq u^T b \end{array}$$

and in order to be valid inequalities we must have $u \leq 0$.

Primal (min)	Dual (max)
$\mathbf{a}_i^T \mathbf{x} \geq b_i$	$u_i \geq 0$
$\mathbf{a}_i^T \mathbf{x} \leq b_i$	$u_i \leq 0$
$\mathbf{a}_i^T \mathbf{x} = b_i$	u_i free
$x_j \geq 0$	$\mathbf{u}^T \mathbf{A}_j \leq c_j$
$x_j \leq 0$	$\mathbf{u}^T \mathbf{A}_j \geq c_j$
x_j free	$\mathbf{u}^T \mathbf{A}_j = c_j$

Figure 3.1: From primal to dual.

If we have $x \leq 0$, then we have to change the 3.1 in the Farkas' Lemma. So we can prove that "if $u \in \mathbb{R}^m$ exists such that $c^T \leq u^T A$, $c_0 \leq u^T b \Rightarrow$ the inequality $c^T x \geq c_0$ is valid for the non empty polyhedron $P := \{x \leq 0 : Ax = b\}$ ".

Multiplying $c^T \leq u^T A$ by $x \leq 0$ we obtain

$$c^T x \geq u^T A x = u^T b \geq c_0 \quad \Rightarrow \quad c^T x \geq c_0.$$

Note that we could simply define the polyhedron as $P := \{-x \geq 0 : Ax = b\}$, so the Farkas' Lemma remains valid as it is.

3.2.1 Fundamental Properties

Proposition 2. *The dual of the dual problem coincides with the primal problem.*

Proof. Without loss of generality we can consider the primal problem in canonical form.

$$\begin{cases} \min c^T x \\ Ax \geq b \\ x \geq 0 \end{cases} \Rightarrow \begin{cases} \max u^T b \\ c^T \geq u^T A \\ u \geq 0 \end{cases} \equiv \begin{cases} -\min -b^T u \\ (-u^T A)^T \geq (-c^T)^T \\ u \geq 0 \end{cases} \rightsquigarrow (-A^T)u \geq (-c)$$

Now we we can compute the dual of the dual

$$\begin{cases} -\min -b^T u \\ (-u^T A)^T \geq (-c^T)^T \\ u \geq 0 \end{cases} \Rightarrow \begin{cases} -\max -y^T c \\ -b^T \geq y^T (-A^T) \\ y \geq 0 \end{cases} \equiv \begin{cases} \min c^T y \\ y^T A^T \geq b^T \\ y \geq 0 \end{cases} \equiv \begin{cases} \min c^T y \\ Ay \geq b \\ y \geq 0 \end{cases}$$

□

Theorem 9 (Strong Duality). *Let $P := \{x \geq 0 : Ax \geq b\} \neq \emptyset$ with $\min\{c^T x : x \in P\}$ finite. Then:*

$$\min\{c^T x : x \in P\} = \max\{u^T b : c^T \geq u^T A, u \geq 0\}.$$

We have already proved this theorem in the previous two sections. Note that this theorem only works if the primal problem is feasible.

Theorem 10 (Weak Duality). *Let $P := \{x \geq 0 : Ax \geq b\} \neq \emptyset$ and $D := \{u \geq 0 : c^T \geq u^T A\} \neq \emptyset$. For all pairs of points of $\bar{x} \in P$ and $\bar{u} \in D$ we have that:*

$$\bar{u}^T b \leq c^T \bar{x}$$

Proof. We have that:

$$\bar{u}^T b \stackrel{\bar{x} \in P \Rightarrow A\bar{x} \geq b, \bar{u} \geq 0}{\leq} \bar{u}^T A\bar{x} \stackrel{\bar{x} \geq 0, c^T \geq \bar{u}^T A}{\leq} c^T \bar{x}$$

□

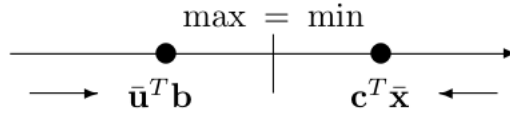


Figure 3.2: All pairs for points $\bar{x} \in P$ and $\bar{u} \in D$ verify $\bar{u}^T b \leq c^T \bar{x}$. The difference between the two value is called **dual gap**.

Dual \ Primal	finite optimum	infeasible	unbounded
finite optimum	✓	X	X
infeasible	X	✓	✓
unbounded	X	✓	X

Table 3.1: Note that the table is symmetrical and this is because of Proposition 2.

The case "The primal problem is unbounded and the dual is infeasible" and "The dual is unbounded and the primal is infeasible" can be easily proved using the weak duality: the existence of a feasible solution in one of the two problem, prevents the other from being unbounded. Therefore, if the primal is unbounded, weak duality cannot hold, implying that the dual is infeasible. The same reasoning applies if the dual is unbounded.

To prove that the statement "Both problem are infeasible" holds, we only have to provide an example:

$$\left\{ \begin{array}{l} \min \quad -4x_1 - 2x_2 \\ \quad -x_1 + x_2 \geq 2 \rightsquigarrow x_1 - x_2 \leq -2 \\ \quad x_1 - x_2 \geq 1 \\ \quad x_1, x_2 \geq 0 \end{array} \right.$$

$$\left\{ \begin{array}{l} \max \quad 2u_1 + u_2 \\ \quad -u_1 + u_2 \leq -4 \rightsquigarrow u_1 - u_2 \geq 4 \\ \quad u_1 - u_2 \leq -2 \\ \quad u_1, u_2 \geq 0. \end{array} \right.$$

as we can see, both primal and dual are infeasible.

3.3 Optimality Conditions

Consider again a primal-dual pair in canonical form $\min\{c^T x : Ax \geq b, x \geq 0\}$ and $\max\{u^T b : c^T \geq u^T A, u \geq 0\}$. From the considerations above, we have that two vectors $\bar{x} \in \mathbb{R}^n$ and $\bar{u} \in \mathbb{R}^m$ are optimal for the corresponding problems if and only if:

- (1) $A\bar{x} \geq b$ (*primal feasibility*)
- (2) $c^T \geq \bar{u}^T A$ (*dual feasibility*)
- (3) $c^T \bar{x} = \bar{u}^T b$ (*equality of the values of the objective function: min = max*)

So if we have a pair of value $\bar{x}$ and $\bar{u}$, if all the conditions above are satisfied, then we can say that we have found an optimal solution.

Note that the condition (3) (an equality between two scalar values) together with the other, implies a nonlinear system of $n + m$ equations in $n + m$ variables $\bar{x}_1, \dots, \bar{x}_n$ and $\bar{u}_1, \dots, \bar{u}_m$.

We can obtain the system as follows: from (1) and (2)

$$\begin{aligned} \bar{u}^T b &\leq u^T A\bar{x} \leq c^T \bar{x} \\ \text{using (3)} \Rightarrow \bar{u}^T b &= u^T A\bar{x} = c^T \bar{x} \Rightarrow \\ \Rightarrow \begin{cases} (3.\alpha) \bar{u}^T b = \bar{u}^T A\bar{x} \\ (3.\beta) c^T \bar{x} = \bar{u}^T A\bar{x} \end{cases} &\Rightarrow \begin{cases} (3.\alpha) \bar{u}^T (A\bar{x} - b) = 0 \\ (3.\beta) (c^T - \bar{u}^T A)\bar{x} = 0 \end{cases} \end{aligned}$$

Note that in (3. α), the product between two vectors of size m must equal zero, implying that these vectors in the dual space (i.e., $\mathbb{R}^m$) must be orthogonal. On the other hand, in (3. β), the two vectors in the primal space (i.e. $\mathbb{R}^n$) must be orthogonal.

Writing (3. α) in full we obtain:

$$\sum_{i=1}^m \bar{u}_i (a_i^T \bar{x} - b_i) = 0 \Rightarrow \begin{cases} \bar{u}_1 (a_1^T \bar{x} - b_1) &= 0 \\ \vdots \\ \bar{u}_m (a_m^T \bar{x} - b_m) &= 0. \end{cases} \quad (3.3)$$

m inequalities in m variables $\bar{u}_1, \dots, \bar{u}_m$. Note that: $\bar{u} \geq 0$ and by condition (1) we have¹ the $a_i^T \bar{x} - b_i \geq 0$. So in order to satisfy the equation, either $\bar{u}_i = 0$ or $a_i^T \bar{x} - b_i = 0$ or both².

¹Remember the notation: a_i^T is i -th row of A .

²If the linear problem has a constraint like $a_i^T x = b_i$ then if $\bar{x}$ is feasible $\Rightarrow a_i^T \bar{x} - b_i = 0$ so in this case $\bar{u}_i$ can

Proceeding in the same way, from (3.β) we obtain:

$$\sum_{j=1}^n (c_j - \bar{u}^T A_j) \bar{x}_j = 0 \Rightarrow \begin{cases} (c_1 - \bar{u}^T A_1) \bar{x}_1 = 0 \\ \vdots \\ (c_n - \bar{u}^T A_n) \bar{x}_n = 0. \end{cases} \quad (3.4)$$

n equations in n variables $\bar{x}_1, \dots, \bar{x}_n$.

Conditions 3.3 and 3.4 are called the *complementary conditions* and are interpreted as follows. Let $\bar{x}$ and $\bar{u}$ be the optimal solutions of the primal and dual problems, respectively:

- the dual variable $\bar{u}_i$ corresponding to a "loose" constraint ($a_i^T \bar{x} > b_i$) must be equal to zero;
- the reduced cost $\bar{c}_j := c_j - \bar{u}^T A_j$ corresponding to a variable $\bar{x}_j > 0$ must be equal to zero.

Hence we have the important:

Theorem 11 (Optimality Conditions). *Two vectors $\bar{x} \in \mathbb{R}^n$ and $\bar{u} \in \mathbb{R}^m$ are optimal for problems $\min\{c^T x : Ax \geq b, x \geq 0\}$ and $\max\{u^T b : c^T \geq u^T A, u \geq 0\}$, respectively, if and only if the following optimality conditions hold:*

$$(1) \quad A\bar{x} \geq b, \quad \bar{x} \geq 0 \quad (\text{primal feasibility}) \quad (3.5)$$

$$(2) \quad c^T \geq \bar{u}^T A, \quad \bar{u} \geq 0 \quad (\text{dual feasibility}) \quad (3.6)$$

$$(3.\alpha) \quad \bar{u}^T (A\bar{x} - b) = 0 \quad (\text{complementary slackness conditions 1}) \quad (3.7)$$

$$(3.\beta) \quad (c^T - \bar{u}^T A)\bar{x} = 0 \quad (\text{complementary slackness condition 2}) \quad (3.8)$$

So in theory if we want to find a pair $(\bar{x}, \bar{u})$ where $\bar{x} \in \mathbb{R}^n$ is an optimal solution for the primal and $\bar{u} \in \mathbb{R}^m$ is an optimal solution for the dual, we could "simply" merge the systems 3.3 and 3.4 to obtain a system with $n + m$ (nonlinear) equations in $n + m$ variables.

3.4 Analysis of the Simplex Algorithm

It is interesting to interpret the simplex algorithm in the light of the optimality conditions just obtained. Consider a linear programming problem in the standard form $\min\{c^T x : Ax = b, x \geq 0\}$; the optimality condition become the following:

be any number ≥ 0 . In the other hand consider the case $a_i^T \bar{x} \geq 0$ and consider $\bar{x}$ feasible such that $a_i^T \bar{x} - b_i > 0$. In this last case $\bar{u}_i$ must be 0.

$$(1') \quad A\bar{x} = b, \bar{x} \geq 0;$$

$$(2') \quad c^T \geq \bar{u}^T A, \bar{u}^T \text{ free};$$

$$(3') \quad (c^T - \bar{u}^T A)\bar{x} = 0.$$

The condition (3.α) is always satisfied by the (1') condition, so we only consider (3.β) which we have renamed in (3').

The simplex algorithm, at each iteration, identifies a feasible basis B of matrix $A = [B \mid F]$ and compute the following vectors:

- $\bar{x} = (x_B, x_F) = (B^{-1}b, 0) \geq 0;$
- $\bar{u}^T := c_B^T B^{-1}$ (we claim that this is the dual solution);
- $\bar{c}^T := c^T - \bar{u}^T A$ (the reduced costs).

The algorithm stops when $\bar{c} \geq 0$. Note that, at each iteration we have a pair $(\bar{x}, \bar{u})$.

By construction, at each iteration the current basic solution $\bar{x} \geq 0$ satisfies the primal feasibility condition (1'), since $A\bar{x} = B\bar{x}_B + F\bar{x}_F = BB^{-1}b + 0 = b$. Also the condition (3') is always satisfied:

$$\begin{aligned} (c^T - \bar{u}^T A)\bar{x} &= ([c_B, c_F]^T - c^T B^{-1}[B, F]^T)[\bar{x}_B, \bar{x}_F]^T \\ &= \underbrace{(c_B^T - c_B^T B^{-1}B)}_{=0^T} \bar{x}_B + (c_F^T - c_B^T B^{-1}F) \underbrace{\bar{x}_F}_{=0} = 0 \end{aligned}$$

In other words, the condition (3') is always satisfied because the reduced costs of the basic variables are always 0 and because the non-basic variables are always zero. The condition (2') is satisfied only when we reach the optimal solution, that is, in the last iteration when all reduced costs are greater or equal to zero ($\bar{c}^T := c^T - \bar{u}^T A \geq 0^T$, i.e. the optimality condition for the primal problem, see Theorem 6.).

Summarizing: at each iteration of the simplex algorithm we have a (primal) feasible solution $\bar{x}$ such that (1') is satisfied; we compute $\bar{u}$ such that (3') is satisfied; we perform a test to verify whether (2') is satisfied. At the last iteration (2') is verified, hence $\bar{u}$ becomes feasible for the dual problem; this guarantees the optimality of $\bar{x}$ and $\bar{u}$ for the respective problems.

3.5 Dual Simplex Algorithm

In the dual Simplex algorithm, at each iteration, the condition (2') and (3') are always satisfied, while the condition (1') is only met at the last iteration. Geometrically, the method starts from an infeasible "super-optimal" (if it were feasible, it would be optimal) basic solution x , and proceeds iteratively approaching the feasible region.

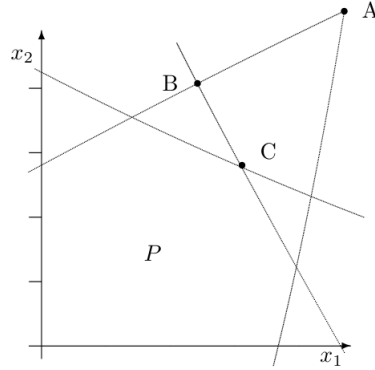


Figure 3.3: The points A and B are "super-optimal" solutions, the point C is the (primal) optimal solution.

The dual simplex algorithm needs an initial tableau in canonical form of the following type:

		x_1	$\dots$	x_t	$\dots$	x_m	x_{m+1}	$\dots$	x_h	$\dots$	x_n
$-z$	$\bar{c}_0$	0	$\dots$	0	$\dots$	0	$\bar{c}_{m+1}$	$\dots$	$\bar{c}_h$	$\dots$	$\bar{c}_n$
x_1	$\bar{b}_1$	1	$\dots$	0	$\dots$	0	$\bar{a}_{1,m+1}$	$\dots$		$\dots$	$\bar{a}_{1n}$
$\dots$	$\dots$	0		0		0	$\dots$	$\dots$		$\dots$	
x_t	$\bar{b}_t$	0		1		0	$\bar{a}_{t,m+1}$	$\dots$	$\bar{a}_{th}$	$\dots$	$\bar{a}_{tn}$
$\dots$	$\dots$	0		0		0	$\dots$	$\dots$		$\dots$	
x_m	$\bar{b}_m$	0	$\dots$	0	$\dots$	1	$\bar{a}_{m,m+1}$	$\dots$		$\dots$	$\bar{a}_{mn}$

In order to satisfy (at each iteration) the condition (2') (the dual feasibility), it is necessary that the reduced costs $\bar{c}_1, \dots, \bar{c}_n$ in row 0 are all greater than or equal to zero. This is the **invariant** of the algorithm.

If the values $\bar{b}_1, \dots, \bar{b}_m$ are all greater than or equal to zero, the **tableau is optimal**. Let's suppose that a value $\bar{b}_t < 0$ exists (so the condition (1') (primal feasibility) is not satisfied). There are two possibilities:

- $\bar{a}_{tj} \geq 0 \forall j = 1, \dots, n$: in this case we will have the following equation associated to row t :

$$\underbrace{\bar{b}_t}_{<0} = \sum_{j=1}^n \underbrace{\bar{a}_{tj}}_{\geq 0} \underbrace{x_j}_{\geq 0}$$

which is impossible: it doesn't exist any $x_j \geq 0$ that satisfies the equation. In other words we cannot find a value of $x_{\beta[t]} \geq 0$ and that means that we cannot find a primal feasible solution $\Rightarrow$ the **primal problem is infeasible**³.

- A term $\bar{a}_{th} < 0$ exists. In this case we can perform a pivot operation in position (t, h) , hence we obtain a new term (in the updated tableau) $\tilde{b}_t$ which is positive.

Lets suppose that we are in the second case, hence it exists a term $\bar{a}_{th} < 0$. As we have seen in previous chapters, the pivot operation will also update the values in row 0. The new row 0 is computed as follows⁴:

$$\tilde{c}_0 := \bar{c}_0 - \frac{\bar{c}_h}{\bar{a}_{th}} \bar{b}_t$$

$$\tilde{c}_j := \bar{c}_j - \frac{\bar{c}_h}{\bar{a}_{th}} \bar{a}_{tj}, \quad j = 1, \dots, n.$$

In order to satisfy the dual feasibility, we must have that all the reduced costs $\tilde{c}_j \geq 0$. So we have to impose:

$$\tilde{c}_j \geq 0 \quad \Rightarrow \quad \bar{c}_j \geq \frac{\bar{c}_h}{\bar{a}_{th}} \bar{a}_{tj} = \frac{\bar{c}_h}{-|\bar{a}_{th}|} \bar{a}_{tj} \quad (\text{given that } \bar{a}_{th} < 0) \quad j = 1, \dots, n$$

If $\bar{a}_{tj} \geq 0$ the inequality is surely verified because, by hypothesis $\bar{c}_j \geq 0$ and $\bar{c}_h \geq 0$. If instead $\bar{a}_{tj} < 0$ then:

$$\bar{c}_j \geq \frac{\bar{c}_h}{-|\bar{a}_{th}|} (-|\bar{a}_{tj}|) \quad \Rightarrow \quad \frac{\bar{c}_j}{|\bar{a}_{tj}|} \geq \frac{\bar{c}_h}{|\bar{a}_{th}|}$$

applies.

It is therefore not possible to arbitrarily choose the pivot element $\bar{a}_{th} < 0$ in row t , but this term has to minimize the ratio $\bar{c}_j/|\bar{a}_{tj}|$ for all values $\bar{a}_{tj} < 0$ (i.e. all the pivot candidates), i.e.:

$$h := \arg \min \left\{ \frac{\bar{c}_j}{|\bar{a}_{tj}|} : j \in \{1, \dots, n\}, \bar{a}_{tj} < 0 \right\}.$$

As a result of the pivot operation, the value of the objective function computed in correspondence of the current basic solution changes from $z = -\bar{c}_0$ to $\tilde{z} = -\tilde{c}_0$, hence it

³Note that: we're working on the space of the variables x ; the dual problem is solved implicitly in our model, so the primal is infeasible, not the dual.

⁴Is the same formula that we have used in the primal simplex algorithm.

increases (so it *gets worse*) by the quantity $\tilde{z} - z \geq 0$.

Using the **Bland's rule**, the dual simplex algorithm converges to an optimal solution.

Note that in the primal simplex method, we first select the entering variable x_h and then determine the variable $x_{\beta[t]}$ that leaves the basis. In the dual simplex method, the order is reversed: we start by selecting the variable $x_{\beta[t]}$ that will leave the basis, followed by the entering variable x_h . Another difference between the two algorithms is in the selection criteria: in the primal simplex, x_h is chosen based on negative reduced costs, whereas in the dual simplex, $x_{\beta[t]}$ is chosen from the current basic variables with negative values.

3.5.1 Dual simplex vs primal simplex

Dual simplex is much better than primal simplex: in practice is faster.

Also the dual simplex algorithm is very useful when we deal with *dynamic generation of inequalities*. Consider a linear programming problem with a lot of inequalities (in 2 dimension). In a sense, we know that only two inequalities are useful because we need only two of them to identify an optimal vertex. The problem is that we don't know which pair identifies such vertex.

Instead of solving directly the original problem, we can proceed in the following way:

1. Choose a few constraints from the original problem (called P). Solve the problem considering only the previously chosen constraints using the (primal or dual) simplex algorithm (we call this new problem P').
2. Check if the optimal solution found verifies all the other constraints.
 - If all the constraints are satisfied $\Rightarrow$ we have found an optimal solution for P .
 - If it exists a constraint that is not satisfied, then add it to P' .
3. We can use the dual on P' to restore the primal feasibility. Before starting the dual simplex we need to add to the tableau a new row (the new inequality) and a new column (for the new slack variable). The new tableau is not in canonical form, so we need to perform a pivot operation for every old basic variables (i.e. the basic variables of the previous tableau). Now we can apply the dual simplex to find an optimal solution for P' and then we go back to step (2).

Note that in the worst case this algorithm adds all the constraints of P in P' . The dynamic generation of inequalities is a very useful technique when we deal with very big problems.

An example can be found on page 80 of the book.

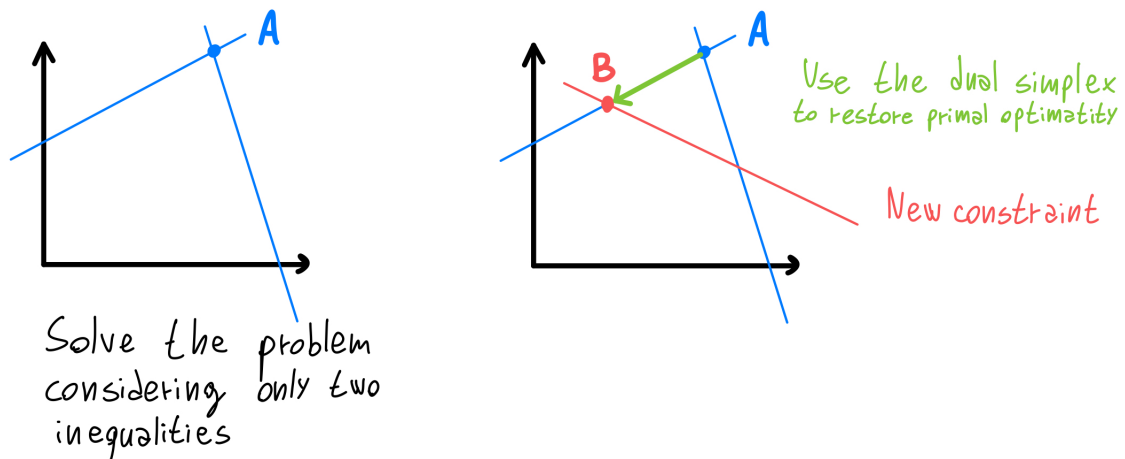


Figure 3.4: Dynamic generation of inequalities.

3.6 Sensitivity analysis

Once we have found a solution for a linear problem, it's often interesting to evaluate the *stability* of this solution with respect to changes in the problem data. The model of the LP problem, is usually deemed more reliable if its solution are less sensitive to changes in the data (often obtained with measurements affected by error).

When we solve a linear problem we define c , A , b and we get in output an optimal basis B . So if we consider the following variation on the data: $c + \Delta c$, $b + \Delta b$; then B is still the optimal basis? Note that we do not consider the variation on the matrix A (i.e. $A + \Delta A$) because the change in that case might be too strong, that is, the problem changes too much.

As we have seen in previous section, B is optimal if and only if it verifies all the optimality

conditions (see theorem 11). Those conditions, written for basis B become the following:

$$(c.1) \quad B^{-1}b \geq 0 \quad (\text{primal feasibility or } \bar{x})$$

$$(c.2) \quad \bar{c}^T := c^T - \underbrace{c_B^T B^{-1} A}_{:= \bar{u}^T} \geq 0 \quad (\text{dual feasibility for } \bar{u})$$

Sensitivity analysis is the study of the perturbations to the initial data whereby conditions (c.1) and (c.2) remain verified. This study makes it possible to define the necessary and sufficient conditions such that basis B remains feasible and optimal when changing data: i.e., the conditions refer to basis B and not to the corresponding basic solution⁵ $\bar{x}$. As we have said earlier, we will consider the following perturbations:

- Changes in the right-hand sides: $b \rightarrow b + \Delta b$
- Changes in the costs of basic variables: $c_B^T \rightarrow c_B^T + \Delta c_B^T$
- Changes in the costs of non-basic variables: $c_F^T \rightarrow c_F^T + \Delta c_F^T$

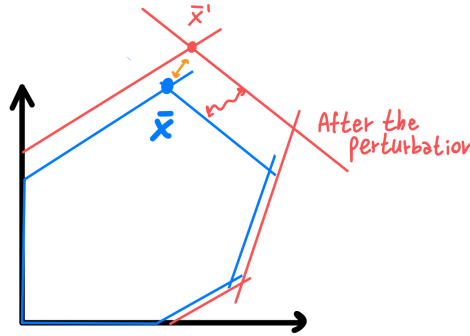


Figure 3.5: The coordinates of the basic solution changes (so does the cost of the optimal solution), but the basis B remains optimal.

Changes in the right hand side The optimality conditions (c.1) and (c.2) for basis B become:

$$(c.2) \quad \bar{c}^T := c^T - c_B^T B^{-1} A \geq 0 \quad (\text{unchanged})$$

$$(c.1) \quad B^{-1}(b + \Delta b) \geq 0 \Rightarrow B^{-1} \Delta b \geq -B^{-1}b$$

Thus, the basis B remains optimal if and only if $B^{-1} \Delta b \geq -B^{-1}b$. This is a system of m inequalities in m variables Δb_i : we can draw a polyhedron in $\mathbb{R}^m$ containing vectors Δb for

⁵In other words: the coordinates of the optimal vertex can change but the basis remains the same.

which the optimal basis does not change.

Note that, when b changes $\Rightarrow$ the coordinates of the corresponding basic solution $\hat{x}$ changes $\Rightarrow$ the optimal value of the objective function changes

$$\text{from } c_B^T B^{-1} b \rightsquigarrow \text{to } c_B^T B^{-1} (b + \Delta b)$$

So we can define the variation on the costs as follows:

$$\Delta z := c_B^T B^{-1} (b + \Delta b) - c_B^T B^{-1} b = \underbrace{c_B^T B^{-1}}_{\bar{u}^T} \Delta b$$

$$\Rightarrow \Delta z = \bar{u}^T \Delta b = \sum_{i=1}^m \bar{u}_i \cdot \Delta b_i.$$

Interesting interpretation: looking the definition of Δz we can say that

- If $\bar{u}_i = 0 \rightarrow$ we can increase Δb_i as much as we want and the costs will remain the same⁶.
In other words, the constraint associated with Δb_i is "irrelevant" to the problem.
- If $|\bar{u}_i|$ is large $\rightarrow$ small variation on Δb_i will increase a lot the costs.
- If $|\bar{u}_i|$ is small $\rightarrow$ big variation on Δb_i will increase the costs only a few.

In a sense, the largest $|\bar{u}_i|$ the more relevant is the constraint associated to Δb_i .

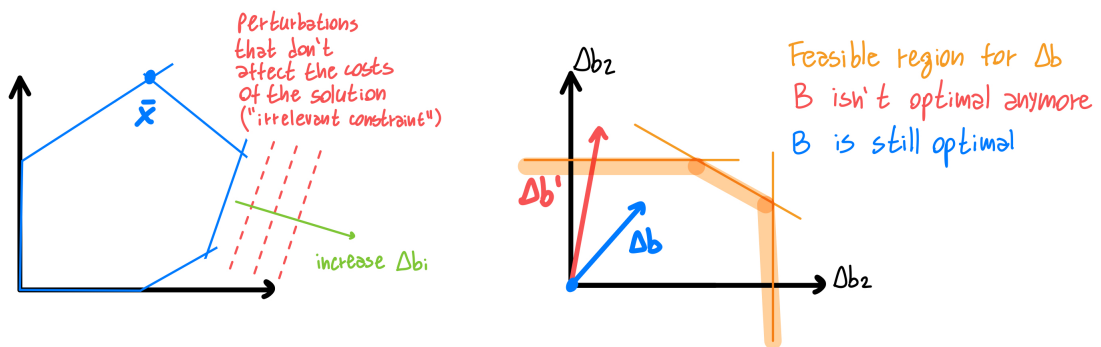


Figure 3.6

⁶The value of $\bar{u}_i$ is surely 0 if it is associated with a "loose" constraint (see Section 3.3).

Changes in the costs of non-basic variables The optimality conditions (c.1) and (c.2) for basis B become:

$$(c.1) \quad B^{-1}b \geq 0 \quad (\text{unchanged})$$

$$(c.2) \quad \bar{c}^T := c^T - c_B^T B^{-1}A \geq 0 \Rightarrow [\bar{c}_B^T, \bar{c}_F^T] = [0^T, \bar{c}_F^T] \geq 0$$

$$\begin{aligned} \bar{c}_F^T &:= (c_F + \Delta c_F)^T - c_B^T B^{-1}F = \bar{c}_F^T + \Delta c_F^T \geq 0 \\ &\Rightarrow \Delta c_F^T \geq -\bar{c}_F^T \end{aligned}$$

Thus, the basis B remains optimal if and only if $\Delta c_F^T \geq -\bar{c}_F^T$, where $\bar{c}_F^T$ are the "old" reduced costs. In this way, we obtain the $n - m$ inequalities, independent from each other:

$$I \Delta c_F^T \geq -\bar{c}_F^T$$

which is the same as:

$$\Delta c_j \geq -\bar{c}_j \quad \forall x_j \text{ non-basic}$$

Interpretation:

- If $|\bar{c}_j|$ is large $\rightarrow$ the variable x_j non-basic will not join the basis even if you decrease his reduced cost a lot.
- If $|\bar{c}_j|$ is small $\rightarrow$ a slight decrease in the reduced cost associated with the non-basic variable x_j could cause it to enter the basis, implying that B would no longer be optimal.

Of course you can always increase the value of the reduced costs of the non-basic variable and B still remains the optimal basis.

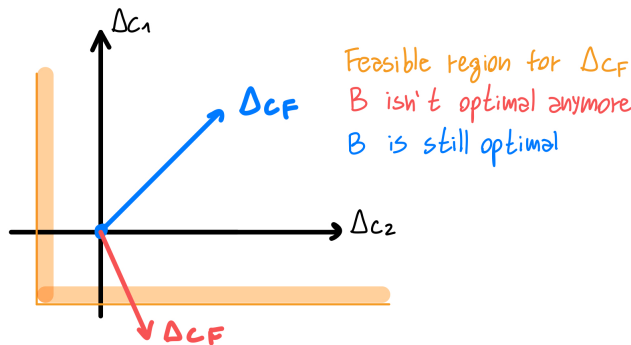


Figure 3.7

Changes in the costs of basic variables The optimality conditions (c.1) and (c.2) for basis B become:

$$(c.1) \quad B^{-1}b \geq 0 \quad (\text{unchanged})$$

$$(c.2) \quad \bar{c}^T := c^T - c_B^T B^{-1}A \geq 0 \Rightarrow [\tilde{c}_B^T, \tilde{c}_F^T] = [0^T, \tilde{c}_F^T] \geq 0$$

$$\tilde{c}_F^T := c_F^T - (c_B + \Delta c_B)^T B^{-1}F = \tilde{c}_F^T - \Delta c_B^T B^{-1}F \geq 0$$

$$\Rightarrow \Delta c_B^T B^{-1}F \leq \tilde{c}_F^T$$

Thus, the basis B remains optimal if and only if $\Delta c_B^T B^{-1}F \leq \tilde{c}_F^T$, where $\tilde{c}_F^T$ are the "old" reduced costs. In this way, we obtain the $n - m$ inequalities in m variables Δc_{Bi} , so we can draw a polyhedron in $\mathbb{R}^m$, whose points correspond to the vectors Δc_B for which the optimal basis does not change.

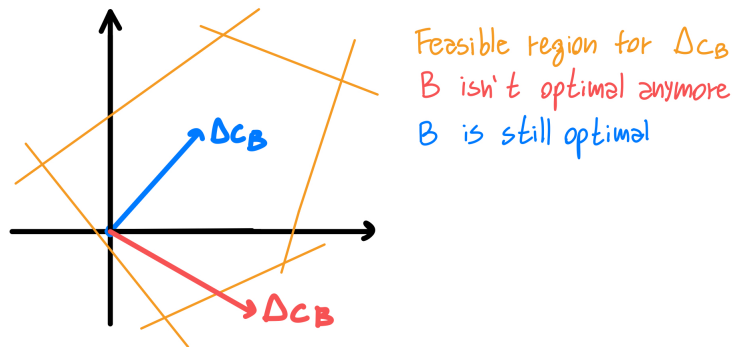


Figure 3.8

(See example page 84 of the book).

Chapter 4

Integer Linear Programming

An integer linear programming (ILP) problem, in canonical form, is defined as:

$$\begin{cases} z_{ILP} := \min c^T x \\ Ax \geq b \\ x \geq 0 \text{ integer} \end{cases}$$

where z_{ILP} is the optimal value of the objective function. Note that we have defined the integrality constraint using a word, but how can we do it in mathematical way? The integrality constraint on variables x_j for $j = 1, \dots, n$ is non linear and can be expressed as:

$$\sin(\pi x_j) = 0 \quad j = 1, \dots, n.$$

In some cases the integrality constraint is only applied on certain variables: in such cases we talk about *Mixed-Integer Linear Programming*.

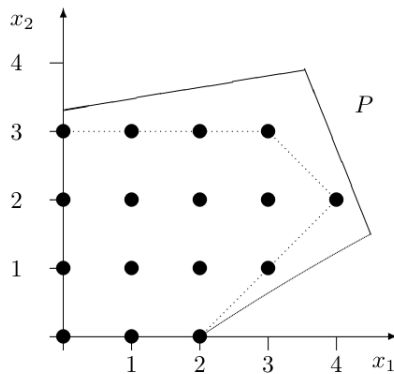


Figure 4.1: Sets P , X and $\text{conv}(X)$.

The integrality constraint on the variables defines a lattice of points on $\mathbb{R}^n$ (see the picture above in $\mathbb{R}^2$).

We will assume (with a little loss of generality) $P := \{x \geq 0 : Ax \geq b\}$ is bounded and non empty. Also we assume (A, b) is integer, this form can always be obtained with some multiplication. The (discrete and finite) set of feasible solutions of the ILP problem is indicated as

$$X := P \cap \mathbb{Z}^n.$$

Note that X is not a convex set because it's a discrete set $\Rightarrow$ local optimum $\neq$ global optimum (in general).

4.1 Equivalent Formulations

In order to try to solve a ILP we can do the following:

1. We eliminate the integrality constraint and solve the so called **linear relaxation** of the original problem.

$$\begin{cases} z_{LP} := \min c^T x \\ Ax \geq b \\ x \geq 0 \end{cases}$$

Note that $c^T x^* = z_{LP} \leq z_{ILP}$, where x^* is the optimal solution of the linear relaxation. This statement is trivially true because: $z_{LP} := \min\{x \geq 0 : x \in P\}$, $z_{ILP} := \min\{x \geq 0 : x \in X = P \cap \mathbb{Z}^n\}$, so $P \supseteq X$, i.e. P is bigger than X . In other words, z_{LP} is a **lower bound** for z_{ILP} .

2. If the optimal solution of the linear relaxation x^* is integer, than it is also optimal for the integer problem. This is trivially true and it follows from the previous observation:

$$c^T x^* = z_{LP} \leq z_{ILP} \stackrel{x^* \in X}{=} c^T x^* \Rightarrow z_{LP} = z_{ILP}.$$

Note that this procedure does not guarantee finding an optimal solution x^* for the ILP problem. This only occurs if the problem formulation has an optimal vertex that is also integer. Additionally, we observe that multiple formulations can define the same feasible region for a given ILP problem, meaning there are many equivalent formulations.

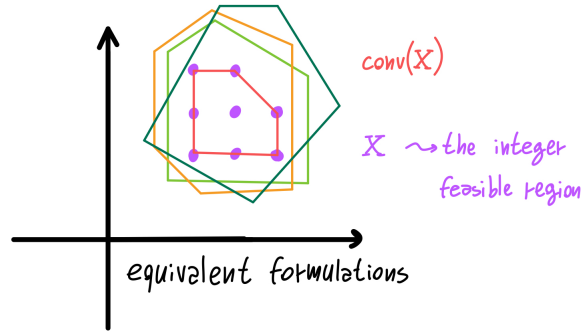


Figure 4.2: Equivalent formulations.

Mathematically speaking it means that the pair (A, b) is not unique: we can find many pairs that identify the same feasible region. The resulting formulations are equivalent while the integrality constraint is maintained, but they have linear relaxations which give lower bounds z_{LP} very different from each other. Theoretically, there also exists an **ideal formulation** (the “tightest” possible) whose relaxation P has only integer vertices, thus guarantying $z_{LP} = z_{ILP}$ for all linear objective functions.

Definition 13 (Convex hull). *Given a set $S \subseteq \mathbb{R}^n$, a convex hull of S is the smallest convex set $\text{conv}(S)$ that contains S .*

In Figure 4.1 the dotted line identifies the convex hull of X , i.e. $\text{conv}(X)$ is a polytope $\tilde{P}$ whose vertices are all integer points. There exist thus $\tilde{A}$ and $\tilde{b}$ such that $\tilde{P} := \{x \geq 0 : \tilde{A}x \geq \tilde{b}\} = \text{conv}(X)$, hence guarantying

$$\min\{c^T x : x \in X\} = \min\{c^T x : \tilde{A}x \geq \tilde{b}, x \geq 0\}$$

for all cost vectors c . Each ILP problem is then equivalent to an LP problem, hence it can be solved, for instance, using the simplex algorithm. Unfortunately $\tilde{A}$ and $\tilde{b}$ are often very difficult to determine and they define a system $\tilde{A}x \geq \tilde{b}$ containing a huge number of constraints, hence this solution strategy is impractical.

In general, in a space of n variables, we need at least 2^n inequalities in order to identify the ideal formulation. In the following section we will consider the case in which the original formulation is already the ideal formulation.

Question: when we solve the linear relaxation of a ILP problem, we get x^* as an optimal solution. If x^* is integer $\Rightarrow$ it is also optimal for the ILP. But if x^* is not integer, then why we can't just round the fractional values? In general the rounding of x^* isn't always a feasible

solution nor an optimal solution. Furthermore we can easily construct a polyhedron with no integer solutions. In such cases, rounding the fractional values of the optimal solution from the linear relaxation is futile, as no integer feasible points exist.

4.2 Total Unimodularity

Consider a ILP problem in standard form $z_{ILP} := \{c^T : Ax = b, x \geq 0 \text{ integer}\}$. (A, b) are assumed to be integer. What we want to do now, is to find the condition under which a generic feasible solution x^* of system $Ax = b$ has fraction components.

We have $x^* = [x_B, x_F]^T = [B^{-1}b, 0]^T$, where B is the basis associated with to x^* . $x_F = 0$ are all integer variables, hence x^* has fractional components if and only if $B^{-1}b$ is noninteger. Matrix B^{-1} can be computed as follows:

$$B^{-1} = \frac{1}{\det(B)} \begin{bmatrix} \alpha_{11} & \dots & \alpha_{1m} \\ \vdots & & \vdots \\ \alpha_{m1} & \dots & \alpha_{mm} \end{bmatrix}^T$$

where $\alpha_{ij} := (-1)^{i+j} \det(M_{ij})$ and M_{ij} is the submatrix obtained from B eliminating row i and column j (M_{ij} is called *minor matrix*). Since B is integer, the values α_{ij} are all integer. It follows that B^{-1} is integer if $\det(B) = \pm 1$ (if that is the case, B is called **unimodular matrix**). In this case, $B^{-1}b$ is integer as well, and the basic solution x^* cannot have fractional components.

So, if B is unimodular $\Rightarrow$ the vertex x^* is integer. Note that this condition is only sufficient, but not necessary for the integrality of $B^{-1}b$.

The lucky case is when all the basis of A are unimodular matrices $\Rightarrow$ all the vertex in the original formulation are integer $\Rightarrow$ the formulation is ideal.

Definition 14 (Unimodular). *An integer matrix A of size $m \times n$ with $m \leq n$ is said to be unimodular if for any¹ of its $m \times m$ submatrices B , $\det(B) \in \{-1, 0, 1\}$.*

Theorem 12. *Let A be unimodular and b an integer vector. Then $P := \{x \geq 0 : Ax = b\}$ (the original formulation) has only integer vertices.*

So far we have considered a ILP in standard form. What about the canonical form? Consider the following ILP in canonical form: $\min\{c^T x : Ax \geq b, x \geq 0 \text{ integer}\}$. We can introduce the

¹We are considering any submatrices, not only the all the possible basis B .

slack variables:

$$\min\{c^T x : Ax - Is = b, x \geq 0 \text{ integer}\}$$

in which the constraint matrix is $A' = [A, -I]$. So, every submatrix $m \times m$ of A' is obtained selecting k (say) columns from A , and $m - k$ columns from $-I$ ($0 \leq k \leq m$). For instance: if $m = 10$ we can select 2 columns from A , and 8 from $-I$; or 5 and 5; or 8 and 2; or etc. (all the possible combinations). After some permutation of the chosen columns, we can rearrange the matrix B in the following form:

$$B = \left[\begin{array}{c|c} -I' & F \\ \hline 0 & Q \end{array} \right]$$

$\underbrace{\hspace{10em}}$
columns from $-I$

$\underbrace{\hspace{10em}}$
columns from A

where I' is the identity matrix of order $m - k$. We have that $\det(B) = \pm \det(Q)$, hence A' is unimodular if and only if $\det(Q) \in \{-1, 1, 0\}$ for all squared submatrices Q of A , of any order.

Definition 15 (Totally unimodular). *An integer matrix A of size $m \times n$ is said to be totally unimodular (TUM) if $\det(Q) \in \{-1, 0, 1\}$ for any of its square submatrices Q of any order.*

Theorem 13. *Let A be totally unimodular and b an integer vector. Then $P := \{x \geq 0 : Ax \geq b\}$ (the original formulation) has only integer vertices.*

To summarize, using the last two theorems, we can say that:

- if the original problem is in standard form and A is unimodular $\Rightarrow$ the current formulation is actually the ideal formulation.
- if the original problem is in canonical form and A is totally unimodular $\Rightarrow$ the current formulation is actually the ideal formulation.

The problem now is: if we have a ILP problem $\min\{c^T x : Ax \geq b, x \geq 0 \text{ integer}\}$, how do we actually check if A is TUM? It exists an algorithm that checks whether the definition is TUM or not. However this algorithm is not efficient, it requires too much time.

First of all let us introduce some properties of TMU matrices that will be useful later.

Proposition 3. *If A is TUM $\Rightarrow$ every entry of the matrix $a_{ij} \in \{-1, 0, 1\}$.*

This is a necessary condition. It's trivially true: you can see by the definition of TUM that the it must be $\det(Q) \in \{-1, 0, 1\}$ for every squared submatrix Q of A (every a_{ij} is a 1×1 squared matrix).

Proposition 4. A is TUM $\Leftrightarrow A^T$ TUM.

Proposition 5.

$$A \text{ is TUM} \Leftrightarrow \left[\begin{array}{c|c} 0 & A \\ \hline 0 & \end{array} \right] \text{ is TUM.}$$

This is because, if you consider the edge case in which the squared matrix Q has some element of the column with only 0's, then $\det(Q) = 0$ by definition.

$$Q = \left[\begin{array}{c|c} 0 & * \\ \hline 0 & * \end{array} \right] \Rightarrow \det(Q) = 0$$

Proposition 6.

$$A \text{ is TUM} \Leftrightarrow \left[\begin{array}{c|c} \pm 1 & A \\ \hline 0 & \end{array} \right] \text{ is TUM.}$$

The only edge case that we have to consider is the following:

$$Q = \left[\begin{array}{c|c} \pm 1 & * \dots * \\ \hline 0 & Q' \\ \vdots & \\ 0 & \end{array} \right] \Rightarrow \det(Q) = \pm \det(Q')$$

In order to test if A is TUM we can use the following theorem (that is sufficient but not necessary).

Theorem 14. Let A be a matrix with $a_{ij} \in \{-1, 0, 1\} \forall i, j$. A is TUM if the following conditions hold:

1. every column of A has no more than two non-zero elements;
2. there exists a partition (I_1, I_2) of the rows of A such that each column with two non-zero elements has these two elements belonging to rows on different sets if and only if the two elements have the same sign.

We have to identify two partition I_1, I_2 of rows in order to satisfy the condition (see figure 4.3).

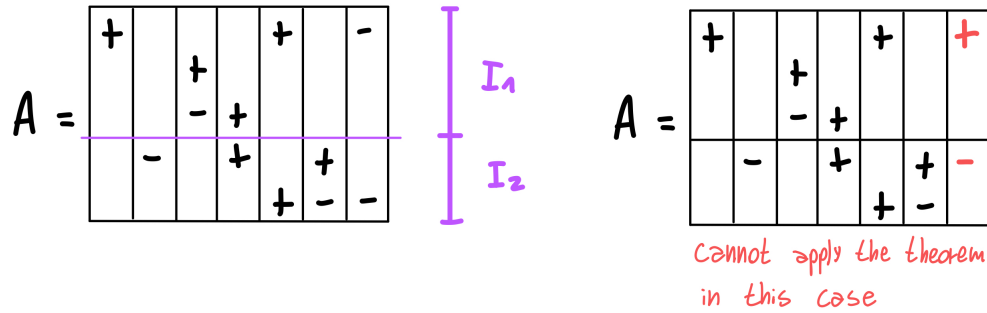


Figure 4.3: Conditions of the theorem. You can think about the + and - as electric charges: charges of same sign repel each other; charges of different sign attract each other.

Proof. We want to prove that $\det(Q) \in \{-1, 0, 1\}$ for any squared submatrix Q of A of order k ($k = 1, \dots, m$). We can prove the theorem by induction on the k parameter.

[B]: $k = 1$, Q is a 1×1 matrix and $\det(Q) \in \{-1, 0, 1\}$ because we know that every element $a_{ij} \in \{-1, 0, 1\}$.

[HP]: Suppose true for every matrix of size $k' \times k'$ with $k' < k$.

[HP $\Rightarrow$ TH]: Q is a $k \times k$ matrix. We have to consider 3 cases:

1. Q has one column of zeros $\Rightarrow \det(Q) = 0$ (see proposition 5).
2. Q has a column with only one element different from zero. In this case, barring permutations of rows and/or columns, Q is of the type:

$$Q = \left[\begin{array}{c|ccc} \pm 1 & * & \dots & * \\ \hline 0 & & & \\ \vdots & & & \\ 0 & & & \end{array} \right] \Rightarrow \det(Q) = \pm \det(Q')$$

Q' is a $(k-1) \times (k-1)$ matrix so, by HP, $\det(Q) = \pm \det(Q') \in \{-1, 0, 1\}$.

3. Every column of Q have exactly two non-zero elements. Let $I(Q)$ be the set of rows of Q . Q has the form shown in Figure 4.4.

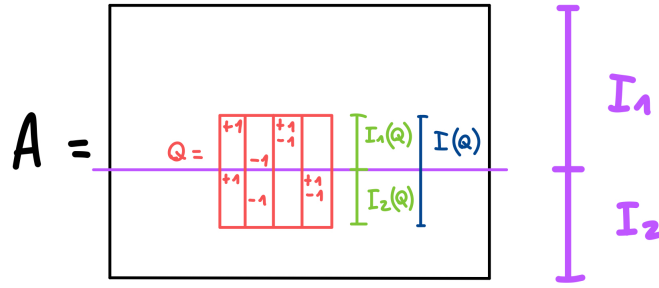


Figure 4.4: $I_1(Q) = I(Q) \cap I_1$; $I_2(Q) = I(Q) \cap I_2$.

So we can do the following:

$$\sum_{i \in I_1(Q)} [\text{row } i \text{ of } Q] - \sum_{i \in I_2(Q)} [\text{row } i \text{ of } Q] = [0, \dots, 0]$$

because in every column the two non-zero elements cancel out. It follows that the rows of Q are linearly dependent (or we can get a row with only zeros) so $\Rightarrow \det(Q) = 0$.

□

4.3 Cutting Plain Algorithm

In general the formulation of a ILP is not ideal $\Rightarrow$ the matrix associated is not TUM. So how we can manage those cases? In those situations we can solve the linear relaxation to find x^* , then what we want is to find an inequality that is feasible to the ILP but violates the current formulation, i.e., violates the current P . This technique is called *cutting plane* technique.

Consider an ILP problem in standard form $\min\{c^T x : Ax = b, x \geq 0 \text{ integer}\}$ and let $P := \{x \geq 0 : Ax = b\}$ be the polytope associated with its continuous relaxation.

Definition 16. Given $x^* \in P$ a cut is an inequality of type $\alpha^T x \leq \alpha_0$ such that:

- (1) $\alpha^T x \leq \alpha_0 \quad \forall x \in X = (P \cap \mathbb{Z}^n)$ (i.e. the cut is valid for the ILP)
- (2) $\alpha^T x^* > \alpha_0$ (i.e. the cut is violated by the current non integer solution x^*)

But how do we actually find those cuts?

Algorithm 4 Cutting Plane Algorithm

-
- 1: Solve the continuous relaxation $\min\{c^T \mathbf{x} : A\mathbf{x} = \mathbf{b}, \mathbf{x} \geq 0\}$ and let $\mathbf{x}^*$ be the optimal basic solution found
 - 2: **while** $\mathbf{x}^*$ is non-integer **do**
 - 3: Solve the separation problem of $\mathbf{x}^*$ from X , identifying an inequality $\alpha^T \mathbf{x} \leq \alpha_0$ valid for X but violated by $\mathbf{x}^*$
 - 4: Add constraint $\alpha^T \mathbf{x} \leq \alpha_0$ to the current continuous relaxation
 - 5: Solve the current continuous relaxation (by means of the dual simplex algorithm) and let $\mathbf{x}^*$ be the new optimal basic solution identified ▷ (see 3.5.1 to understand why we use the dual simplex)
 - 6: **end while**
-

4.3.1 Chvátal's Inequalities

Chavátal have found a simple procedure that generates valid inequalities for X starting from the continuous relaxation of $\min\{c^T x : Ax = b, x \geq 0\}$.

(See the numerical example at page 96).

In the general case: given a system $Ax = b, x \geq 0$, choose randomly $u \in \mathbb{R}^m$, we obtain

$$u^T Ax = u^T b \quad (\text{valid equation for } P).$$

By reducing the left hand side, we obtain:

$$\lfloor u^T A \rfloor x \leq u^T b \quad (\text{valid inequality for } P).$$

now the left hand side is a sum of integer values, so we can reduce also the right hand side:

$$\lfloor u^T A \rfloor x \leq \lfloor u^T b \rfloor \quad (\text{valid inequality for } X = P \cap \mathbb{Z}^n).$$

To summarize, we have defined:

- $\alpha^T x := \lfloor u^T A \rfloor x$;
- $\alpha_0 := \lfloor u^T b \rfloor$.

Using those definition, we obtain the cut $\alpha^T x \leq \alpha_0$ which satisfies the condition (1) but it doesn't always satisfies the condition (2) (that invalidates the optimal (fractional) solution x^* of the linear relaxation). The fact that the Chavátal's inequality satisfies condition (2) depends on the choice of $u \in \mathbb{R}^m$: some vectors u generates "useless" inequalities that do not invalidate x^* .

The question now is: how can we find a "useful" vector u ? It is possible to prove that, given a fractional vertex x^* of P , there exists a vector $u \in \mathbb{R}^m$ for which the corresponding inequalities $\alpha^T x \leq \alpha_0$ is violated by x^* . We will see how to find such u in the next subsection.

Important note: in the previous chapter we have seen a method that can produce all the possible valid inequalities for a polytope P (see the Farkas' lemma). In this case, it would be wrong to think that all valid inequalities for X are obtained *directly* starting from the initial system $Ax = b$: there exist valid inequalities for X that are not obtained as $\lfloor u^T A \rfloor x \leq \lfloor u^T b \rfloor$ for any $u \in \mathbb{R}^m$.

However Chavátal has prove some interesting properties. Define $P_0 := \{x \geq 0 : Ax = b\}$ as the current formulation. Now, define $P_1 := \{x \geq 0 : Ax = b, A^{(1)}x \leq b^{(1)}\} \subset P_0$ where $A^{(1)}x \leq b^{(1)}$ are the family of inequalities $\lfloor u^T A \rfloor x \leq \lfloor u^T b \rfloor$ and P_1 is called **first Chavátal clousure** of P_0 . Although $A^{(1)}x \leq b^{(1)}$, in principle contains ∞^m inequalities, Chavátal has proven that only a finite number of its inequalities are necessary to describe P_1 , hence P_1 is polytope.

So by adding this (finite number of) inequalities we improve the initial formulation $Ax = b, x \geq 0$. Now, if $P_1 = \text{conv}(X)$ then P_1 is the ideal formulation. Otherwise there exist fractional vertices of P_1 , that can be cut generating new Chvátal's inequalities obtained combining those defining P_1 . In this way we obtain a new (finite) family of inequalities, say $A^{(2)}x \leq b^{(2)}$. Adding this inequalities to P_1 , we obtain:

$$P_2 := \{x \geq 0 : Ax = b, A^{(1)}x \leq b^{(1)}, A^{(2)}x \leq b^{(2)}\} \subset P_1$$

By iterating the procedure for a finite number of times, say k times, we obtain a succession of polytopes $P_1, P_2, \dots, P_k$ such that: $P_0 \supset P_1 \supset \dots \supset P_k = \text{conv}(X)$. Value k is said to be the **Chavátal rank** of $\text{conv}(X)$ with respect to P_0 . Note that rank k , even if finite, can be very large: there exist cases in which k is even greater than the number of variables of the problem.

The procedure described makes it possible to obtain an ideal formulation of $\text{conv}(X)$ starting from any initial formulation, however the number of necessary inequalities, even though finite, is generally huge.

All of this was to simply say that the cutting plain algorithm will converge to an optimal solution for the ILP problem taken in input, at some point.

4.3.2 Gomory's Cuts

The Gomory's Cuts satisfy condition (1) and (2) of definition 16.

The computation of a Gomory's cut is very simple, we work only on the tableau of the linear relaxation. In this subsection we will answer the question asked in the previous subsection: how do we choose $u \in \mathbb{R}^m$ in order to obtain a cut that is not "useless"?

So at first we solve the linear relaxation of the ILP problem using the simplex method $\Rightarrow$ we have a basis B and an optimal solution x^* . Lets say that x^* have at least one fractional value x_h^* , i.e. the solution isn't integer feasible. The variable x_h^* must be a basic variable (the non-basic have a value of 0) on a certain row t (i.e. $h = \beta[t]$).

The equation associated at row t of the optimal tableau is the following:

$$\underbrace{x_h}_{\text{basic var}} + \underbrace{\sum_{j \in F} \bar{a}_{tj} x_j}_{\text{non-basic vars}} = \bar{b}_t \quad (= x_h^*) \quad (4.1)$$

where F is the set of indices of the non-basic variables. This is a valid equation for P because is an equation of the form $u^T A x = u^T b$, where u^T is the t -th row of the current matrix B^{-1} .

Now we can easily get the Gomory's cut by applying the Chavátal truncation procedure starting from the equation above:

$$x_h + \sum_{j \in F} \lfloor \bar{a}_{tj} \rfloor x_j \leq \lfloor \bar{b}_t \rfloor \quad (4.2)$$

We know that the Gomory's cut is feasible for $X = P \cap \mathbb{Z}^n$ because we have only applied the Chavátal truncation. But, we're sure that it also invalidate the current x^* ?

$$x_h^* + \sum_{j \in F} \lfloor \bar{a}_{tj} \rfloor \underbrace{x_j^*}_{=0} \leq \lfloor \bar{b}_t \rfloor \quad \Rightarrow \quad x_h^* \leq \lfloor \bar{b}_t \rfloor$$

but we know that $x_h^* = \bar{b}_t \Rightarrow x_h^* = \lfloor \bar{b}_t \rfloor$, which is a contradiction because we have assumed that x_h^* is fractional.

So we have proved that the current x^* is not feasible anymore after we add the Gomory's cut.

Note that the 4.2 isn't in standard form, to bring the constraint in standard form we need to add

a new slack variable:

$$x_h + \sum_{j \in F} \lfloor \bar{a}_{tj} \rfloor x_j + \sigma = \lfloor \bar{b}_t \rfloor \quad (4.3)$$

So at every iteration of the Cutting Plane algorithm, we need to add a new slack σ ! Note that, in the equation above: the coefficient of x_h (one) is integer; all the coefficients of x_j are integer; the right hand side is also integer. So it must be that σ is integer by construction, we don't need to add the integrality constraint!

The formulation in 4.2 is called **integer form**. We can also define an equivalent formulation called **fractional form** which is obtained by subtracting 4.3 from 4.1:

$$\sum_{j \in F} \underbrace{-(\bar{a}_{tj} - \lfloor \bar{a}_{tj} \rfloor)}_{\geq 0} x_j + \sigma = - \underbrace{(\bar{b}_t - \lfloor \bar{b}_t \rfloor)}_{\geq 0} \quad (4.4)$$

this formulation is better because we do not need to perform any minipivot operations on the new tableau. This new constraint can easily be added to the current tableau maintaining the dual feasibility required for the application of the dual simplex method of the *cutting plane algorithm*.

(See example page 100 of the book).

		x_h					$x_j, j \in F$			s
	$-z_{PL}$	0	...	0	...	0	...	$\bar{c}_j \geq 0$	...	0
generating row t	$\bar{b}_t$	0	...	1	...	0	...	$\bar{a}_{tj}$	...	0
Gomory's cut	$-\varphi(\bar{b}_t)$	0	...	0	...	0	...	$-\varphi(\bar{a}_{tj})$	...	1

Figure 4.5: Where $\varphi(x) := x - \lfloor x \rfloor$ (the fractional part of x).

How do we discover that the ILP is infeasible $\Rightarrow X = P \cap \mathbb{Z}^n = \emptyset$? Using the cutting plane algorithm, solving the linear relaxation using the dual simplex algorithm, at some point we will reach an inconsistent row (i.e. we still have $x_{\beta[t]} < 0$ but $\bar{a}_{tj} \geq 0 \forall j = 1, \dots, n$).

(See example page 103 of the book).

Commercial solvers actually detect this situation during the preprocessing computation.

In theory the Cutting Plane using the Gomory's cuts is a very good algorithm, but in practice it isn't that good. The first cut added to the problem is usually a very deep one, the following ones become increasingly shallower. This procedure will converge to a solution at some point but it

may require a huge number of iterations. Note also that, after many iterations of the algorithm imply numerical instability issue caused by the presence of fractional values in the tableau.

4.4 Branch-and-Bound

An alternative algorithm for solving an ILP problem (and, more generally, a combinatorial optimization problem) is known in the literature as implicit enumeration, or branch-and-bound. It is a divide and conquer procedure that (recursively) casts the solution of a "difficult" ILP problem into the solution of two "easier" ILP subproblems.

The algorithm starts from the root node, which represents the ILP that we want to solve.

1. The relaxation of ILP is considered by removing the integrality constraint from the variables. Once the relaxation is solved, three cases may occur:
 - (a) The linear relaxation is infeasible $\Rightarrow$ the ILP is infeasible, so the algorithm terminates.
 - (b) The relaxation has an optimal solution x_{LP}^* that is integer $\Rightarrow x_{LP}^*$ is also optimal for the ILP (so $x_{LP}^* = x_{ILP}^*$), and the algorithm terminates.
 - (c) If instead x_{LP}^* is fractional, we have found a **lower bound** on the optimal value of *ILP*.

If we encounter case (c), then the algorithm continues.

2. Branching is performed on the root node. The branching operation applied to a node identifies its child nodes: these are subproblems obtained by imposing restrictions on the parent problem. We call this new problem ILP_i . At least one of the solution variables x_{LP}^* does not satisfy the integrality constraint, i.e., $\exists j \in J : x_j^* \notin \mathbb{Z}$. Consequently, two child nodes can be created by imposing one of the following constraints on each:

$$x_j \leq \lfloor x_j^* \rfloor, \quad x_j \geq \lceil x_j^* \rceil. \quad (4.5)$$

The nodes are added to the queue Q of subproblems to be solved.

3. A subproblem $ILP_i \in Q$ is considered, and its linear relaxation LP_i is solved. Four cases may arise:
 - (a) If the relaxation is infeasible $\Rightarrow$ the node can be discarded (**pruning by infeasibility**).

- (b) If the optimal solution $x_{LP_i}^*$ of the relaxation is integer $\Rightarrow$ we can save the value of $x_{LP_i}^*$ in $x_{OPT} :=$ the current optimal solution (the best integer solution so far, called **incumbent**) and we can compute the upper bound $z_{OPT} = c^T x_{OPT}$ valid for the optimal solution of initial *ILP*.
 - (c) If $x_{LP_i}^*$ is fractional, z_{OPT} is compared with the solution found: if $LB_i := c^T x_{LP_i}^* \geq z_{OPT}$, then the node can be discarded (**pruning by optimality**), as LB_i is the lower bound on the optimal solution of *ILP*_{*i*}, and therefore cannot lead to a better solution than the current z_{OPT} .
 - (d) Otherwise ($LB_i := c^T x_{LP_i}^* < z_{OPT}$), branching is necessary on the node, and its children are added to Q .
4. The smallest value LB_i returned by the linear relaxations of open nodes is a valid lower bound for the optimal solution of *ILP*. Using this property, the lower bound found in the first step is updated. Note that the lower bound can only increase because, going from parent to child, the problems become more and more constrained.
 5. The algorithm proceeds until the queue of subproblems Q is empty. In this situation, the value of the incumbent coincides with the lower bound, and therefore z_{OPT} corresponds to the optimal solution of the initial *ILP* found by the algorithm.

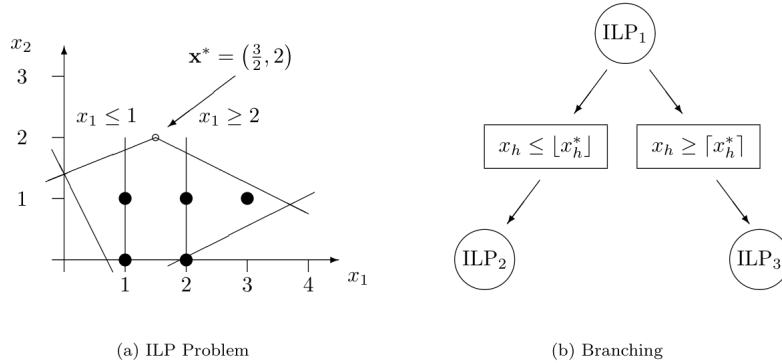


Figure 4.6: Branch-and-Bound

Figure 4.8 shows the branching tree typically used to describe the procedure. Node 1 corresponds to the original ILP problem (root node). Each child node is obtained from the parent node by adding the constraint shown on the branch connecting them. In general, the constraints characterizing the problem associated with a given node t are:

1. the original constraints $Ax = b, x \geq 0$ integer, and

2. all the branching constraints found in the branching tree along the path from the root node to node t.

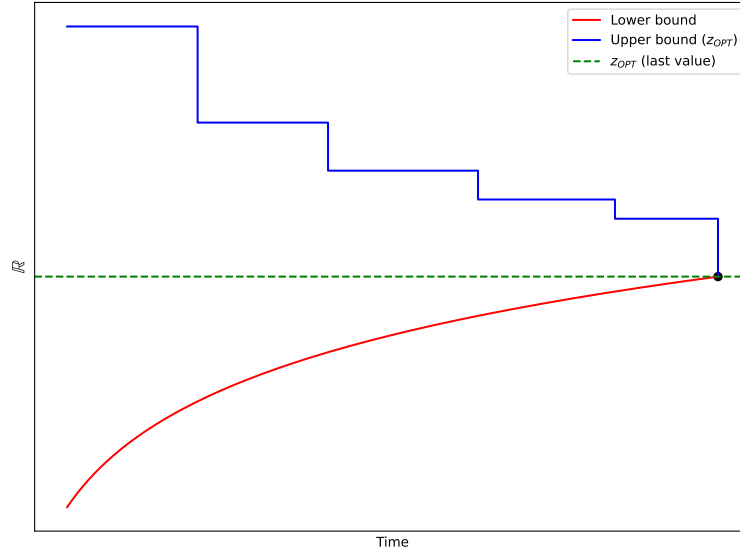


Figure 4.7: Branching tree

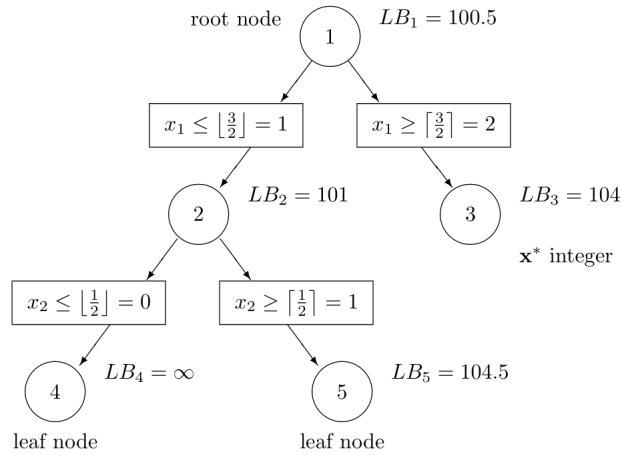


Figure 4.8: Branching tree. Node 3 has initialized x_{OPT} and therefore z_{OPT} ; nodes 4 is pruned by infeasibility; node 5 is pruned by optimality $LB_5 \geq z_{OPT}$.

Will the B&B converges at some point? Yes. You can consider, without loss of generality², an example in which all the variable are not only integer but binary: $x_j \in \{0, 1\}$, $\forall j = 1, \dots, n$. At

²Wlog: if we have a x_j that is a generic integer, then we can convert it to a binary variable using the "bit notation": $x_j = x_j^0 + 2x_j^1 + 4x_j^2$, where x_j^i are all binary variables. The number of variables x_j^i that we need to add depends on the upper bound of x_j .

each branching operation, we will then have $x_h \leq \lfloor x_h^* \rfloor = 0$ $x_h \geq \lceil x_h^* \rceil = 1$, hence in both cases variable x_h will be fixed (to either 0 or 1). As we go down along the branching tree, the number of fixed variables increases and thus, in the *worst case*, the tree will have a depth of n and will contain 2^n leaf nodes (plus $2^n - 1$ intermediate nodes).

(See page 109 of the book for some implementation considerations).

4.5 Branch-and-Cut

The major problem of the Cutting Plane algorithm is that the convergence of the algorithm is guaranteed in theory but not in practice. Define as *gap* the difference between the optimal value of the problem z_{OPT} and the current lower bound. Using the Cutting Plane algorithm we can observe that, in the very first iterations this gap decrease a lot, i.e., we get big improvements towards an optimal solution. As the iteration increases, the improvements become smaller and smaller and at some point it will eventually converge. In practice this is not true due to numerical issue caused by the representation of the numbers in a computer. As we have seen, Gomory's cuts invalidate the current optimal (non-integer) solution and are feasible for the ILP problem. After many iterations this cuts become less and less deep and, due to approximation error, what we see in practice is that the new generated cuts no longer invalidate the current (non-integer) solution. A practical trick to avoid this problem is the following: add in the first iteration of the algorithm, not just a single cut but as many as you can (i.e. consider all the fractional variable). Using this trick increases the improvement on the gap in the first iterations even more.

The problem with the B&B algorithm is that, in the worst case the number of nodes that we need to explore in order to get a solution is exponential. The key element in the B&B is the lowerbound, so in order to improve his quality, the idea is to add cuts to the formulation.

The basic idea is simple: design a branch-and-bound algorithm in which, at each node t of the branching tree, some cuts are generated in the hope of obtaining an integer solution $x_{PL_t}^*$, or even just a tighter (and hence more useful for pruning) lower bound $c^T x_{PL_t}^*$. As soon as we see that these cuts become ineffective, cut generation stops and we switch to branching (in order to avoid the *tailing off* problem).

The Gomory's cuts that we have introduced, works well for general purpose ILP problems, but in many real world cases they're not enough: we have to find a class of inequalities that works for the peculiar structure of the problem. The problem of finding this class of inequalities

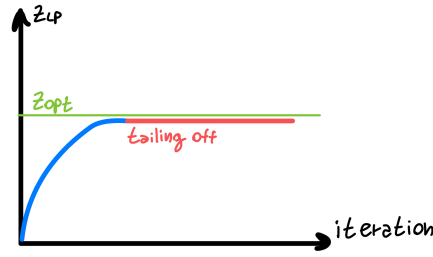


Figure 4.9: Tailing off problem.

is called *finding a separation procedure*. To be more precise, what we want is to define an effective procedure for extract solution of the following

Definition 17 (Separation problem). *Given x^* , identify (if there exists) an inequalities $\alpha^T x \leq \alpha_0$ belonging to class C such that $\alpha^T x^* > \alpha_0$.*

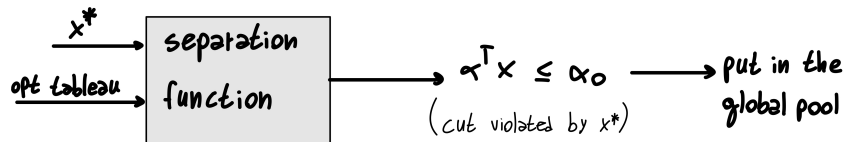


Figure 4.10

The global pool is a data structure where we memorize all the global cuts. When processing a new node of the branching tree, we start from formulation $Ax = b, x \geq 0$, we add the branching constraints, and we solve the corresponding continuous relaxation obtaining solution x^* (say). If x^* is fractional, the pool is scanned for constraints violated by x^* to be added to the current formulation. We then solve the new formulation and we proceed in this way until x^* satisfies all the constraints of the pool. If x^* is still fractional, a convenient separation procedure can be called in order to identify new globally-valid cuts to be inserted into the pool. If none of the separation procedures generate any cut, then we can just branch.

(See page 115 for an example in which we identify two classes of inequality for the problem of the index selection).

Chapter 5

Graph Theory

5.1 Undirected Graphs

Definition 18. *A undirected graph is a pair $G = (V, E)$ in which V is a finite set and E is a family of unordered pairs of elements of V . The elements of V are called nodes (or vertex) of G , the elements of E are called edges of G .*

We will always denote $n := |V|$ (number of nodes in the graph) and $m := |E|$ (number of edges in the graph).

Definition 19 (Weighted edge/vertex). *When G is edge-weighted (or vertex-weighted) a function $c : E \rightarrow \mathbb{R}$ (or $w : V \rightarrow \mathbb{R}$, respectively) is specified.*

Typically $V = \{1, \dots, n\}$ (the names of the nodes), and the edges have the form $\{i, j\}$ with $i, j \in V$; for typographical reasons, we will use the equivalent notation $[i, j] \equiv [j, i]$.

Definition 20 (Simple graph). *A graph without multiple edges (also called parallel edges) is said to be simple.*

In some cases edges of the type $[i, i]$ (called loops) are not allowed.

We say that edge $[i, j]$ connects (or is incident to) i and j .

Definition 21 (Adjacent vertices and edges). *Two vertices are said to be adjacent if there exists an edge connecting them. Two edges are said to be adjacent (or consecutive) if they have a vertex in common.*

Definition 22 (Degree of a vertex). *The number of edges incident on a vertex v is called degree of the vertex. It's usually denoted by $d_G(v)$.*

Definition 23 (Isolated vertex). *A vertex is said to be isolated if $d_G(v) = 0$.*

Theorem 15. Consider a graph $G = (V, E)$. It holds

$$\sum_{v \in V} d_G(v) = 2|E|.$$

Definition 24 (Complete graph). Graph G is said to be complete if it contains all possible edges.

Definition 25 (Subgraph). $G' = (V', E')$ is said to be a subgraph of $G = (V, E)$ if $V' \subseteq V$ and $E' \subseteq E$.

Definition 26 (Partial graph). $G' = (V', E')$ is said to be a partial graph of $G = (V, E)$ if $V' = V$, i.e., when it has the same vertices of G but only a subset of its edges.

Definition 27 (Path and cycle). A path of G is a sequence of consecutive edges $e_1, e_2, \dots, e_k \in E$ of the type: $e_1 = [v_1, v_2], e_2 = [v_2, v_3], \dots, e_k = [v_k, v_{k+1}]$. We say that the path connects v_1 and v_{k+1} , visiting in sequence the intermediate vertices $v_2, v_3, \dots, v_k$. The path is a cycle if $v_1 = v_{k+1}$.

Definition 28 (Elementary and simple path/cycle). A path/cycle is said to be elementary if it never uses the same edge twice. An elementary path/cycle is said to be simple if it never visits the same vertex twice, except for vertex $v_1 = v_{k+1}$ in the case of a cycle.

Definition 29 (Connected vertices and connected graph). A vertex v is said to be connected to another vertex w if there exists a path connecting them. By definition every vertex v is connected to itself (even if $[v, v] \notin E$). A graph G is said to be connected if vertices v and w are connected for every $v, w \in V$.

Definition 30 (Cut in graphs). A cut of graph G is a set of edges of the type:

$$\delta_G(S) := \{[i, j] \in E : |S \cap \{i, j\}| = 1\}$$

where S is the subset of vertices that induces the cut (i.e., the cut contains all edges with one endpoint in S and the other in $V \setminus S$).

Theorem 16. Graph G is connected if and only if $\delta(S) \neq \emptyset$ for every $\emptyset \subset S \subset V$ (without considering $S = V$ and $S = \emptyset$).

Definition 31 (Forest). A partial graph $G' = (V, E')$ of $G = (V, E)$ is called a forest if it is acyclic, i.e., it doesn't contain any cycle.

Definition 32 (Tree). A minimal connected forest, if it exists, is called spanning tree, or simply tree.

Theorem 17. *Every tree has exactly $|V| - 1$ edges.*

Definition 33 (Bipartite graph). *A graph is said to be bipartite if there exists a partition (V_1, V_2) of V such that each edge $[i, j] \in E$ connects a vertex $i \in V_1$ to a vertex $j \in V_2$ (i.e. we cannot have an edge that connects two vertex that belong to the same partition).*

Theorem 18. *Graph G is bipartite if and only if it doesn't contain odd cycles, i.e. cycles with an odd number of edges.*

There exists a simple algorithm that can answer whether a graph is bipartite or not.

Definition 34 (Planar graph). *A graph is said to be planar if it can be drawn on the Euclidean plane in such way that the lines representing the edges do not "cross" each other outside of the vertices.*

There exists an algorithm that can answer whether a graph is planar or not.

Definition 35 (Eulerian path/cycle and graph). *An elementary path or cycle is said to be Eulerian if it visits edges of the graph exactly once. A connected graph is considered Eulerian if there exists an Eulerian cycle.*

Theorem 19. *Graph G is Eulerian if and only if it is connected and each vertex $v \in V$ has even degree $d_G(v)$.*

Definition 36 (Hamiltonian path/cycle and graph). *A simple path or cycle is said to be Hamiltonian if it visits all vertices of the graph exactly once. G is an Hamiltonian graph if it contains a Hamiltonian cycle.*

There are no known "simple" conditions to verify whether a given graph G is Hamiltonian.

Definition 37 (Clique). *A clique is a subgraph $G' = (V', E')$ of G in which every pair of vertices in V' is connected by an edge. By definition, $G_v := (\{v\}, \emptyset)$ is a clique for all $v \in V$. A clique G' is maximal if V' cannot be enlarged. A maximal clique $G' = (V', E')$ is maximum if there is no clique $G'' = (V'', E'')$ with $|V''| > |V'|$.*

Definition 38 (Stable set). *A stable set (or independent set) of G is a subgraph $G' = (V', E')$ induced by V' such that $E' = \emptyset$ (there exists no edge connecting two vertices in V'). As with cliques, it is possible to define the concepts of maximal and maximum stable sets.*

5.2 Directed Graphs

Definition 39. A directed graph (or digraph) is a pair $G = (V, A)$ in which V is a finite set of vertices (or nodes) and A is a family of ordered pairs of vertices, called arcs. The arcs are pairs of the type (i, j) with $i, j \in V$. The order in which the two vertices appear is relevant, i.e., $(i, j) \neq (j, i)$.

Many other definitions valid for undirected graphs also apply to directed graphs.

Definition 40 (Directed path). A directed path is a sequence $a_1, a_2, \dots, a_k$ of consecutive arcs of the type $a_1 = (v_1, v_2), a_2 = (v_2, v_3), \dots, a_k = (v_k, v_{k+1})$. The path starts from v_1 and reaches v_{k+1} , passing through the intermediate vertices $v_2, \dots, v_k$. The existence of such a path makes v_{k+1} reachable from v_1 , and let v_1 reach v_{k+1} .

Definition 41 (Reachable and reached vertices). For all $v \in V$ we will denote by Γ_v^+ the set of vertices reachable starting from v , and with Γ_v^- the set of vertices starting from which v can be reached. By definition, $v \in \Gamma_v^+ \cap \Gamma_v^-$ for all $v \in V$.

Definition 42 (Strongly connected graph). A directed graph G is said to be strongly connected if $\Gamma_v^+ = V$ for all $v \in V$, i.e., if there exists a path from v to w (and hence also from w to v) for all pairs $v, w \in V$.

Definition 43 (Directed cut). A directed cut is a subset of arcs of the type:

$$\delta_G^+(S) := \{(i, j) \in A : i \in S, j \notin S\}$$

or of the type:

$$\delta_G^-(S) := \{(i, j) \in A : i \notin S, j \in S\}$$

for some $S \subset V$. Note that $\delta_G^+(S) \equiv \delta_G^-(V \setminus S)$.

Theorem 20. Graph G is strongly connected if and only if $\delta_G^+(S) \neq \emptyset$ for all $\emptyset \subset S \subset V$.

Definition 44 (Eulerian directed graph). A directed and strongly connected graph is said to be Eulerian if there exists a directed circuit (i.e. a directed cycle) that uses all arcs of the graph exactly once.

Definition 45 (Hamiltonian directed graph). A directed graph is Hamiltonian if there exists a simple circuit passing exactly once through all the vertices of the graph.

5.3 Graph Representation

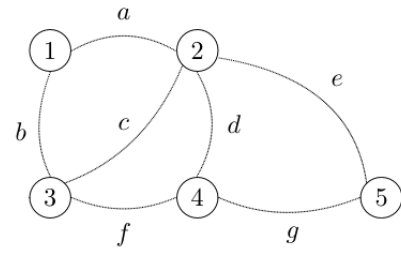
Definition 46 (Node-edge incidence matrix - undirected graph). *The node-edge incidence matrix D of an undirected graph $G = (V, E)$ is the $|V| \times |E|$ matrix with elements:*

$$d_{ij} := \begin{cases} 1 & \text{if the } j\text{-th edge is incident to vertex } i \\ 0 & \text{otherwise.} \end{cases}$$

The columns are the edges and the rows are the vertices. Every column of D has exactly two elements different from zero.

$$\begin{array}{c} \begin{matrix} & a & b & c & d & e & f & g \end{matrix} \\ \begin{matrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{matrix} \begin{bmatrix} 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 \end{bmatrix} \end{array}$$

Figure 5.1



(a) Undirected graph

Figure 5.2

Theorem 21. *The node-edge incidence matrix D of an undirected graph $G = (V, E)$ is TUM if and only if G is bipartite.*

Indeed, if G is bipartite with vertex partition (V_1, V_2) , then total unimodularity derives from the theorem 14 choosing $I_1 = V_1$ and $I_2 = V_2$.

Definition 47 (Node-edge incidence matrix - directed graph). *The node-edge incidence matrix D of a directed graph $G = (V, A)$ is the $|V| \times |A|$ matrix with elements:*

$$d_{ij} := \begin{cases} 1 & \text{if the } j\text{-th arc leaves vertex } i \\ -1 & \text{if the } j\text{-th arc enters vertex } i \\ 0 & \text{otherwise.} \end{cases}$$

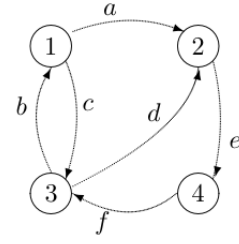
Every column of D has exactly one 1 and one -1 .

Theorem 22. *The node-edge incidence matrix D of a directed graph $G = (V, A)$ is always TUM.*

From the theorem 14: we can choose the partition $I_1 = V$ and $I_2 = \emptyset$, and we get that D is TUM.

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>
1	1	-1	1	0	0	0
2	-1	0	0	-1	1	0
3	0	1	-1	1	0	-1
4	0	0	0	0	-1	1

Figure 5.3



(b) Directed graph

Figure 5.4

	1	2	3	4	5	6	7	8	9	10	11	12	13	14
FROM	1	1	2	2	2	2	3	3	3	4	4	4	5	5
TO	2	3	1	3	4	5	1	2	4	2	3	5	2	4

	1	2	3	4	5	6
FIRST	1	3	7	10	13	15

Figure 5.5

A commonly-used data structure for undirected graphs stores, for all $v \in V$, its star $\delta(v)$ (figure 5.5).

Note that every edge $[i, j]$ is then stored twice, given that $[i, j] \in \delta(i) \cap \delta(j)$. Also note that the edges of a graph are stored in the array so that the edges in $\delta(1)$ occupy the first $|\delta(1)|$ positions, those of $\delta(2)$ the following $|\delta(2)|$ positions, and so on.

We can actually get rid of the array $FROM[1, \dots, n]$: we can use instead an array $FIRST[1, \dots, n+1]$ in which we store the pointer of the first element of $\delta(i)$ for every $i \in V$. For instance, with $TO[FIRST[3]]$ we can read the first edge in $\delta(3)$ (which is $[3, 1]$ in the picture). We use the convention $FIRST[1] = 1$, $FIRST[n+1] = 2m+1$ (in order to point to the (last + 1) column of $TO[...]$, see the figure).

A similar data structure can be used for directed graphs, storing for each $v \in V$ the corresponding forward star $\delta^+(v)$ and/or the corresponding backward star $\delta^-(v)$.

Particular graphs can sometimes be stored more compactly. In particular, a directed tree can (every node has exactly one entering arc) be stored in an array $pred[1, \dots, n]$, in which the

elements are defined as follows:

$$pred[j] = \begin{cases} s & \text{if } j = s \text{ is the root} \\ i & \text{if arc } (i, j) \text{ belongs to the tree} \\ 0 & \text{if } (i, j) \text{ does not belong to the tree} \end{cases}$$

5.4 Minimum spanning tree

Consider an undirected graph $G = (V, E)$ and a cost function $c : E \rightarrow \mathbb{R}$, let c_e be the cost associated with the edge $e \in E$. What we want to identify is a spanning tree $G_T = (V, T)$ associated with the minimum cost, i.e., with the minimum $\sum_{e \in T} c_e$. Obviously we assume that G is a connected graph, otherwise the problem is infeasible.

We can define a ILP model of the minimum spanning tree problem. Define a decision variable for every edge $e \in E$:

$$x_e = \begin{cases} 1 & \text{if } e \in T \\ 0 & \text{otherwise.} \end{cases}$$

The model is the following:

$$\begin{cases} \min \sum_{e \in E} c_e x_e \\ \sum_{e \in E} x_e = |V| - 1 \\ \sum_{e \in E(S)} x_e \leq |S| - 1 \quad \forall S \subseteq V, S \neq \emptyset \\ x_e \in \{0, 1\} \quad \forall e \in E \end{cases} \quad (5.1)$$

The first constraint is used in order to choose exactly $|V| - 1$ edges (the number of edges in a tree is exactly that number). The second constraint is used in order to avoid the presence of cycle in the tree: we consider all the subset S of vertices in V and what we want is that the sum of the edges in S is at most $|S| - 1$, i.e. the number of vertices in S minus 1. Note that this property holds only for undirected graph, not for the directed ones. We define

$$E(S) = \{[i, j] \in E \mid i, j \in S\}$$

i.e., we $E(S)$ contains all the edges that connect the vertices inside S .

Some considerations:

- If $|S| = 1 \Rightarrow E(j) = \emptyset$, so we can skip the subset of size one.
- If $|S| = 2 \Rightarrow$ the constraint becomes $x_e \leq 1$ (because we are considering simple graph so we have only one arc in S in this case), therefore the constraint is redundant because we have already impose that the variables are binary.
- If $S = V \Rightarrow E(V) = E$, therefore the constrain is redundant since it becomes the first constrain of the problem.

The last constraint can be rewritten as follows:

$$\sum_{e \in E(S)} x_e \leq |S| - 1 \quad \forall S \subset V, S \neq \emptyset, |S| \geq 3$$

However the considerations above does not solve the problem of our formulation: the number of possible subset S is still exponential $\Rightarrow$ exponential number of constraints.

One can prove that the formulation is ideal therefore we can remove the integrality constraint, but this doesn't solve our problem.

5.4.1 Prim-Dijkstra's Algorithm

Assumption:

1. G is connected.
2. $c_e \neq c_f \quad \forall e, f \in E, \quad e \neq f$ i.e: the costs are all different.

Assumption (1) is needed otherwise the problem is simply infeasible. Assumption (2) is needed to guarantee the uniqueness of the minimum spanning tree, which allow us to formulate more precisely its fundamental properties. In case condition (2) does not apply, the proposed algorithms would however still be correct, and would allow to identify one of the optimal solution of the problem.

Theorem 23. Let $G_{T^*} = (V, T^*)$ be a minimum spanning tree. For any $e \in E$ we have that $e \in T^*$ if and only if it exists an $S \subset V$ s.t. $e = \arg \min\{c_f : f \in \delta(S)\}$.

Proof. First part: prove that " $e = \arg \min\{c_f : f \in \delta(S)\}$ for any cut $\delta(S) \Rightarrow e \in T^*$ ".

- Assume that $e \notin T^*$.
- As show in the figure 5.6, the set $T^* \cup \{e\}$ contains a cycle $C[T^*, e]$ that "crosses" the cut $\delta(S)$ at least twice¹.

¹Because a spanning tree must have $|V| - 1$ edges, if you add edges you can surely find a cycle.

- Choose any edge $f \in C[T^*, e] \cap \delta(S)$, $f \neq e$, and observe that $c_e < c_f$ by assumption.
- So we can define $T' := T^* \setminus \{f\} \cup \{e\}$ and we get a better solution because the cost associated to the tree $c(T') < c(T^*)$.
- This is a contradiction, therefore we have proved that we must have $e \in T^*$.

- We can consider $T' := T^* \setminus \{e\}$ and choose S to be one of the two connected components of the partial graph $G_{T'} = (V, T')$.
- If there existed $f \in \delta(S) \setminus \{e\}$ with $c_f < c_e$ then T^* would not be an optimal solution because in that case $c(T') < c(T^*)$ (a contradiction).

1

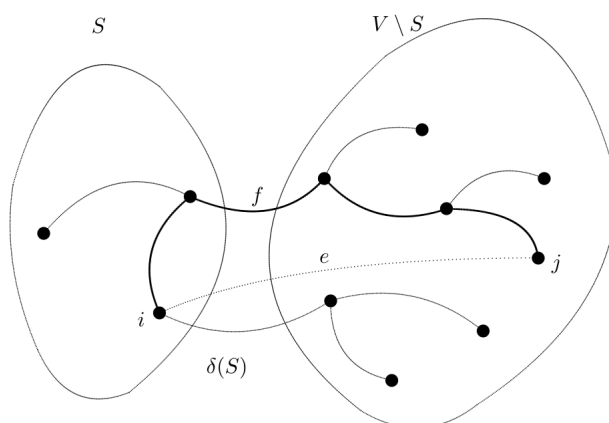


Figure 5.6: Cycle $C[T^*, e]$ in $T^* \cup \{e\}$.

Algorithm 5 Prim-Dijkstra's Algorithm (1st version)

```

1:  $T^* \leftarrow \emptyset$ 
2:  $S \leftarrow \{1\}$ 
3: while  $|T^*| \neq n - 1$  do
4:   Identify edge  $[v, h] \in \delta(S)$  of minimum cost, with  $v \in S$  and  $h \notin S$ 
5:    $T^* \leftarrow T^* \cup \{[v, h]\}$ 
6:    $S \leftarrow S \cup \{h\}$ 
7: end while

```

Prim-Dijkstra's Algorithm The algorithm above shows an implementation of the Prim-Dijkstra's algorithm. The most consuming time operation is the one at step 4: we start by adding to S the node 1; in the first iteration of the while loop we have to scan all the arcs that connects the node 1 to all the nodes $h \in V \setminus S$, i.e we need to scan the arcs $[1, h] \forall h \in V \setminus S$ in order to choose the minimum cost; in the second iteration we have that $S = \{1, 3\}$ (say), so we need to check all the arcs $[1, h], [3, h] \forall h \in V \setminus S$ in order to choose the minimum cost; and so on.

Choosing the edge $[v, h]$ using this approach is not efficient, the complexity is $O(m)$ so the total complexity is $O(n \cdot m)$ and if the graph is complete (it has $m = (n-1)n/2$ edges) the complexity becomes $O(n^3)$. It is however possible to write an algorithm with complexity $O(n^2)$: this is actually the algorithm that Prim and Dijkstra invented.

Consider the following approach:

1. At first we add to S the node 1.
2. Before entering the loop, we compute

$$L_j = \min\{c_{ij} : i \in S\} \quad \forall j \notin S$$

with $i = 1$ at the beginning. By computing the value of L_j we can identify the lowest cost associated to the edge that connects node 1 to each node $j \in V \setminus S$. We can initialize all the value of L_j at the beginning of the algorithm, this requires $O(n)$ time.

3. In order to choose the new edge to add, we can compute

$$h := \arg \min\{L_j : j \notin S\}$$

so we add to S the node h (i.e. $[1, h]$ is the edge that connects node 1 to node h that has the lowest cost). The time complexity required is $O(n)$.

4. Now we need to update the values of $L_j \forall j \notin S$. To do that we have simply to compute

$$L_j = \min\{L_j, c_{hj}\} \quad \forall j \notin S$$

i.e. we change the value if and only if the new added node h to S has an arc $[h, j]$ that has an associated cost that is lower than the cost associated to $[i, j] \forall i \in S, i \neq h$ (remember that in the first iteration $i = 1$). The time complexity is $O(n)$.

5. We have to proceed in this way until we add to T^* exactly $n - 1$, or in an equivalent way, until $S = V$.

Algorithm 6 Prim-Dijkstra's Algorithm (2st version)

```

1:  $S \leftarrow \{1\}$ 
2:  $L_j \leftarrow c_{1j} \quad \forall j \notin S$ 
3: while  $|S| \neq n$  do
4:    $h \leftarrow \arg \min\{L_j : j \notin S\}$ 
5:    $S \leftarrow S \cup \{h\}$ 
6:    $L_j \leftarrow \min\{L_j, c_{hj}\} \quad \forall j \notin S$ 
7: end while

```

Steps 2, 4 and 7 of the second version of the Prim-Dijkstra's Algorithm require $O(n)$ time each, therefore the algorithm takes $O(n^2)$ time. Note that the output of the algorithm above tells us only the costs of the Minimum Spanning Tree. In order to retrieve the actual tree, we need to store additional informations about the edge added.

5.4.2 Kruskal's Algorithm

The Prim-Dijkstra's algorithm is the best choice for graph that are complete. However, in many real world problems, the graph is sparse², i.e., $m \ll n^2/2$. In such cases we can use an algorithm that takes the advantage of the sparsity in order to achieve better performance: the Kruskal's algorithm does exactly that.

Kruskal's algorithm is a greedy algorithm that provides an optimal solution. The idea of the algorithm is the following:

1. **Preprocessing:** sort all the edges by incremental costs. Let $L \leftarrow \{c_{i_1}, c_{i_2}, \dots, c_{i_m}\}$ be the sorted list.
2. **Greedy choice:** choose the edge associated to the minimum cost, i.e. c_{i_1} , and add it to T^* . Pop the head of L .
3. While $|T^*| \neq n - 1$
 - (a) $c_e :=$ the first element of L (let e be the edge associated to the cost)
 - (b) If $T^* \cup \{e\}$ leads to a cycle
 - then $L \leftarrow L \setminus \{c_i\}$ (skip the edge)

²If the graph is planar, then $m \approx 3n$.

- else $T^* \cup \{e\}$; $L \leftarrow L \setminus \{c_i\}$

The step (1) requires to sort a list $\Rightarrow O(m \log m)$ time that we can rewrite as $O(m \log n)$ because $\log m < \log n^2 = 2 \log n$. The step (3.b) require an algorithm that tells if, by doing the union $T^* \cup \{e\}$ we create a cycle or not. In order to do that we need to use additional memory where we store the information about the current connected components of the graph $G_{T^*} = (V, T^*)$ that we're constructing. To be more precise:

- In the first iteration we have exactly n connected components.
- Lets say that we want to add the edge $e = [i, j]$ to T^* .
- If the node i and j belong to the same connected component then they will for sure create a cycle in G_{T^*} . So we add e to T^* only if i and j belong to different connected component.
- Lets say that we have added $e = [i, j]$ to T^* , then we need to update the information stored: now i and j belong to the same connected component.

The steps described above are performed by an algorithm known as the *union-find* algorithm. A state-of-the-art implementation of this algorithm is highly efficient. By employing it to execute step (3.b), Kruskal's Algorithm achieves a time complexity of $O(m \log n)$, determined primarily by the sorting step.

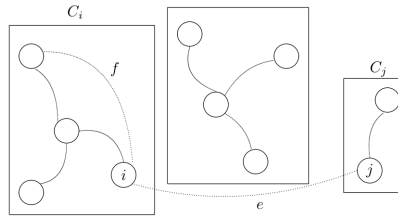


Figure 5.7: Connected components of $G_{T^*} = (V, T^*)$.

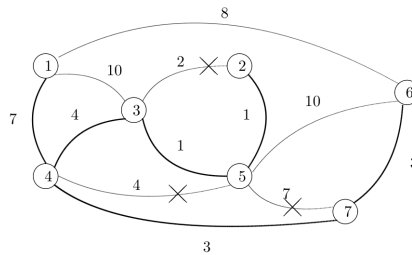


Figure 5.8: Application of the Kruskal's algorithm.

5.5 Path in a directed graph

5.5.1 Reachability

The problem of reachability consider a directed graph $G = (V, A)$, a $s \in V$ called *source* and a node $t \in V \setminus \{s\}$ called *terminal*. The goal is to find (if it exists) a path that starts from node s and reach node t . In other words we want to compute:

$$\Gamma_s^+ = \{j \in V : \exists \text{ path from } s \text{ to } j\}.$$

In order to solve this problem we will consider an algorithm that implement the *labelling method*. Using this algorithm we're able to tell if we can reach every $t \in V$ starting from a fixed node s .

The idea of the algorithm is pretty simple:

- Start from the node s and mark it with a label (because is reachable); then mark all the node j such that $(s, j) \in \delta^+(s)$.
- Repeat the process with all the new marked nodes j .

Algorithm 7 Pathfinding Algorithm

```

1: for  $j \leftarrow 1$  to  $n$  do
2:    $pred[j] \leftarrow 0$                                 ▷ At the beginning all nodes are "non reachable"
3: end for
4:  $pred[s] \leftarrow s$                                 ▷ The source  $s$  is for sure reachable
5:  $Q \leftarrow \{s\}$                                     ▷ Node not yet elaborated
6: while  $Q \neq \emptyset$  do
7:   Choose a vertex  $h \in Q$  and set  $Q \leftarrow Q \setminus \{h\}$ 
8:   for each  $(h, j) \in \delta^+(h)$  do
9:     if  $pred[j] = 0$  then
10:       $pred[j] \leftarrow h$                             ▷ Now node  $j$  is reachable from  $h$ 
11:       $Q \leftarrow Q \cup \{j\}$ 
12:    end if
13:  end for
14: end while

```

In the algorithm above we use:

- $pred[j]$ = vertex preceding j in a path from s to j ($= 0$ if vertex j is not reachable from s). Using this data structure we know if j is reachable or not, and also how to reach j from s .
- Q = queue of the vertices reachable from s and not yet processed.

The preprocessing part of the algorithm requires $O(n)$ time; every node j is inserted into the queue Q at most once, so the while loop is performed at most once for the same node h , hence each arc (h, j) of the graph is considered at most once at step 8. Using If we store the graph by means of the forward-star $\delta^+(v)$ of its vertices (see 5.3), step 7 is efficient (aka we don't have to scan all the edges of the graph), so the algorithm has a complexity of $O(n + m) \Rightarrow$ its linear in the size of the graph.

5.5.2 Shortest Path Problem

The shortest path problem, or the minimum cost path problem, considers a directed graph $G = (V, A)$ where each arc has an associated cost of c_{ij} (cost of arc $(i, j) \in A$), a source node $s \in V$ and a terminal node $t \in V \setminus \{s\}$. The goal is to find a path of minimum cost $P \subseteq A$ that connects s to t .

Note that in many applications we can have multiple source nodes s and multiple terminal nodes t . In such cases, what we can do is to add a new fictitious start node s' and also the arcs (s', h) for each $h = \text{old start node}$. By doing a similar thing for the fictitious terminal node t' , we get a graph with only one source node and one terminal node.

The problem of finding the minimum cost path that starts from s and ends in t is

- an easy problem if all the costs associated to all the arcs are positive, i.e. $c_{ij} \geq 0 \quad \forall (i, j) \in A$;
- an easy problem if there are negative costs but there aren't any negative cycles in the graph;
- otherwise is difficult.

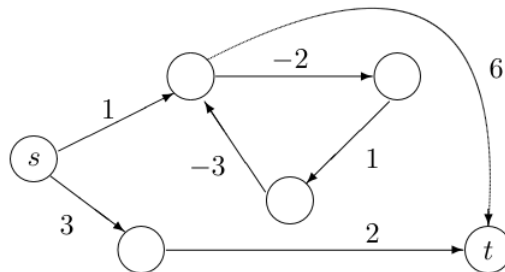


Figure 5.9: Graph with a negative cost cycle: $-2 + 1 - 3 = -4 < 0$.

ILP Model The model needs to choose which arcs define a minimum cost path P that connects the source node s to the terminal node t . Therefore we introduce a binary variable, one for each arc in the graph:

$$x_{ij} = \begin{cases} 1 & \text{if } (i, j) \in P \\ 0 & \text{otherwise} \end{cases} \quad \forall (i, j) \in A.$$

The model:

$$\begin{cases} \min \sum_{(i,j) \in A} c_{ij} x_{ij} \\ \sum_{(i,j) \in \delta^+(h)} x_{ij} - \sum_{(i,j) \in \delta^-(h)} x_{ij} = \begin{cases} 1 & h = s \\ -1 & h = t \\ 0 & \forall h \in V \setminus \{s, t\} \end{cases} \\ x_{ij} \in \{0, 1\} \quad \forall (i, j) \in A \end{cases} \quad (5.2)$$

The first constraint imposes that the difference between the number of leaving arcs from h and the number of entering arcs in h is: 1 for the source node (i.e. only one leaving arc); -1 for the terminal node (i.e. only one entering arc); 0 for every other node (i.e. same number of leaving and entering arcs). Using this constraint we're sure that the solution found identifies a directed path from s to t .

This model is correct if we assume that the graph have only positive costs associated to each arc, or we have some negative costs but we do not have any negative cycle (i.e., if the problem is "easy").

However, if the problem is "difficult", then we must also add to the previous model the so called **subtour elimination** constraints. The model is the following:

$$\begin{cases} \min \sum_{(i,j) \in A} c_{ij} x_{ij} \\ \sum_{(i,j) \in \delta^+(h)} x_{ij} - \sum_{(i,j) \in \delta^-(h)} x_{ij} = \begin{cases} 1 & h = s \\ -1 & h = t \\ 0 & \forall h \in V \setminus \{s, t\} \end{cases} \\ \sum_{(i,j) \in A(S)} x_{ij} \leq |S| - 1 \quad \forall S \subseteq V, S \neq \emptyset \\ x_{ij} \in \{0, 1\} \quad \forall (i, j) \in A \end{cases} \quad (5.3)$$

where $A(S) := \{(i, j) \in A : i, j \in S\}$. The subtour elimination constraints break the cycles, with that we avoid the situations like the one shown in the next figure.

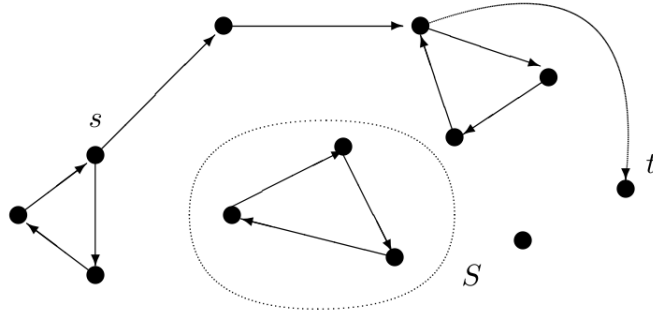


Figure 5.10: A possible solution of the model without subtour elimination constraints: we can see a non-sense cycle and also the path found by the model is non-simple.

As we have seen for the Minimum Spanning Tree problem, the number of possible subset of vertices S is exponential.

If we bring the formulation 5.3 in the form of $Ax = b$, we can prove that the matrix A is TUM, so the formulation is ideal and therefore we can remove the integrality constraint on the x_{ij} variables.

5.5.3 Dijkstra's Algorithm

The algorithm requires that $c_{ij} \geq 0 \forall (i, j) \in A$. Dijkstra's algorithm is based on the following property.

Theorem 24. Assume that the cost L_i of the shortest paths from s to each vertex i belonging to a given set $S \subset V$ with $s \in S$ ($L_s := 0$) is known, and let $(v, h) := \arg \min \{L_i + c_{ij} : (i, j) \in \delta^+(S)\}$. If $c_{ij} \geq 0 \forall (i, j) \in A$, then $L_v + c_{vh}$ is the cost of the shortest path from s to h .

Proof. Consider $L_v + c_{vh}$ to be the cost of a path from s to h formed by the shortest path from s to v with cost L_v followed by arc (v, h) . We want to prove that any other path P (say) that goes from s to h has a cost $\text{cost}(P) \geq L_v + c_{vh}$.

Let (i, j) be the first arc in $P \cap \delta^+(S)$. We can partition P in $P_1 \cup \{i, j\} \cup \{P_2\}$, where P_1 is a path from s to i and P_2 is a path from j to h .

What we have is that:

$$\text{cost}(P) = \underbrace{\text{cost}(P_1)}_{\geq L_i} + c_{ij} + \underbrace{\text{cost}(P_2)}_{\geq 0} \geq L_i + c_{ij} \geq L_v + c_{vh}.$$

Where: $L_i + c_{ij} \geq L_v + c_{vh}$ because of how we have defined (v, h) . □

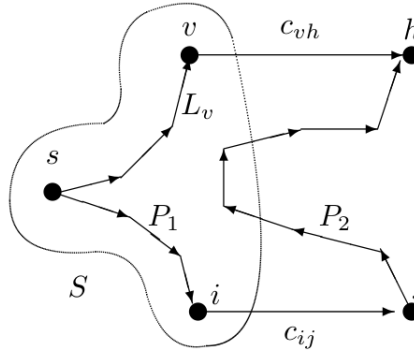


Figure 5.11

The pseudocode of the Dijkstra's algorithm is very similar to the pseudocode of the Prim-Dijkstra's algorithm.

Algorithm 8 Dijkstra's Algorithm

```

1:  $S \leftarrow \{s\}$ 
2:  $L_j \leftarrow c_{sj} \quad \forall j \notin S$ 
3: while  $|S| \neq n$  do
4:    $h \leftarrow \arg \min\{L_j : j \notin S\}$ 
5:    $S \leftarrow S \cup \{h\}$ 
6:    $L_j \leftarrow \min\{L_j, L_h + c_{hj}\} \quad \forall j \notin S$ 
7: end while

```

L_j tells us the cost associated to the path that goes from s to j , $\forall j \in V$, i.e., we can find all the paths that start from s and ends in any node j . So in output we get a spanning tree, but in general it isn't a Minimum Spanning Tree. In order to retrieve the spanning tree we have to store additional information: we need to use the vector $pred[j]$ and store the predecessor of each node.

Steps 2, 4, 6 require $O(n)$ time complexity, so in the end the algorithm is $O(n^2)$ (like the Prim-Dijkstra's algorithm).

5.5.4 Floyd-Warshall's Algorithm

The Floyd-Warshall's algorithm is used if the graph contains some negative costs, i.e., $\exists c_{ij} < 0$. The algorithm has a very simple implementation, the time complexity required is $O(n^3)$ and this is because, in order to compute the minimum cost path from a node s to all the other nodes j (like the Dijkstra's algorithm does), we have to compute all the minimum cost path starting

from each node of the graph³.

Data structure used in the algorithm:

- $d_{ij} :=$ cost of the shortest path from i to j .
- $pred[i, j] :=$ predecessor of j in the shortest path from i to j .

Algorithm 9 Floyd-Warshall's algorithm

```

1:  $d[i, j] \leftarrow c_{ij} \quad \forall i, j = 1, \dots, n$ 
2:  $pred[i, j] \leftarrow i \quad \forall i, j = 1, \dots, n$ 
3: for  $h \leftarrow 1$  to  $n$  do
4:   for  $i \leftarrow 1$  to  $n$  do
5:     for  $j \leftarrow 1$  to  $n$  do
6:       if  $d[i, h] + d[h, j] < d[i, j]$  then
7:          $d[i, j] \leftarrow d[i, h] + d[h, j]$ 
8:          $pred[i, j] \leftarrow pred[h, j]$ 
9:       end if
10:    end for
11:   for  $i \leftarrow 1$  to  $n$  do
12:     if  $d[i, i] < 0$  then
13:       Found a negative cycle  $\Rightarrow$  STOP
14:     end if
15:   end for
16: end for
17: end for

```

In the step 1 we initialize the matrix. The update of the value of the matrix is based on the *triangular property* (figure 5.12). We update the value of d_{ij} whenever the triangular property is not satisfied. At the end of the algorithm the property is satisfied for each entry of the matrix.

The algorithm doesn't work when the graph contains negative cycles. But how we detect them? The matrix d must have all zeros in the diagonal, i.e., $d[i, i] = 0 \quad \forall i = 1, \dots, n$ and this is because the cost in order to reach i from i is always zero. However this value can become strictly lower than zero if there exists a **negative cycle** in the graph. Therefore the Floyd-Warshall's algorithm can be also used for discovering the presence of negative cycles in a graph.

Theorem 25. Consider matrix d obtained after performing the triangular operation on vertices $1, \dots, h$. In the absence of negative cycles, for all $i, j \in V$, value $d[i, j]$ is the cost of the shortest path from i to j in the subgraph induced by the set of vertices $\{1, \dots, h\} \cup \{i, j\}$.

³Note that Dijkstra's algorithm provide the minimum cost path from a chosen node s to all the other nodes j in time $O(n^2)$. So if we want to compute all the path of minimum cost starting from each node of the graph, then we have to apply Dijkstra's algorithm n times $\Rightarrow$ in total we get $O(n^3)$.

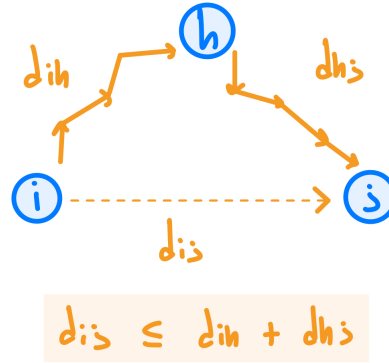


Figure 5.12: Triangular property.

Proof. We will use the induction on h .

Base: When $h = 0$, we have that $d[i, j] = c_{ij} \quad \forall i, j$, so the matrix d represents the cost of the shortest path from i to j in the subgraph induced by $\{i, j\}$, as claimed. We are considering a graph with only those vertices. Consider for example a node $i = 4$ and all the arcs $(4, j)$ for all $j \in V$. $h = 0$ means that we are considering that the paths goes directly from the source node $i = 4$ to all the other terminal nodes j . In this case the property is true.

Hypothesis: The property is true for every $h' < h$.

Thesis: We can fix the value of h . Let i and j be any number and consider the shortest path P_{ij} from i to j in the subgraph induced by $\{1, \dots, h\} \cup \{i, j\}$ (i.e. we are not considering the node $h + 1, \dots, n$, none of the path can pass through those nodes in the current iteration), and let $\text{cost}(P_{ij})$ be its cost.

We have two cases:

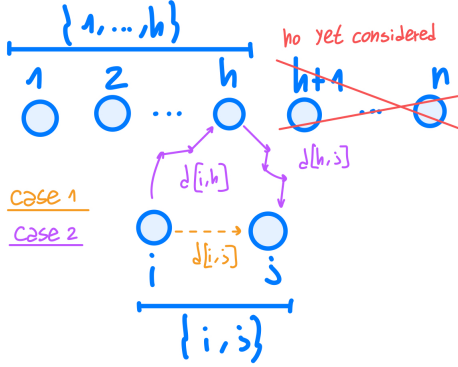
1. P_{ij} does not pass through vertex h : in this case $\text{cost}(P_{ij}) = d[i, j]$ by the inductive hypothesis.
2. P_{ij} passes through h : we can partition P_{ij} in $P_{ih} \cup P_{hj}$, where P_{ih} is a shortest path from i to h in the subgraph induced by $\{1, \dots, h-1\} \cup \{i, h\}$, and P_{hj} is a shortest path from h to j in the subgraph induced by $\{1, \dots, h-1\} \cup \{h, j\}$. By the inductive hypothesis, we then have that $\text{cost}(P_{ih}) = d[i, h]$ and $\text{cost}(P_{hj}) = d[h, j]$, and hence $\text{cost}(P_{ij}) = d[i, h] + d[h, j]$.

Therefore it holds that the minimum cost path that goes from i to j is the minimum between the

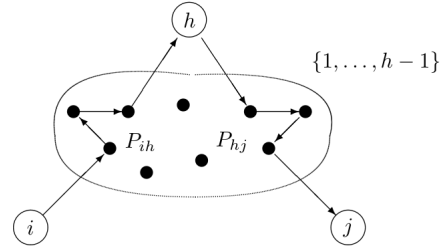
two cases:

$$\text{cost}(P_{ij}) = \min\{d[i, j], d[i, h] + d[h, j]\}$$

thus, after the triangular operation on h , we have that $d[i, j] = \text{cost}(P_{ij})$ as requested. $\square$



(a)



(b)

5.5.5 Project Planning

An important application of the shortest path problem can be found in the planning of complex projects.

Definition 48. A project is a set of activities A_i ($i = 1, \dots, m$), each having a known duration $d_i \geq 0$. Among some activities, possible precedence relations of the type $A_i \prec A_j$ can be specified (meaning that activity A_j can begin only after the completion of activity A_i).

We are interested in finding the *make span*, i.e., the maximum time required in order to complete all the activities A_j .

Consider the following problem:

Activity: A, B, C, D, E Precedence: $A \prec B, A \prec C, B \prec D, C \prec D, B \prec E$

One way to model this problem is using a graph.

In the following, instead of writing the name of the activity on the arcs, we will write its duration.

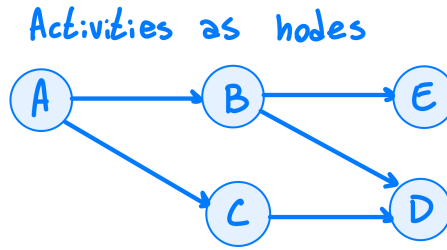


Figure 5.14

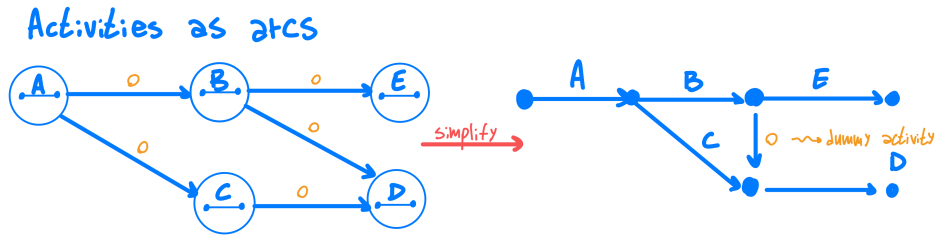


Figure 5.15

CPM algorithm (or PERT algorithm) Assumption:

1. The graph does not have any circuits. If the graph associated with the problem has a circuit, it means that the problem is infeasible. For instance $A_1 \prec A_2 \prec A_3 \prec A_1$ identify a circuit (deadlock) in the path and we can clearly see that the problem is infeasible.
2. It exists exactly one starting node (initial activity) s : $\exists! s : \delta^-(s) = \emptyset$.
3. It exists exactly one terminal node (terminal activity) t : $\exists! t : \delta^+(t) = \emptyset$.
4. The node are numbered in topological order, i.e., if $(i, j) \in A$ then $i < j$ (this is not ambiguous because circuits are not present).

The CMP algorithm implements a Shortest Path Problem but the complexity required is $O(m)$ so it is better than the one required by the Dijkstra's algorithm, and this is because we don't have to check the presence of circuits.

The algorithm:

- Associate to each node h the value $TMIN_h$ which indicates the minimum time to reach node h . $TMIN_h$ indicates also the earliest starting time of the activity associated to the arcs that leave the node.

- From previous point, it follows that $TMIN_n$ is the minimum duration of the project, i.e., is the make span.
- Associate to each node h the value $TMAX_h$ which indicates the maximum time to reach h without compromising the minimum duration of the project. If $TMIN_h < TMAX_h$ it means that the activity can start with a delay without compromising the overall duration of the project.

Algorithm 10 Critical Path Method (CPM)

```

1:  $T_{MIN}[1] \leftarrow 0$ 
2: Sort the vertices topologically
3: for  $h \leftarrow 2$  to  $n$  do ▷ Forward pass
4:    $T_{MIN}[h] \leftarrow \max\{T_{MIN}[i] + d[i, h] : (i, h) \in \delta^-(h)\}$ 
5: end for
6:  $T_{MAX}[n] \leftarrow T_{MIN}[n]$  ▷ Minimum duration of the project (make span)
7: for  $h \leftarrow n - 1$  downto 1 do ▷ Backward pass
8:    $T_{MAX}[h] \leftarrow \min\{T_{MAX}[j] - d[h, j] : (h, j) \in \delta^+(h)\}$ 
9: end for
  
```

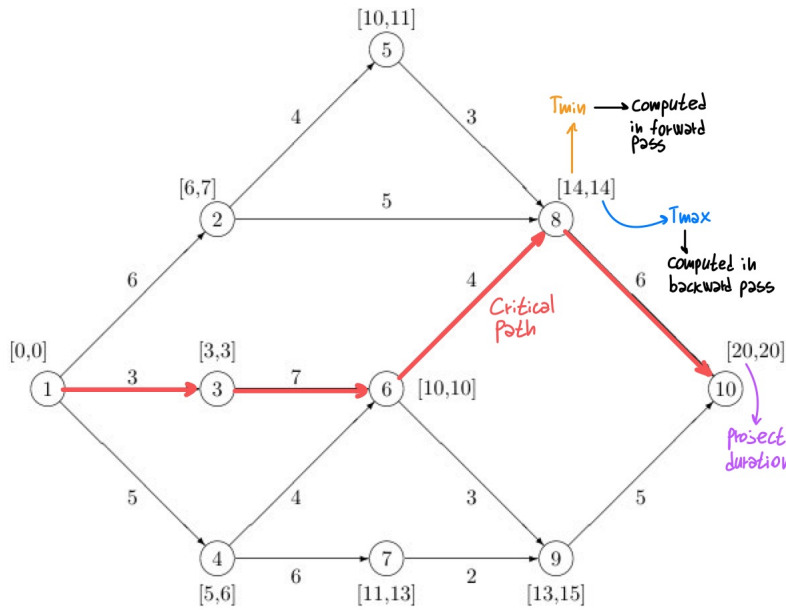


Figure 5.16

Critical arc: Activities (i, j) with zero float are said to be critical, and satisfy $TMIN_i = TMAX_i$, $TMIN_j = TMAX_j$, and $TMIN_i + d_{ij} = TMAX_j$.

Critical path: A path from s to t in G is said to be critical if all its arcs are critical.

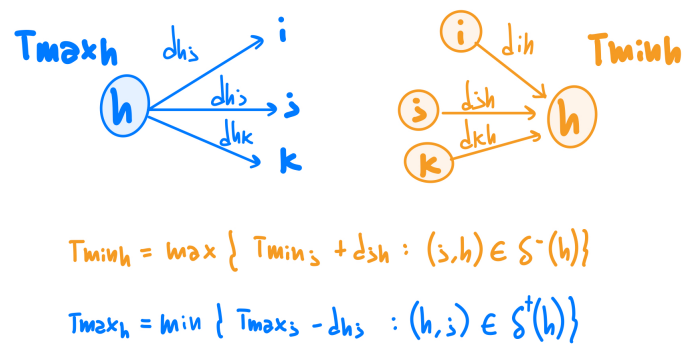


Figure 5.17

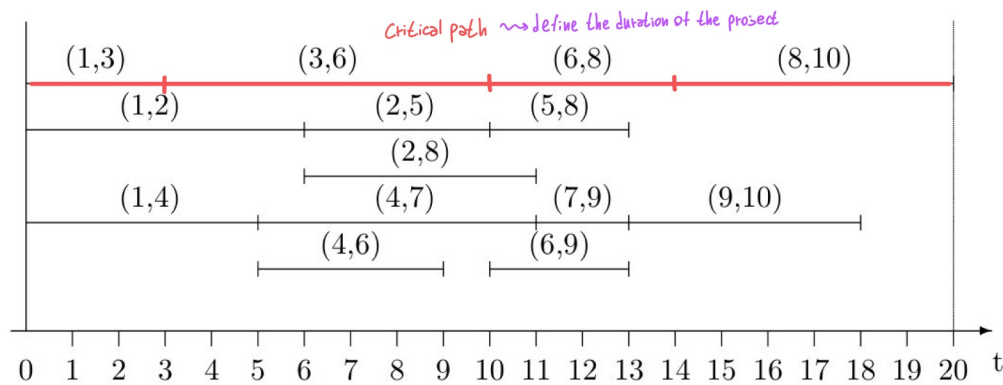


Figure 5.18: Gantt chart.

5.6 Flow problems

A flow network is a directed graph $G = (V, A)$ in which a capacity $k_{ij} \geq 0 \forall (i, j) \in A$. In addition, two vertices s and t are specified, called *source vertex* and *sink vertex*, respectively. Typically, but not always, we have $\delta^-(s) = \delta^+(t) = 0$, i.e., the network does not have arcs entering into s nor arcs leaving from t .

Definition 49 (MAX-FLOW). Given a flow network, a maximum flow from s to t can be viewed

as a linear programming problem:

$$\left\{ \begin{array}{l} \varphi_0 := \max \underbrace{\sum_{(s,j) \in \delta^+(s)} x_{sj}}_{\text{leaving } s} - \underbrace{\sum_{(i,s) \in \delta^-(s)} x_{is}}_{\text{entering in } s} \\ \underbrace{\sum_{(i,j) \in \delta^+(h)} x_{ij}}_{\text{leaving } h} - \underbrace{\sum_{(i,j) \in \delta^-(h)} x_{ij}}_{\text{entering in } h} = 0 \quad \forall h \in V \setminus \{s, t\} \\ 0 \leq x_{ij} \leq k_{ij} \quad \forall (i, j) \in A \end{array} \right.$$

where the first constraint is used to impose that the flow x_{ij} entering in each node $h \neq s, t$ must be equal to the flow leaving it; the second constraint impose that the flow x_{ij} are lower then or equal to the capacity k_{ij} associated to the arc $(i, j) \in A$.

If we rewrite the formulation in the form of $Ax = b$ we can easily prove that A is TUM, therefore (if the capacities k_{ij} are all integer values) the formulation identifies a feasible region with integer vertices.

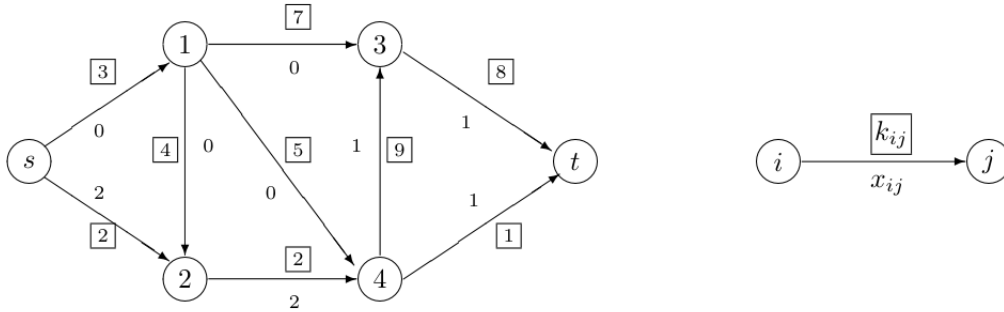


Figure 5.19: Example of feasible flow.

A more general problem consists in identifying a minimum-cost flow from s to t .

Definition 50 (MIN-COST FLOW). Given a flow network, a maximum flow from s to t can be

viewed as a linear programming problem:

$$\left\{ \begin{array}{l} \min \sum_{(i,j) \in A} c_{ij} x_{ij} \\ \underbrace{\sum_{(i,j) \in \delta^+(h)} x_{ij}}_{\text{leaving } h} - \underbrace{\sum_{(i,j) \in \delta^-(h)} x_{ij}}_{\text{entering in } h} = 0 \quad \forall h \in V \setminus \{s, t\} \\ \underbrace{\sum_{(s,j) \in \delta^+(s)} x_{sj}}_{\text{leaving } s} - \underbrace{\sum_{(i,s) \in \delta^-(s)} x_{is}}_{\text{entering in } s} = W \\ 0 \leq x_{ij} \leq k_{ij} \end{array} \right. \quad \forall (i, j) \in A$$

where the second constraint is used to set the flow required for the application.

5.6.1 Fundamental Properties

Definition 51 (Section or cut or st-cut). A section (or cut or st-cut) is a partition $(S, V \setminus S)$ of the set of vertices such that $s \in S$ and $t \in V \setminus S$. We associate to the cut also $\delta^+(S)$ and $\delta^-(S)$.

Definition 52 (Flow through a cut). Given a feasible flow x , we call flow through a cut $(S, V \setminus S)$ the quantity

$$\varphi(S) := \sum_{(i,j) \in \delta^+(S)} x_{ij} - \sum_{(i,j) \in \delta^-(S)} x_{ij}.$$

Quantity

$$\varphi_0 := \varphi(\{s\}) = \sum_{(s,j) \in \delta^+(s)} x_{sj} - \sum_{(i,s) \in \delta^-(s)} x_{is}.$$

is said to be the value of flow x .

Definition 53 (Capacity of a cut). The capacity of a cut $(S, V \setminus S)$ is the quantity:

$$k(S) := \sum_{(i,j) \in \delta^+(S)} k_{ij}$$

Theorem 26. Let x be a feasible flow. For every cut $(S, V \setminus S)$ the flow $\varphi(S) = \varphi_0$, i.e., the flow through every cut is a constant.

Proof. Take any cut $(S, V \setminus S)$. We have that:

$$\begin{aligned}
 \varphi_0 &:= \varphi(\{s\}) = \sum_{(s,j) \in \delta^+(s)} x_{sj} - \sum_{(i,s) \in \delta^-(s)} x_{is} \\
 &= \sum_{h \in S} \underbrace{\left[\sum_{(h,j) \in \delta^+(h)} x_{hj} - \sum_{(i,h) \in \delta^-(h)} x_{ih} \right]}_{=0 \text{ for all } h \neq s} \\
 &= \sum_{h \in S} \sum_{(h,j) \in \delta^+(h)} x_{hj} - \sum_{h \in S} \sum_{(i,h) \in \delta^-(h)} x_{ih}
 \end{aligned}$$

But

$$\begin{aligned}
 (i) \quad & \sum_{h \in S} \sum_{(h,j) \in \delta^+(h)} x_{hj} = \sum_{(i,j) \in A(S)} x_{ij} + \sum_{(i,j) \in \delta^+(S)} x_{ij} \\
 (ii) \quad & \sum_{h \in S} \sum_{(i,h) \in \delta^-(h)} x_{ih} = \sum_{(i,j) \in A(S)} x_{ij} + \sum_{(i,j) \in \delta^-(S)} x_{ij}
 \end{aligned}$$

So in the end:

$$\Rightarrow \varphi_0 = (i) - (ii) = \sum_{(i,j) \in \delta^+(S)} x_{ij} - \sum_{(i,j) \in \delta^-(S)} x_{ij} =: \varphi(S)$$

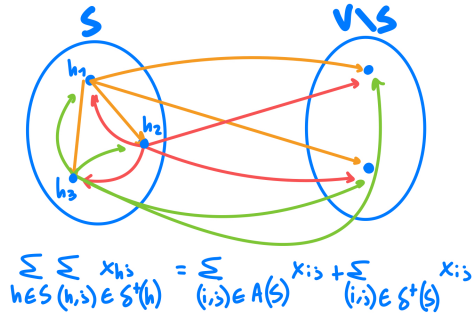


Figure 5.20

□

Theorem 27. For every feasible flow x and every cut $(S, V \setminus S)$, one has

$$\varphi(S) \leq k(S).$$

Proof. We have that:

$$\varphi(S) := \sum_{(i,j) \in \delta^+(S)} \underbrace{x_{ij}}_{\leq k_{ij}} - \sum_{(i,j) \in \delta^-(S)} \underbrace{x_{ij}}_{\geq 0} \leq \sum_{(i,j) \in \delta^+(S)} k_{ij} =: k(S).$$

□

The previous two theorems imply a "weak" duality relation: the value of ϕ_0^* of the maximum flow cannot exceed the capacity $k(S^*)$ of the minimum-capacity cut $(S^*, V \setminus S^*)$ (the dual problem).

We will see that the Ford-Fulkerson Algorithm always find (if exists) an optimal flow ϕ_0^* which is equal to the minimum capacity $k(S^*)$ (strong duality relation).

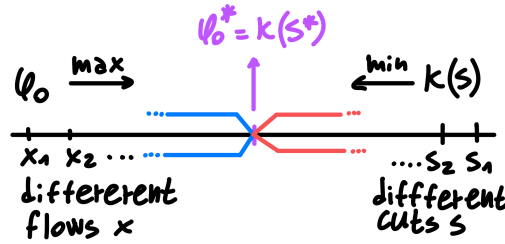


Figure 5.21: Strong duality relation between the maximum flow ϕ_0^* and the capacity $k(S^*)$ of the minimum-capacity cut.

Before defining the Ford-Fulkerson method, we need to introduce some more definition.

Definition 54 (Saturated and empty arcs). *Let x be a feasible flow. An arc $(i, j) \in A$ is said to be saturated if $x_{ij} = k_{ij}$, empty if $x_{ij} = 0$.*

Definition 55 (Residual network). *Let x be a feasible flow. The residual network $\bar{G} = (V, \bar{A})$ associated with x is obtained from the original network $G = (V, A)$ by replacing each arc $(i, j) \in A$ with two arcs:*

- a forward arc (i, j) of residual capacity $\bar{k}_{ij} := k_{ij} - x_{ij} \geq 0$;
- a backward arc (j, i) of residual capacity $\bar{k}_{ji} := x_{ij} \geq 0$;

and then eliminating all arcs with zero residual capacity.

Theorem 28. *A feasible flow x is optimal for the MAX-FLOW problem if and only if vertex t is not reachable from vertex s in the residual network $\bar{G} = (V, \bar{A})$ associated with x .*

Proof. Let ϕ_0 be the value of flow x .

Now consider $\bar{G}$. If t is reachable from s in $\bar{G}$, then it exists an augmenting path P from s to t in $\bar{G}$.

Define:

$$\delta := \min\{\bar{k}_{uv} : (u, v) \in P\} > 0$$

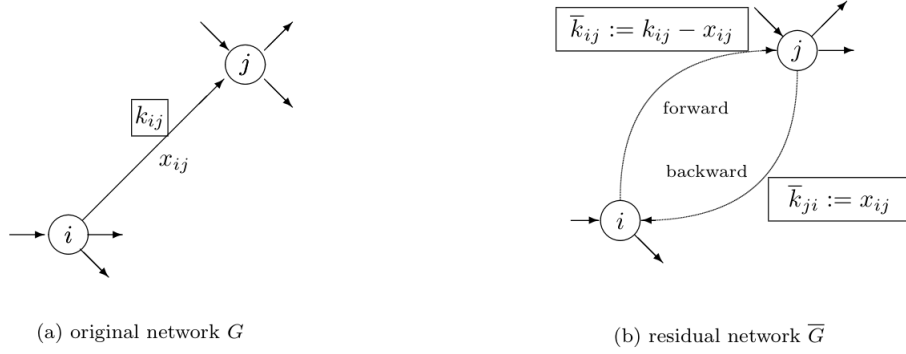


Figure 5.22: Residual network: arcs (u, v) with $\bar{k}_{uv} = 0$ must be eliminated.

(δ is the bottleneck). Perform the update for all arcs $(u, v) \in P$:

$$x_{uv} := x_{uv} + \delta \quad \text{if } (u, v) \text{ is a forward arc}$$

$$x_{uv} := x_{uv} - \delta \quad \text{if } (u, v) \text{ is a backward arc}$$

The new flow vector x define a feasible solution of value $\varphi_0 + \delta$. This proves that the starting solution x was not optimal.

Now consider that t is not reachable from s in $\bar{G}$ (last iteration of the algorithm). There hence exists a cut $(S^*, V \setminus S^*)$ in the residual network $\bar{G}$ such that $\delta_G^+(S^*) = \emptyset$.

By definition of residual network, we have that in the original network G :

- each $(i, j) \in \delta_G^+(S^*)$ (leaving arc) is saturated;
- each $(i, j) \in \delta_G^-(S^*)$ (entering arc) is empty;

We can compute the flow through S^* :

$$\varphi(S^*) := \underbrace{\sum_{(i,j) \in \delta_G^+(S^*)} x_{ij}}_{\text{all saturated}} - \underbrace{\sum_{(i,j) \in \delta_G^-(S^*)} x_{ij}}_{\text{all empty}} = \sum_{(i,j) \in \delta_G^+(S^*)} k_{ij} := k(S^*)$$

From theorem 27 we know that the flow x is an optimal solution and also $(S^*, V \setminus S^*)$ is a cut of minimum capacity in the original network. $\square$

Interpretation of the backward arcs: decrease the flow in some arc in order to increase the flow somewhere else.

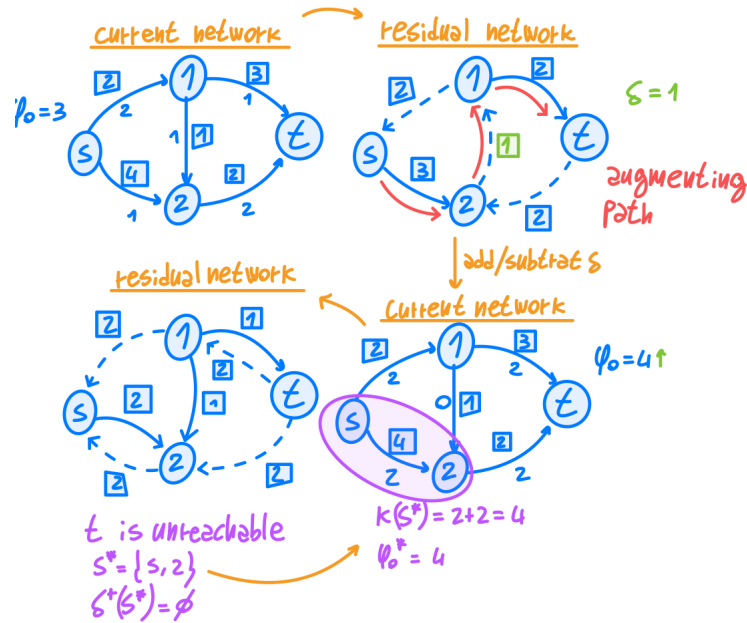


Figure 5.23

Interpretation of the minimum-capacity cut $(S^*, V \setminus S^*)$: the arcs in $\delta^+(S^*)$ are the weakest points in the network, they represent the bottleneck.

5.6.2 Ford-Fulkerson Algorithm for the MAX-FLOW problem

At each iteration:

1. Draw the current network G .
2. Draw the residual network $\bar{G}$.
3. Find one augmenting path P in $\bar{G}$ that starts from s and ends in t . If it doesn't exist, then stop, $\phi_0^* := \phi_0$ is an optimal solution.
4. Define the $\delta := \min\{\bar{k}_{uv} : (u, v) \in P\} > 0$.
5. Perform the update for all arcs $(u, v) \in P$:

$$x_{uv} := x_{uv} + \delta \quad \text{if } (u, v) \text{ is a forward arc}$$

$$x_{uv} := x_{uv} - \delta \quad \text{if } (u, v) \text{ is a backward arc.}$$

6. Update $\phi_0 := \phi_0 + \delta$.

7. Repeat from step (1).

At every iteration $\Delta\phi_0 = \delta > 0$. Will eventually converge? Yes if all the k_{ij} are integers values, the method converges in at most ϕ_0^* iterations (i.e. in the worst case it increases by 1 unit at each iteration). If k_{ij} are fractional values, then the method will never converge.

In order to apply the method we don't need to redraw all the graph at each iteration. By defining a proper data structure we can use only the original graph. (See the example in the notes).

Chapter 6

NP-hard Problems

NP-hard means non-deterministic polynomial.

6.1 Knapsack Problem

Description:

- Input: n items associated to a profit p_j and a weight w_j ; knapsack with capacity W .
- Output: we have to identify the subset $S \subseteq \{1, \dots, n\}$ of items that we want to put in the knapsack.
- Constrains: the sum of the chosen weight must not exceed W , so $\sum_{j \in S} w_j \leq W$.
- Objective: maximize the profit, so $\max \sum_{j \in S} p_j$.
- Assumptions: $w_j > 0$, $p_j > 0$ and both integer $\forall j = 1, \dots, n$; also $W < \sum_{j=1}^n w_j$ otherwise the solution is simply to choose all the items.

The Knapsack problem is an example of problem which is hard in theory but easy in practice. Is difficult in theory because we can construct an artificial problem that can be solved in exponential time; in practice this kind of situation doesn't appear, so the related problem can be efficiently solved by some ad-hoc solver.

Let

$$x_j := \begin{cases} 1 & j \in S \\ 0 & j \notin S. \end{cases} \quad \forall j = 1, \dots, n$$

The model of the Knapsack is the following:

$$\begin{cases} \max \sum_{j=1}^n p_j x_j \\ \sum_{j=1}^n w_j x_j \leq W \\ x_j \in \{0, 1\} \quad \forall j = 1, \dots, n \end{cases} \quad (6.1)$$

This is the easiest MIP problem: it has only one constraint.

How can we solve the Knapsack problem? We can use the B&B algorithm.

6.1.1 Solving the Knapsack B&B + Dantzig solution

Usually in the B&B we solve the linear relaxation associated to the problem of a certain node by using the simplex method. However for the Knapsack problem there exists a solution that is much more efficient and it was invented by Dantzig himself.

The idea: a solution $x_j = 0.5$ can be interpreted as selecting 50% of item j . This interpretation suggest an algorithm th solve the continuous relaxation of the Knapsack model.

Consider an item j and split it into w_j micro items of weight 1 and profit p_j/w_j .

Now assume that

$$p_1/w_1 \geq p_2/w_2 \geq \dots \geq p_n/w_n$$

So the procedure of the algorithm is simple:

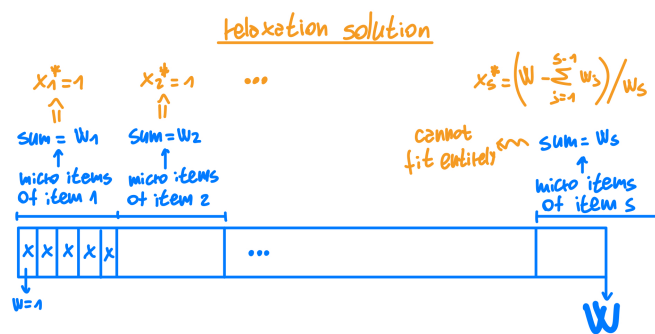


Figure 6.1: Relaxation solution: x_s^* is called critical item.

The solution provided: selects all the first $s - 1$ items; partially selects the critical items s ; all the items $s + 1, \dots, n$ are discarded.

Algorithm 11

```

1: Sort items (once at the root node)
2:  $tot_w \leftarrow 0$  ▷ Total weight of the selected items
3: for  $j \leftarrow 1$  to  $n$  do
4:    $tot_w \leftarrow tot_w + w_j$ 
5:   if  $tot_w > W$  then
6:      $s \leftarrow j$ ; Break ▷ The critical item
7:   end if
8: end for
9: return  $x_s^* \leftarrow \frac{W - (tot_w - w_s)}{w_s}$ 

```

The linear relaxation is implemented by the pseudocode above. In the B&B, the branching is applied to x_s^* . The complexity: $O(n \log n)$ at the root for the sorting, then the relaxation is solved in $O(n)$ time.

6.2 TSP

We have a directed graph $G = (V, A)$ where each arc $(i, j) \in A$ is associated to a cost c_{ij} . The Travelling Salesman Problem consists in finding an Hamiltonian circuit of minimum cost.

Let define

$$x_{ij} := \begin{cases} 1 & \text{if arc } (i, j) \text{ is chosen} \\ 0 & \text{otherwise} \end{cases} \quad \forall (i, j) \in A$$

The model is the following:

$$\left\{ \begin{array}{l} \min \sum_{(i,j) \in A} c_{ij} x_{ij} \\ \sum_{(i,j) \in \delta^+(h)} x_{ij} = 1 \quad \forall h \in V \\ \sum_{(i,j) \in \delta^-(h)} x_{ij} = 1 \quad \forall h \in V \\ \sum_{(i,j) \in \delta^+(S)} x_{ij} \geq 1 \quad \forall S \subset V, 1 \in S \\ x_{ij} \in \{0, 1\} \quad \forall (i, j) \in A \end{array} \right. \quad (6.2)$$

We have: the first constraint impose that each node has exactly one leaving arc; the second constraint impose that each node has exactly one entering arc; the third constraint are actually the class of constraint that in a previous chapter we have called *subtour elimination constraints*, that is, they are equivalent to write $\sum_{(i,j) \in A(S)} x_{ij} \leq |S| - 1 \quad \forall S \subset V$. Note that S must be a

proper subset of V (i.e. we cannot have $S = V$) because in the end we want a circuit.

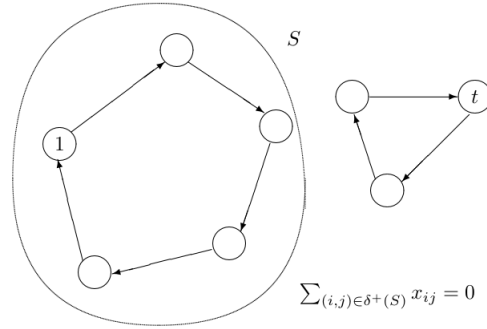


Figure 6.2: Feasible solution for the TSP if we do not specify the subtour elimination constraints.

How do we solve an instance of the TSP? We can use the B&C algorithm: the number of subtour elimination constraints grows exponentially in the size of the graph, so instead of adding all the constraints to the initial model, we can add by fly the ones that are violated by the current solution. Aka we use the B&C and we specify a separation problem:

- **Input:** a solution x^* of the linear relaxation.
- **Output:** a subset $\tilde{S} \subset V$ such that $1 \in \tilde{S}$ and $\sum_{(i,j) \in \delta^+(\tilde{S})} x_{ij} < 1$.

How can we find such $\tilde{S}$? If the solution x^* is integer, then we only have to use an algorithm that identifies the connected components of the graph associated to the solution:

- If the graph has exactly one connected component $\Rightarrow$ the solution x^* satisfies all the subtour elimination constraints.
- If the graph has more than one connected component, then we are in similar situation to the one shown in the figure above. Therefore the algorithm provide a subset $\tilde{S}$ that contains all the nodes that belong to the same connected component.

But what we do when x^* is fractional? If we think about it, we can draw the current solution as a network in which each arc (i, j) is associated to the capacity $k_{ij} := x_{ij}^*$, then we have:

$$\sum_{(i,j) \in \delta^+(\tilde{S})} k_{ij} := k(\tilde{S}).$$

Therefore we can identify the subset $\tilde{S}$ by solving the MAX-FLOW problem associated with the network: by solving the MAX-FLOW we get also the minimum capacity cut.

The only problem is that in the MAX-FLOW problem we have to fix the source node s and the terminal node t : s can always be fixed to node 1; instead t can be any other node. Therefore we have to solve the MAX-FLOW problem for every terminal node $t \in V \setminus \{1\}$.

If the solution of the MAX-FLOW provide $k(\tilde{S}) < 1$ than we return the set $\tilde{S}$. Otherwise, if $k(\tilde{S}) \geq 1$, then all the subtour constraints are satisfied, so we cannot add any constraint.

6.3 Steiner Tree Problem

We have: a directed graph $G = (V, A)$, a root node $r \in V$ (given), some terminal nodes $t \in T \subseteq V \setminus \{r\}$ and some Steiner nodes in $V \setminus (T \cup \{r\})$. The Steiner Tree Problem consists in identifying a partial arborescence of root r reaching at minimum cost all vertices of T , possibly using one or more Steiner nodes.

Define:

$$x_{ij} := \begin{cases} 1 & \text{if arc } (i, j) \in A \text{ is selected} \\ 0 & \text{otherwise} \end{cases} \quad \forall (i, j) \in A.$$

The model is the following:

$$\left\{ \begin{array}{l} \min \sum_{(i,j) \in A} c_{ij} x_{ij} \\ \sum_{(i,j) \in \delta^-(h)} x_{ij} = \begin{cases} 0 & h = r \\ 1 & h \in T \\ \leq 1 & h \in V \setminus (T \cup \{r\}) \end{cases} \\ \sum_{(i,j) \in \delta^+(S)} x_{ij} \geq \sum_{(i,j) \in \delta^-(t)} x_{ij} \quad \forall S \subset V, r \in S, \forall t \in V \setminus S \\ x_{ij} \in \{0, 1\} \quad \forall (i, j) \in A \end{array} \right. \quad (6.3)$$

where in the last constraint we have that $\sum_{(i,j) \in \delta^-(t)} x_{ij} = 0$ if t is isolated (a Steiner node can be isolated in the solution); $\sum_{(i,j) \in \delta^-(t)} x_{ij} = 1$ if t is either a terminal node or is a selected Steiner node and in this case the cut $\delta^+(S)$ must be crossed at least once.

As for the TSP, the Steiner Tree Problem can be solved by means of the B&C. Also we can use almost the same separation problem procedure that we have introduced for the TSP. The difference is that we need to provide $\tilde{S}$ and also $\tilde{t}$ such that $k(\tilde{S}) < \sum_{(i,j) \in \delta^-(\tilde{t})} x_{ij}$, where $\tilde{t}$ is the terminal node in $V \setminus \{s\}$ in the associated MAX-FLOW problem (the source is always the root,

so $s = r$; the terminal node $\tilde{t}$ can be any node). If we cannot find any pair $\tilde{S}, \tilde{t}$ then all constraints are satisfied.

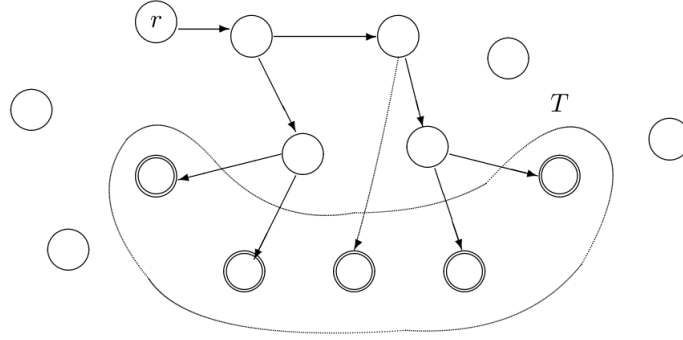


Figure 6.3: Steiner Tree Problem.

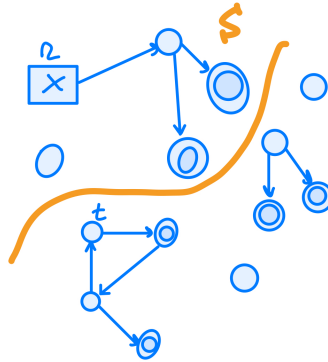


Figure 6.4: Feasible solution if we don't put the (exponential) class of constraints in the model.

6.4 Plant/Facilities Location Problem

Input of the problem: possible sites with activation cost associated $d_j \forall j = 1, \dots, m$ (fixed costs); customers with costs associated $c_{ij} \forall i = 1, \dots, n, \forall j = 1, \dots, m$ (proportional to the distance from customer i to site j) (connection costs). The goal is to minimize the cost (derived by the construction of the sites and the distance between the clients and the opened sites).

Let

$$y_j := \begin{cases} 1 & \text{if location } j \text{ is used} \\ 0 & \text{otherwise} \end{cases} \quad \forall j = 1, \dots, m$$

$$x_{ij} := \begin{cases} 1 & \text{if user } i \text{ use location } j \\ 0 & \text{otherwise} \end{cases} \quad \forall i = 1, \dots, n \quad \forall j = 1, \dots, m$$

The model is the following:

$$\begin{cases} \min \sum_{j=1}^m d_j y_j + \sum_{j=1}^m \sum_{i=1}^n c_{ij} x_{ij} \\ \sum_{j=1}^m x_{ij} = 1 & \forall i = 1, \dots, n \\ x_{ij} \leq y_j & \forall i = 1, \dots, n \quad \forall j = 1, \dots, m \\ x_{ij}, y_j \in \{0, 1\} & \forall i = 1, \dots, n \quad \forall j = 1, \dots, m \end{cases} \quad (6.4)$$

where the first constraint is used in order to ensure that each user is associated to a single location; the second constraint is used to model the logic implication " $x_{ij} = 1 \Rightarrow y_j = 1$ ", i.e., each user must be associated to an active location. Note that the model is very similar to the Index Selection Problem.

We can solve such problem using the B&C: the number of constraints $x_{ij} \leq y_j$ can be huge, therefore we can just avoid to write explicitly all the constraints and define a simple function that implements the following separation problem:

- **Input:** x^* .
- **Output:** x_{ij} such that $x_{ij} > y_j$.

This function can be easily implemented with two nested for loops (break the cycle when you find $x_{ij}^* > y_j^*$).

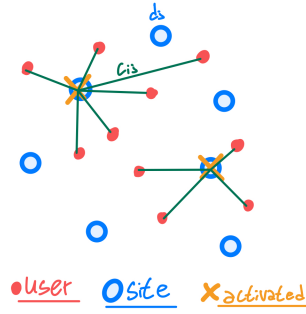


Figure 6.5

6.5 Set Covering/Partitioning Problem

The model of a Set Covering Problem is the following:

$$\begin{cases} \min c^T x \\ Ax \geq b \\ x_j \in \{0, 1\} \quad \forall j = 1, \dots, n \end{cases}$$

where $b = [1, \dots, 1] \in \mathbb{R}^m$ and $A \in \{0, 1\}^{m \times n}$ (is a matrix with only 0's and 1's). If A is TUM, then the problem is pretty easy, but in most of the practical cases A is not TUM. When we have $Ax = b$ then we're talking about Set Partitioning.

The goal is to cover all the items using the minimum number of sets. Let

$$x_j := \begin{cases} 1 & \text{if set } j \text{ is used} \\ 0 & \text{otherwise} \end{cases} \quad \forall j = 1, \dots, n$$

The constraints:

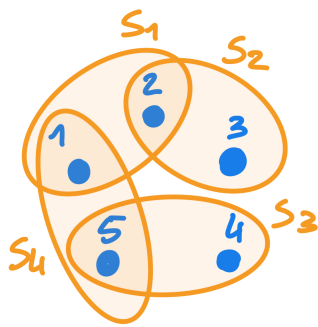
$$\sum_{j=1}^n a_{ij} x_j \geq 1 \quad \forall i = 1, \dots, m$$

it means that each items must be at least in one set.

If we're talking about Set Partitioning:

$$\sum_{j=1}^n a_{ij} x_j = 1 \quad \forall i = 1, \dots, m$$

it means that the sets must be mutually disjoint.



	S_1	S_2	S_3	S_4
1	1	0	0	1
2	1	1	0	0
3	0	1	0	0
4	0	0	1	0
5	0	0	1	1

Figure 6.6